

# Теория формальных языков и трансляций

Папшев Сергей Владимирович

2021

# Литература

1. Ахо А., Лам М., Сети Р., Ульман Дж. Компиляторы: принципы, технологии и инструментарий. – М.: Издательский дом «Вильямс», 2008. – 1185 с.
2. Ахо А., Ульман Дж. Теория синтаксического анализа, перевода и компиляции (Том 1, 2)
3. Пентус А.Е., Пентус М.Р. Математическая теория формальных языков. – М: Бином. Лаборатория знаний, 2006. – 248 с.
4. Миронов С.В. Языки программирования и методы трансляции
5. Волкова И.А., Вылиток А.А., Руденко Т.В. Формальные грамматики и языки. Элементы теории трансляции: уч. пособие для студентов II курса. – М.: Издательский отдел факультета ВМиК МГУ, 2009. – 115 с.
6. Ишакова Е.Н. Теория формальных языков, грамматик и автоматов: Методические указания к лабораторному практикуму. - Оренбург: ГОУ ОГУ, 2005. – 54 с.

# Введение в компиляторы

# Введение

- Языки высокого уровня



← Трансляторы (компиляторы)

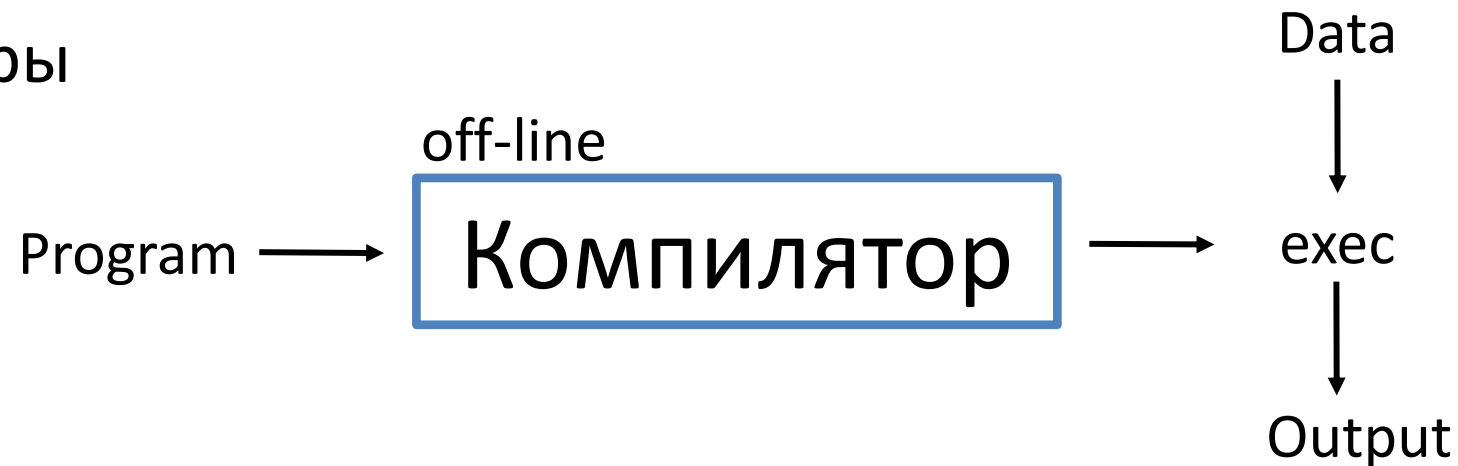
- Ассемблеры



- Машинные языки

# Введение

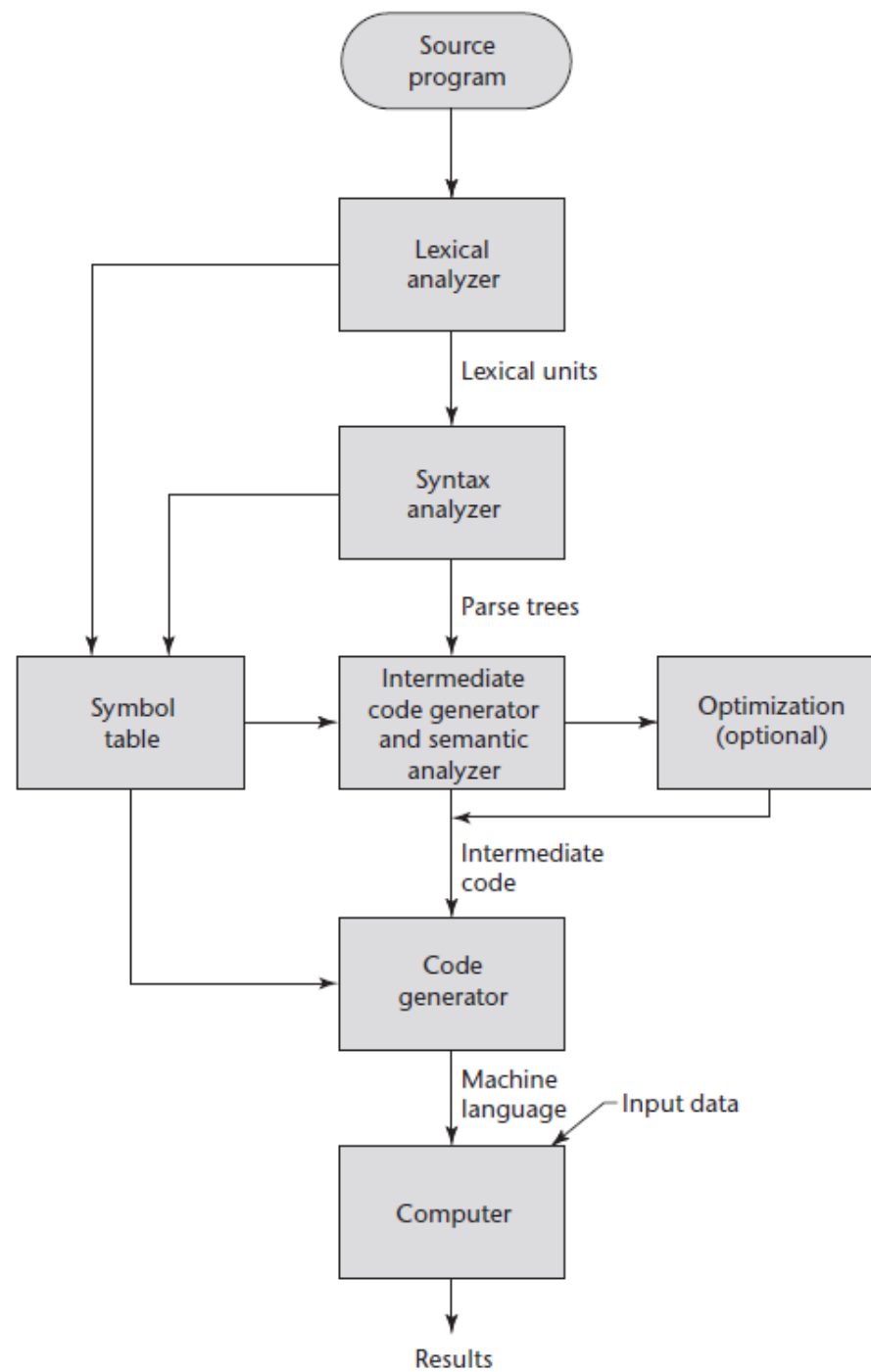
- Компиляторы



- Интерпретаторы

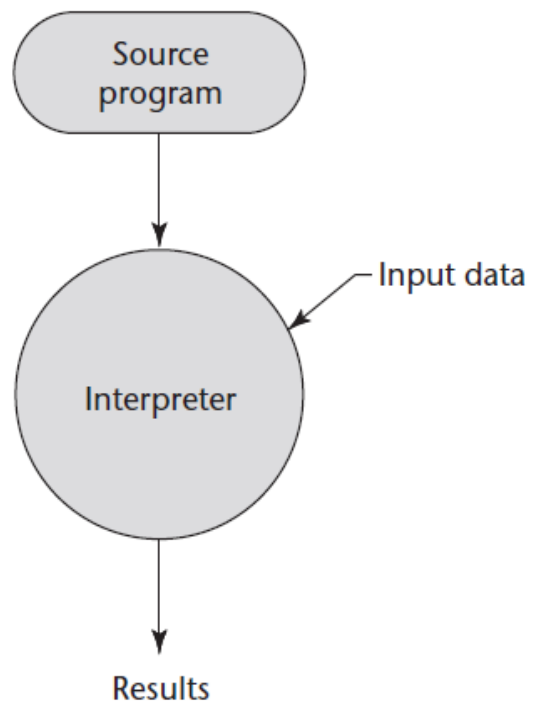


# Процесс КОМПИЛЯЦИИ

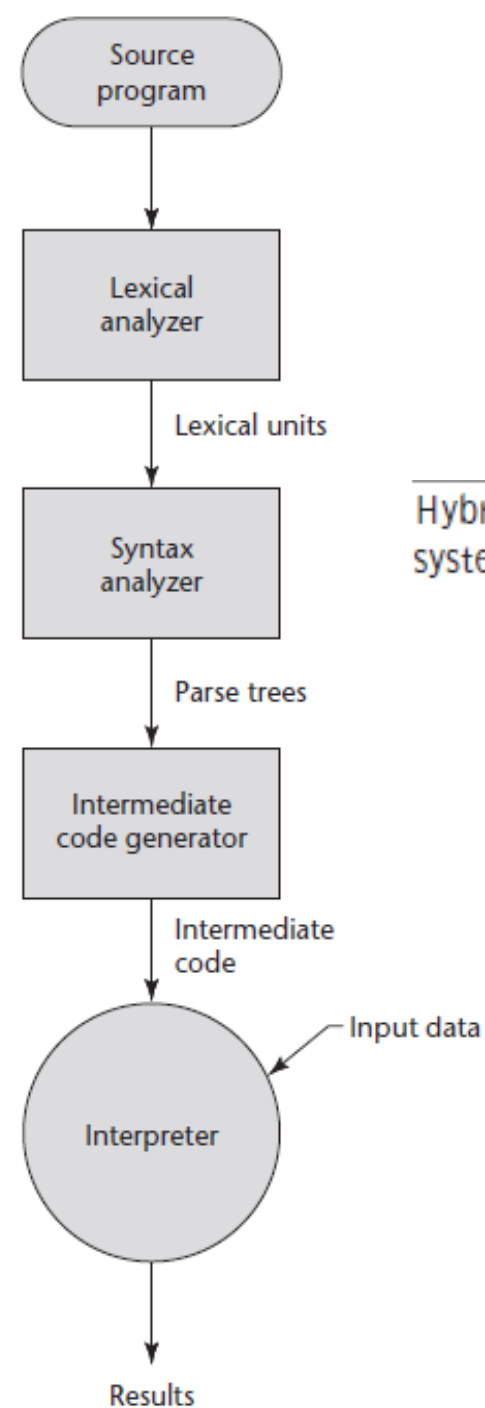


# Интерпретация

Pure Interpretation



Hybrid Implementation system



# Введение

- Fortran 1
- Formula Translation
- 1954-1957 гг
- 1958 – 50% программ на Фортране
- Первый компилятор – огромное воздействие на область Computer Science
- Большое число теоретических работ (теория)
- Практический задел: Современные компиляторы сохраняют основную структуру транслятора Фортрана



# Введение



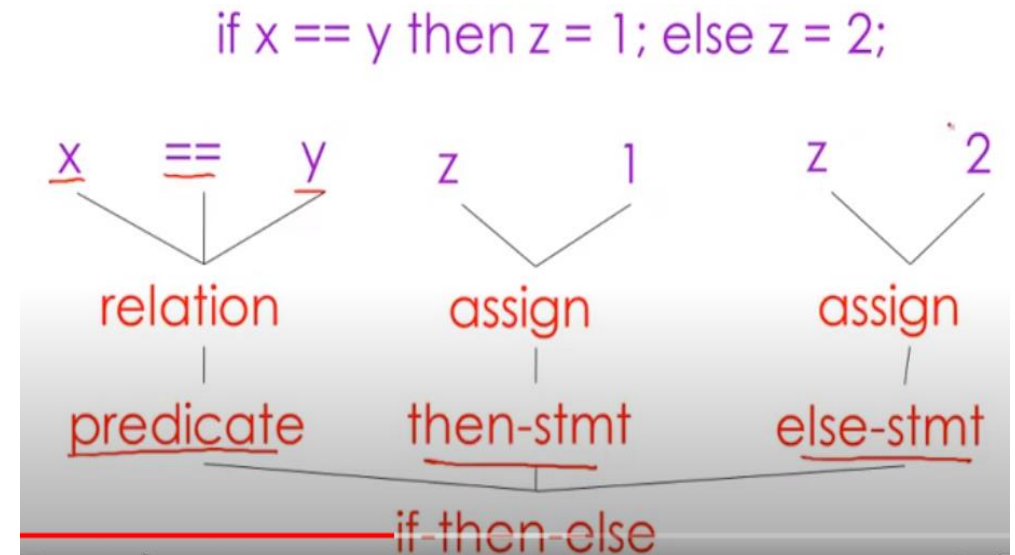
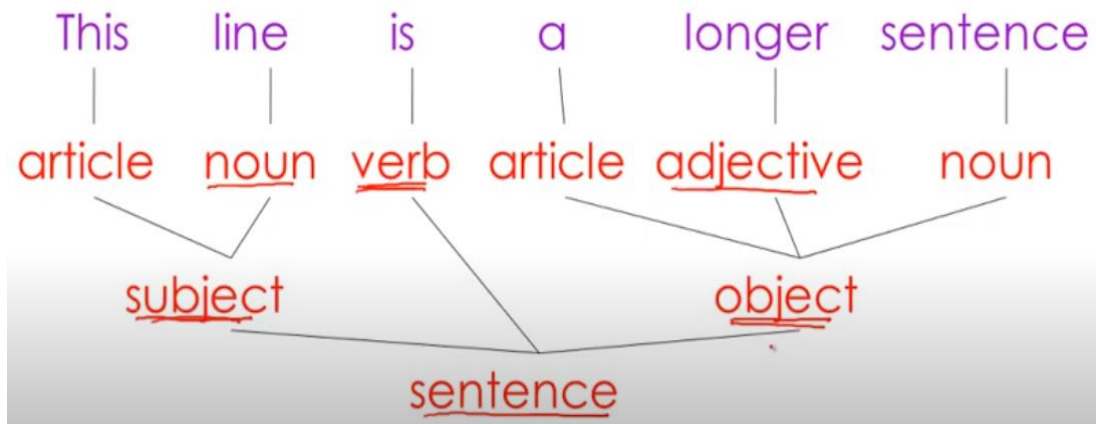
1. Lexical Analysis
  2. Parsing
  3. Semantic Analysis
  4. Optimization
  5. Code Generation
- Syntactic
- Types, scope
- Translation

# Введение

- 1. Лексический анализ – деление текста на слова, выделение «токенов»
- This is a sentence
- Thi sis ase nte nce
- if  $x==y$  then  $z=1$ ; else  $z=2$ ;

# Введение

- 2. Синтаксический анализ (parsing) – определение структуры предложения



# Введение

- 3. Семантический анализ (устранение неоднозначностей)
- Пример:
  - Петя сказал Мите оставить его задание дома
- Более сложный пример:
  - Петя сказал Пете оставить его задание дома

# Введение

- Языки программирования определяют строгие правила избегания двусмысленности

```
{  
    int Jack = 3;  
    {  
        int Jack =4;  
        cout << Jack;  
    }  
}
```

# Введение

- Компиляторы выполняют множество семантических проверок относительно переменных
- Пример:
  - Петя оставил её задание дома
- Из-за «несоответствия типов» между «её» и «Петя» мы узнаём, что это разные люди.

# Введение

- 4. Оптимизация кода
- На естественном языке оптимизация не имеет строгих правил и сводится к редактированию.
- Для программы: Автоматическая модификация, чтобы:
  - Работать быстрее
  - Использовать меньше памяти
  - (энергосбережение, сети, БД)
- $X=Y*0$
- $X=0$

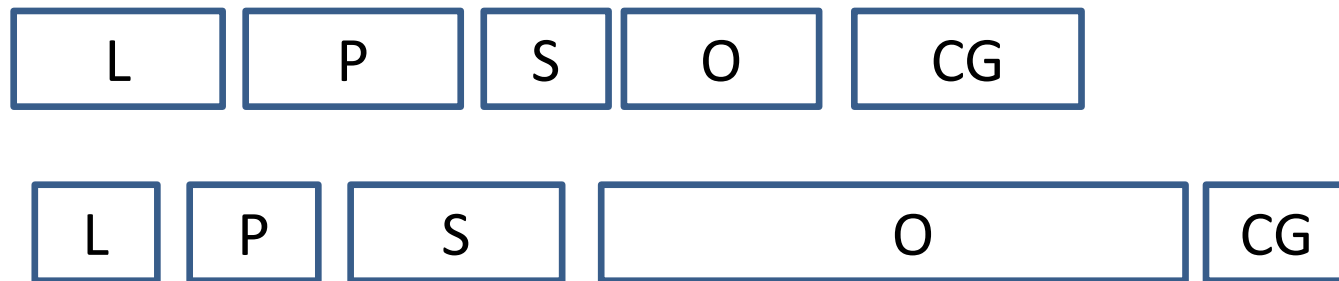
# Введение

- 5. Генерация кода
- Обычно результатом является ассемблерный код  
или
- Трансляция на другой язык программирования



# Введение

- Общая структура почти всех компиляторов соответствует нашему описанию
- Пропорции в объеме вычислений в процессе трансляции изменились:



# Введение

- Причины многообразия и многочисленности языков программирования
- Почему ЯП так много?
- Почему появляются новые ЯП?
- Какой ЯП является «хорошим»?

# Введение

- Почему ЯП так много?
- Прикладные области имеют различные/конфликтные требования
- Вычисления в науке
  - Хорошие вычисления FP
  - Эффективная работа с массивами
  - Параллелизм
- Бизнес-приложения
  - Надежность
  - Генерация отчетов
  - Анализ данных
- Системное программирование
  - Контроль ресурсов
  - Real-time ограничения



FORTRAN



SQL



C/C++

# Введение

- Почему появляются новые ЯП?
- Тезис: Обучение программистов является доминирующим в стоимости языка программирования
- 1. Широко распространенные ЯП медленно меняются
- 2. Легко начать разработку нового языка



Продуктивность > стоимости обучения

- Новые ЯП заполняют пустоту
- Новые ЯП имеют тенденцию выглядеть как старые:



Java vs C/C++

# Введение

- Какой ЯП является «хорошим»?
- Нет общепризнанной метрики для оценки дизайна ЯП
- «Хороший язык - это тот, который используют много людей» ?
- VB – лучший ЯП?

# Введение

- Прикладные области имеют конфликтные требования к ЯП
- Трудно разработать одну систему для всего
- Обучение программистов является доминирующим в стоимости языка программирования

# Лексический анализ

- Просмотр потока символов программы (слева направо) и выделение лексических единиц – токенов

`position := initial + rate * 60`

`position` – идентификатор

`:=` - операция

`initial` - идентификатор

`+` - операция

`rate` - идентификатор

`*` - операция

`60` - число (целое)

# Лексический анализ

```
if (i == j)  
    Z = 0;  
else  
    Z = 1;
```

```
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;
```

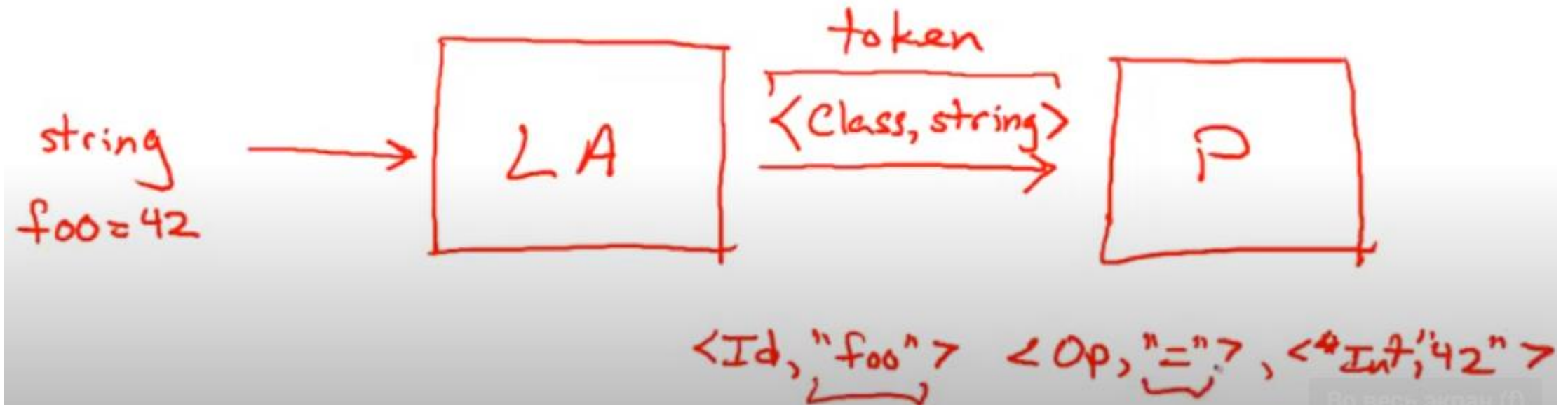


# Лексический анализ

- Классам токенов соответствуют множества строк
- Примеры:
- Идентификатор ::= строка букв и цифр, начинающаяся с буквы
- Целое число ::= непустая строка цифр
- Ключевое слово ::= < if | then | else | while | for | ... >
- Пробел ::= непустая последовательность кодов символов для пробела новой строки, табуляции

# Лексический анализ

- Задача ЛА – выделить токены (лексемы), классифицировать их и передать на стадию синтаксического анализа



# Лексический анализ

```
if (i == j)
    Z = 0;
else
    Z = 1;
```

# Лексический анализ

```
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;
```

- Оператор
- Пробел
- Идентификатор
- Ключевое слово
- Число
- Спец. символы

```
|      if (i==j)
|              z=0;
|      else
|              z=1;
```

# Лексический анализ

- Примеры
- FORTRAN – пробелы не учитываются
- VAR1 – то же самое, что VA R1
- DO 5 i=1,25
- DO 5 i=1.25
- Необходимо заглядывать при просмотре строки вперёд

# Лексический анализ

- PL/1 – нет зарезервированных ключевых слов

IF ELSE THEN THEN = ELSE; ELSE ELSE = THEN

DECLARE (ARG1 .. . ., ARGN) =

# Лексический анализ

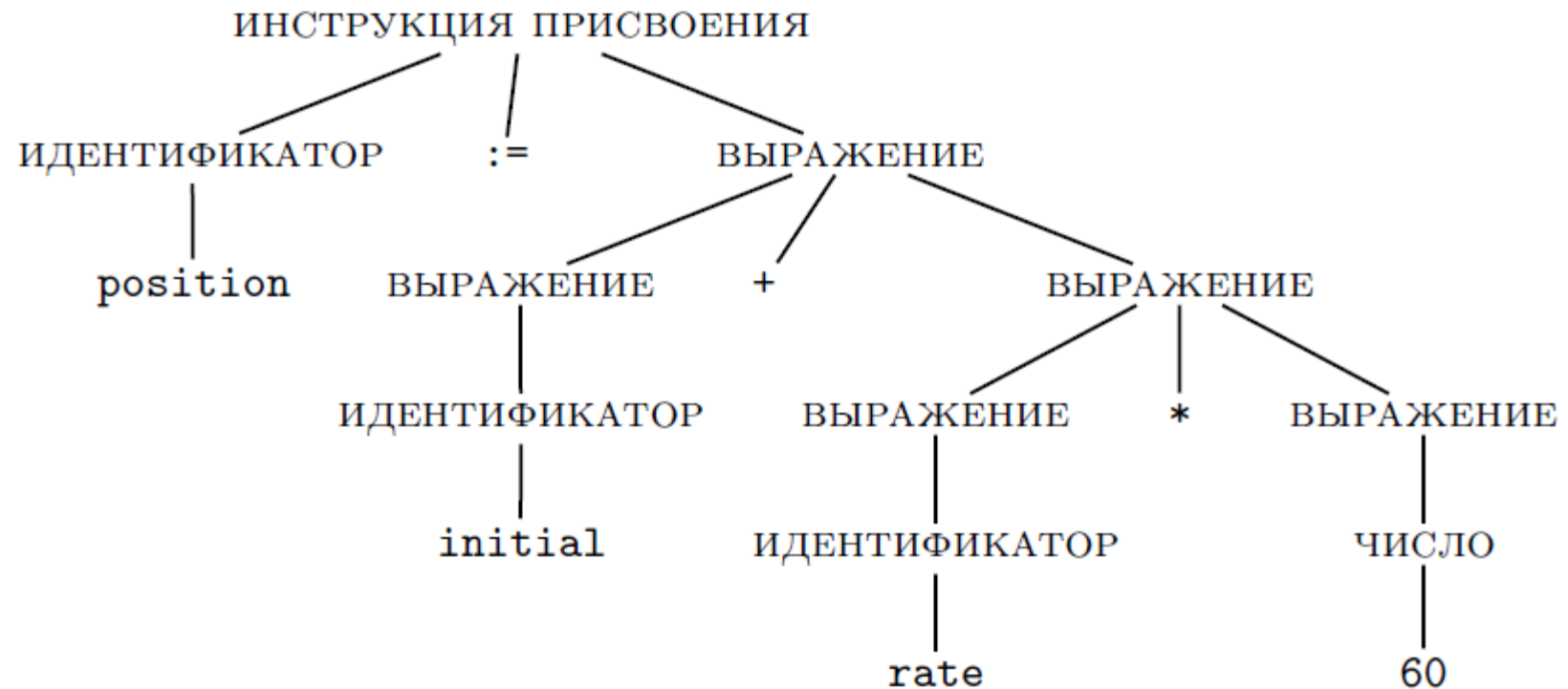
- C++
- Синтакс :
  - Foo<Bar>
  - cin>>Bar
- Foo<Bar<Bazz>>

# Синтаксический анализ

- Синтаксический разбор, parsing

position := initial + rate \* 60

- Задача: иерархическая группировка в соответствии с грамматикой ЯП





# Синтаксический анализ

Иерархическая структура программы обычно выражается рекурсивными правилами.

1. Любой *идентификатор* есть *выражение*.
2. Любое *число* есть *выражение*.
3. Если  $expression_1$  и  $expression_2$  являются выражениями, то выражениями являются и
  - (a)  $expression_1 + expression_2$
  - (b)  $expression_1 * expression_2$
  - (c)  $(expression_1)$ .

# Семантический анализ

Семантический анализ позволяет проверить, насколько корректно совместное размещение компонентов программы.



# Генерация промежуточного кода

- Программа для абстрактной машины

Load A; — загрузка значения переменной A в сумматор;

Load \$i; — загрузка значения i-го регистра в сумматор;

Load =C; — загрузка константы C в сумматор;

Store A; — сохранение значения сумматора в переменной A;

Store \$i; — сохранение значения сумматора в i-ом регистре;

Add A; — увеличение значения в сумматоре на величину переменной A;

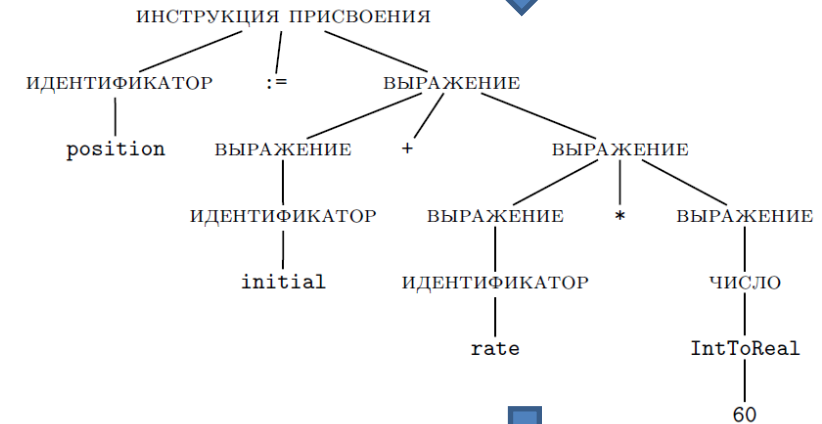
Add \$i; — увеличение значения в сумматоре на величину i-го регистра;

Add =C; — увеличение значения в сумматоре на величину константы C;

Mul A; — умножение значения в сумматоре на величину переменной A;

Mul \$i; — умножение значения в сумматоре на величину i-го регистра;

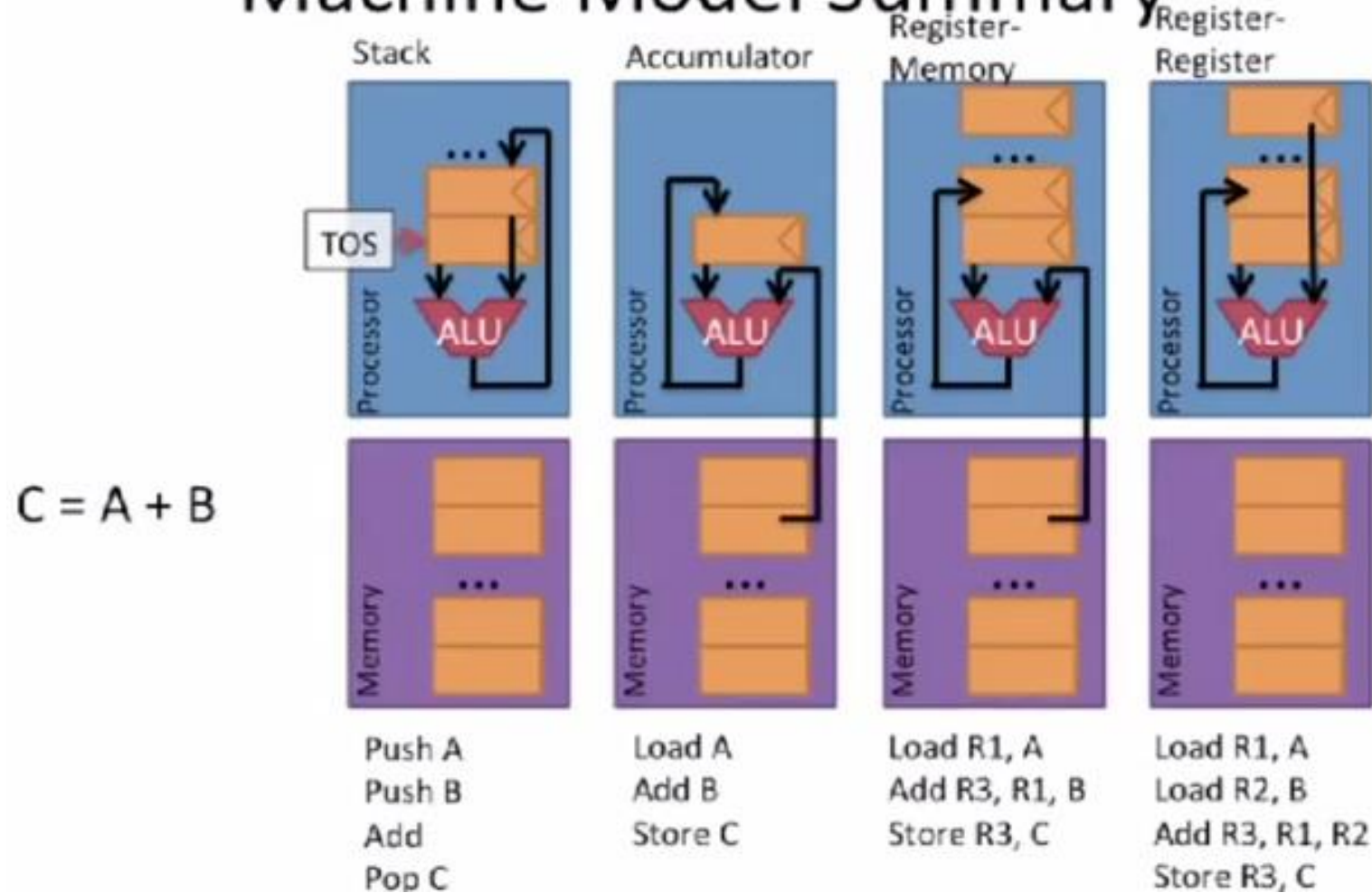
Mul =C; — умножение значения в сумматоре на величину константы C.



Load initial;  
Store \$2;  
Load rate;  
Store \$1;  
Load =60;  
Mul \$1;  
Add \$2;  
Store position;

# Зависимость от архитектуры компьютера

## Machine Model Summary



# Оптимизация кода

- Оптимизация использования регистров (ресурсов), (м.б. сокращение длины кода)

```
Load initial;  
Store $2;  
Load rate;  
Store $1;  
Load =60;  
Mul $1;  
Add $2;  
Store position;
```



```
Load rate;  
Mul =60;  
Add initial;  
Store position;
```

# Генерация целевого кода

- Генерация ассемблерного кода
- Привязка переменных к конкретным регистрам и ячейкам памяти

```
Load  rate;  
Mul   =60;  
Add   initial;  
Store position;
```



```
MOVF rate, R2  
MULF #60.0, R2  
MOVF initial, R1  
ADDF R2, R1  
MOVF R1, position
```

Формальные языки

# Основные понятия и определения формальных языков

- **Опр. *Алфавит*** — это конечное множество символов.
- **Опр. *Цепочкой символов (или словом) в алфавите*  $\Sigma$**  называется любая конечная последовательность символов этого алфавита.
- **Опр.** Цепочка, которая не содержит ни одного символа, называется *пустой цепочкой*. Обозначение:  $\varepsilon$  (в алфавит  $\Sigma$  не входит, она лишь помогает обозначить пустую последовательность символов).
- **Опр.** Если  $a$  и  $b$  — цепочки, то цепочка  $ab$  (результат приписывания цепочки  $b$  в конец цепочки  $a$ ), называется *конкатенацией* (или *сцеплением*) цепочек  $a$  и  $b$ . Конкатенацию можно считать двуместной операцией над цепочками:  $a \times b = ab$ .  
Например, если  $w = ab$  и  $z = cd$ , то  $w \times z = abcd$ .  
Для любой цепочки  $a$  :  $a\varepsilon = \varepsilon a = a$ .
- Для любых цепочек  $a, b, g$  справедливо свойство ассоциативности операции конкатенации  $(ab)g = a(bg) = abg$ .



# Операции над цепочкам символов

- **Опр.** *Обращением* (или *реверсом*) цепочки  $\alpha$  называется цепочка, символы которой записаны в обратном порядке.

Обращение цепочки  $\alpha$  будем обозначать  $\alpha^R$ .

Например, если  $\alpha = abcdef$ , то  $\alpha^R = fedcba$ .

Для пустой цепочки:  $\varepsilon^R = \varepsilon$ .

- **Опр.**  $n$ -ой степенью цепочки  $\alpha$  (будем обозначать  $\alpha^n$ ) называется конкатенация  $n$  цепочек  $\alpha$ :

$$\alpha^n = \underbrace{\alpha\alpha \dots \alpha\alpha\alpha}_n.$$

Свойства степени:  $\alpha^0 = \varepsilon$ ;  $\alpha^n = \alpha\alpha^{n-1} = \alpha^{n-1}\alpha$

- **Опр.** *Длина цепочки* — это число составляющих ее символов (или длина последовательности символов).

Длину цепочки  $a$  будем обозначать  $|a|$ .

Например, если  $a = abbcaad$ , то  $|a| = 7$ .

Длина  $\varepsilon$  равна 0.

# Операции над цепочкам символов

Для любой цепочки  $\gamma$  справедливы равенства

$$\varepsilon\gamma = \gamma\varepsilon = \gamma.$$

*Определение 26 (Подцепочка).* Пусть  $\alpha$  — цепочка, представимая в виде  $\alpha = \beta\gamma\varphi$ , где  $\beta$ ,  $\gamma$ ,  $\varphi$  — некоторые цепочки. Тогда цепочки  $\beta$ ,  $\gamma$  и  $\varphi$  называют подцепочками цепочки  $\alpha$ . Подцепочку  $\beta$  называют префиксом цепочки  $\alpha$ , а  $\varphi$  называют суффиксом цепочки  $\alpha$ . Если  $\beta \neq \alpha$ , то  $\beta$  называют собственным префиксом. Если  $\varphi \neq \alpha$ , то  $\varphi$  называют собственным суффиксом.

*Определение 27 (Степень алфавита).* Пусть  $\Sigma$  — некоторый алфавит. Определим  $\Sigma^k$  как множество всевозможных цепочек длины  $k$ , состоящих из символов алфавита  $\Sigma$ .

$\Sigma^0 = \{\varepsilon\}$  независимо от алфавита  $\Sigma$ ,

# Формальный язык

- **Опр.** Обозначим через  $\Sigma^*$  множество, содержащее все цепочки в алфавите  $\Sigma$ , включая пустую цепочку  $\varepsilon$ .  
Например, если  $\Sigma = \{0, 1\}$ , то  $\Sigma^* = \{\varepsilon, 0, 1, 00, 11, 01, 10, 000, 001, 011, \dots\}$ .
- **Опр.** Обозначим через  $\Sigma^+$  множество, содержащее все цепочки в алфавите  $\Sigma$ , исключая пустую цепочку  $\varepsilon$ .  
Следовательно,  $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ .
- **Опр.** *Язык* в алфавите  $\Sigma$  — это подмножество множества всех цепочек в этом алфавите. Для любого языка  $L$  справедливо  $L \subseteq \Sigma^*$ .

# Примеры

1.  $L = \{\varepsilon, 01, 0011, 000111, 00001111, \dots\}$  - язык над алфавитом  $\{0, 1\}$ , в котором цепочки состоят из последовательностей  $k$  единиц вслед за  $k$  нулями ( $k \geq 0$ ).
2.  $L = \{\varepsilon\}$ . Язык в любом алфавите, состоящий только из пустого слова
3.  $L = \emptyset$  Пустой язык в любом алфавите.
4.  $L = \{\varepsilon, 01, 10, 0011, 1100, 1001, 0110, 0101, \dots\}$  язык над алфавитом  $\{0, 1\}$ , в котором равное число нулей и единиц.

# Операции над языками

- **Опр.** *Конкатенацией* двух языков  $L_1$  и  $L_2$  называется язык

$$L_3 = \{\alpha\beta \mid \alpha \in L_1, \beta \in L_2\}.$$

$$\text{Обозн.: } L_3 = L_1 L_2$$

**Пример:**  $\{01, 111, 10\} \{00, 01\} = \{0100, 0101, 11100, 11101, 1000, 1001\}.$

- **Опр.** *Объединением* двух языков  $L_1$  и  $L_2$  называется язык

$$L = L_1 \cup L_2 = \{\alpha \mid \alpha \in L_1 \text{ или } \alpha \in L_2\}.$$

**Пример:**  $\{01, 111, 10\} \cup \{00, 01\} = \{01, 111, 10, 00\}.$

# Операции над языками

- **Опр.** *Степень* языка  $L$ :

1.  $L^0 = \{\varepsilon\}$

2.  $L^1 = L$

3.  $L^k = L^{k-1}L$

- **Опр.** *Итерация* языка  $L$ :

$$L^* = \bigcup_{k=0}^{\infty} L^k \qquad L^+ = \bigcup_{k=1}^{\infty} L^k \qquad L^+ = L^*L$$

$$L^* = \{\varepsilon\} \cup L \cup LL \cup LLL \cup \dots$$

**Пример:**  $\{0,10\}^* = \{\varepsilon, 0, 10, 00, 010, 100, 1010, \dots\}$

# Способы описания языков

- Конечный язык можно описать простым перечислением его цепочек.
- Как представлять бесконечные языки?
  - спецификация (описание)
  - механизм распознавания
  - механизм порождения (генерации).
- Не каждый формальный язык можно задать с помощью конечного описания.

- *Спецификация* – Описание языка, как множества слов, удовлетворяющих некоторому условию. (Для регулярных языков – это регулярное выражение\_.
- Механизм распознавания (*распознаватель*), по сути, является процедурой специального вида, которая по заданной цепочке определяет, принадлежит ли она языку.
- Если принадлежит, то процедура останавливается с ответом «да», т. е. *допускает* цепочку; иначе — останавливается с ответом «нет» или за циклируется.
- *Язык, определяемый распознавателем* — это множество всех цепочек, которые он допускает.
- Основной способ реализации механизма порождения — использование *порождающих грамматик*



# Регулярные выражения

- **Опр.** Пусть  $\Sigma$  — алфавит, не содержащий символов  $*$ ,  $+$ ,  $\varepsilon$ ,  $\emptyset$ ,  $($ ,  $)$ .

Определим рекурсивно *регулярное выражение*  $\gamma$  над алфавитом  $\Sigma$  и регулярный язык  $L(\gamma)$ , задаваемый этим выражением:

*Базис индукции:*

- 1)  $a \in \Sigma$  есть регулярное выражение;  $L(a) = \{a\}$
- 2)  $\varepsilon$  есть регулярное выражение;  $L(\varepsilon) = \{\varepsilon\}$
- 3)  $\emptyset$  есть регулярное выражение;  $L(\emptyset) = \emptyset$

*Индукция:*

Если  $\alpha$  и  $\beta$  — регулярные выражения, то:

- 1)  $(\alpha) + (\beta)$  — регулярное выражение;  $L((\alpha) + (\beta)) = L(\alpha) \cup L(\beta)$ ;
- 2)  $(\alpha)(\beta)$  — регулярное выражение;  $L((\alpha)(\beta)) = L(\alpha)L(\beta)$ ;
- 3)  $(\beta)^*$  — регулярное выражение;  $L((\beta)^*) = (L(\beta))^*$ ;

Никаких других регулярных выражений, кроме тех, что построены в соответствии с описанным определением, нет.

# Приоритет операций

- Если считать, что операция «+» имеет самый низкий приоритет, а операция «\*» — наивысший, то в регулярных выражениях можно опускать лишние скобки.

# Примеры RE

- $\Sigma = \{0, 1\}$
- $L(\mathbf{01}) = \{01\}$ .
- $L(\mathbf{01+0}) = \{01, 0\}$ .
- $L(\mathbf{0(1+0)}) = \{01, 00\}$ .
- $L(\mathbf{0^*}) = \{\epsilon, 0, 00, 000, \dots\}$ .
- $L((\mathbf{0+10})^*(\epsilon+\mathbf{1})) =$  все строки из 0 и 1 без двух последовательных 1.

# Примеры RE

**Примеры** регулярных выражений над алфавитом  $\{a, b\}$ :

$$a + b, (a + b)^*, \quad (aa(ab)^*bb)^*.$$

Соответствующие языки:

$$L(a + b) = \{a\} \cup \{b\} = \{a, b\},$$

$$L((a + b)^*) = \{a, b\}^*,$$

$$L((aa(ab)^*bb)^*) = \{\varepsilon\} \cup \{ aax_1bbaax_2bb \dots x_kbb \mid k \geq 1, \\ x_i \in \{(ab)^n \mid n \geq 0\} \text{ для } i = 1, \dots, k \}.$$

# Алгебраические законы для регулярных выражений

- Объединение и конкатенация ведут себя как сложение и умножение.
- $+$  является коммутативным и ассоциативным;
- Конкатенация является ассоциативной операцией.
- Конкатенация распределяется по  $+$ .
- Исключение: Конкатенация не коммутативна.

# Грамматики

**Опр.** *Порождающая грамматика*  $G$  — это четверка  $\langle T, N, P, S \rangle$ ,

где

$T$  — алфавит терминальных символов (терминалов);

$N$  — алфавит нетерминальных символов (нетерминалов),  $T \cap N = \emptyset$ ;

$P$  — конечное подмножество множества  $(T \cup N)^+ \times (T \cup N)^*$ ;

где элемент  $(\alpha, \beta)$  записывается в виде  $\alpha \rightarrow \beta$  и называется *правилом вывода*;

$\alpha$  называется *левой частью* правила,  $\beta$  — *правой частью* правила.

Левая часть любого правила из  $P$  обязана содержать хотя бы один нетерминал;

$S$  — начальный символ грамматики,  $S \in N$ .

Для записи правил вывода с одинаковыми левыми частями

$$\alpha \rightarrow \beta_1 \quad \alpha \rightarrow \beta_2 \quad \dots \quad \alpha \rightarrow \beta_n$$

используют сокращенную запись  $\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ .

# Пример

$$G_{example} = \langle \{0, 1\}, \{A, S\}, P, S \rangle$$

$$\begin{array}{lcl} P: & S & \rightarrow 0A1 \\ & 0A & \rightarrow 0A1 \\ & A & \rightarrow \varepsilon \end{array}$$

# Выводимость цепочки

**Опр.** Цепочка  $\beta \in (T \cup N)^*$  *непосредственно выводима* из цепочки  $\alpha \in (T \cup N)^+$  в грамматике  $G = \langle T, N, P, S \rangle$  (обозначается  $\alpha \rightarrow_G \beta$ ), если

$$\alpha = \xi_1 \gamma \xi_2, \beta = \xi_1 \delta \xi_2,$$

где

$$\xi_1, \xi_2, \delta \in (T \cup N)^*, \gamma \in (T \cup N)^+$$

и правило вывода  $\gamma \rightarrow \delta$  содержится в  $P$ .



$$G_{example} = \langle \{0, 1\}, \{A, S\}, P, S \rangle$$

$P$ :

$$\begin{array}{lll} S & \rightarrow & 0A1 \\ 0A & \rightarrow & 00A1 \\ A & \rightarrow & \varepsilon \end{array}$$

Цепочка 00A11 *непосредственно выводима* из 0A1 в грамматике

$$G_{example} : \underline{0A1} \rightarrow \underline{00A11}.$$

$S \Rightarrow 000A111$  в грамматике  $G_{example}$ , т.к. существует вывод

$$S \rightarrow 0A1 \rightarrow 00A11 \rightarrow 000A111.$$

Длина вывода равна 3

# Выводимость цепочки

**Опр.** Цепочка  $\beta \in (T \cup N)^*$  *выводима* из цепочки  $\alpha \in (T \cup N)^+$  в грамматике  $G = \langle T, N, P, S \rangle$  (обозначается), если существуют цепочки  $\gamma_0, \gamma_1, \dots, \gamma_n$  ( $n \geq 0$ ), такие, что

$$\alpha = \gamma_0 \rightarrow \gamma_1 \rightarrow \dots \rightarrow \gamma_n = \beta.$$

Последовательность  $\gamma_0, \gamma_1, \dots, \gamma_n$  называется *выводом длины  $n$* .  
Длину вывода  $n$  показывают обозначением:  
Вывод за некоторое число шагов (м.б. 0 шагов) обозначается:

Индекс  $G$  в обозначении  $\Rightarrow_G$  опускают, если понятно, какая грамматика подразумевается.

# Язык грамматики

**Опр.** *Языком, порождаемым грамматикой*  $G = \langle T, N, P, S \rangle$ , называется множество

$$L(G) = \{\alpha \in T^* \mid S \Rightarrow \alpha\}.$$

Другими словами,  $L(G)$  — это все цепочки в алфавите  $T$ , которые выводимы из  $S$  с помощью правил  $P$ .

Например,  $L(G_{example}) = \{0^n 1^n \mid n > 0\}$ .

**Опр.** Цепочка  $\alpha \in (T \cup N)^*$ , для которой  $S \Rightarrow \alpha$ , называется *сентенциальной формой* в грамматике  $G = \langle T, N, P, S \rangle$ .

Таким образом, *язык, порождаемый грамматикой*, можно определить как *множество терминальных сентенциальных форм*.

**Опр.** Грамматики  $G_1$  и  $G_2$  называются *эквивалентными*, если  $L(G_1) = L(G_2)$ .

*Пример.* Грамматики  $G_1 = \langle \{0,1\}, \{A,S\}, P_1, S \rangle$  и  $G_2 = \langle \{0,1\}, \{S\}, P_2, S \rangle$  с правилами

|        |               |               |
|--------|---------------|---------------|
| $P_1:$ |               |               |
| $S$    | $\rightarrow$ | $0A1$         |
| $0A$   | $\rightarrow$ | $00A1$        |
| $A$    | $\rightarrow$ | $\varepsilon$ |

|        |               |               |
|--------|---------------|---------------|
| $P_2:$ |               |               |
| $S$    | $\rightarrow$ | $0S1 \mid 01$ |

эквивалентны, т. к. обе порождают язык  $L(G_1) = L(G_2) = \{0^n 1^n \mid n > 0\}$ .

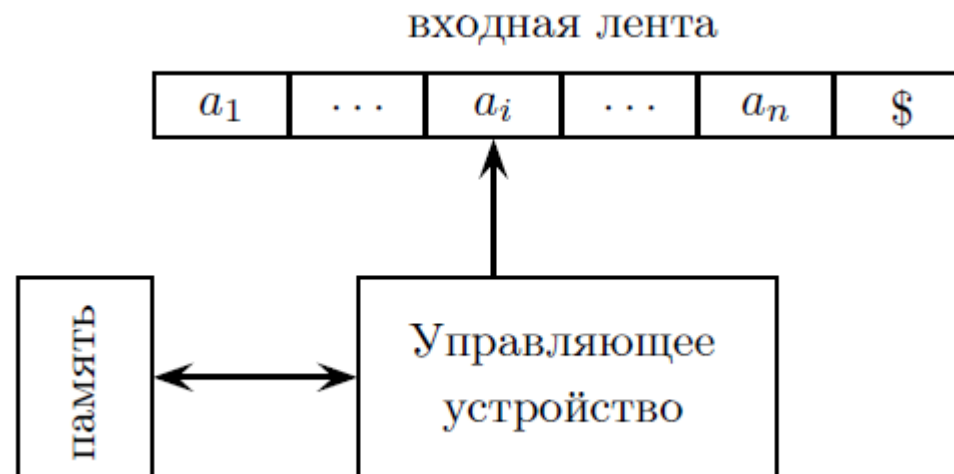
**Опр.** Грамматики  $G_1$  и  $G_2$  *почти эквивалентны*, если

$$L(G_1) \cup \{\varepsilon\} = L(G_2) \cup \{\varepsilon\}.$$

Т.е., грамматики почти эквивалентны, если языки, ими порождаемые, отличаются не более, чем на  $\varepsilon$ .

# Распознаватель

- Механизм, который является процедурой специального вида, которая по заданной цепочке определяет, принадлежит ли она языку.
- Если принадлежит, то процедура останавливается с ответом «да», т. е. *допускает* цепочку; иначе — останавливается с ответом «нет» или за циклируется.
- *Язык, определяемый распознавателем* — это множество всех цепочек, которые он допускает.



# Классификация грамматик и языков по Хомскому

- Тип грамматики определяется типом ограничений на вид правил вывода.
- Всего определено четыре типа грамматик:  
тип 0, тип 1, тип 2, тип 3.
- Каждому типу грамматик соответствует свой класс языков.
- Если язык порождается грамматикой типа  $i$  (для  $i = 0, 1, 2, 3$ ), то он является *языком типа  $i$* .

# Классификация грамматик и языков по Хомскому: Тип 0

## Тип 0

Любая порождающая грамматика является грамматикой типа 0.

На вид правил грамматик этого типа не накладывается никаких дополнительных ограничений.

Класс языков типа 0 совпадает с классом рекурсивно перечислимых языков.

# Классификация грамматик и языков по Хомскому: Тип 1

**Опр.** Грамматика  $G = \langle T, N, P, S \rangle$  называется *неукорачивающей*, если правая часть каждого правила из  $P$  не короче левой части (т. е. для любого правила  $\alpha \rightarrow \beta \in P$  выполняется неравенство  $|\alpha| \leq |\beta|$ ).

В виде исключения в неукорачивающей грамматике допускается наличие правила  $S \rightarrow \varepsilon$ , при условии, что  $S$  (начальный символ) не встречается в правых частях правил.

Грамматикой *типа 1* называют неукорачивающую грамматику.



# Классификация грамматик и языков по Хомскому: Тип 1

Другое определение:

**Опр.** Грамматика  $G = \langle T, N, P, S \rangle$  называется *контекстно-зависимой (КЗ)*, если каждое правило из  $P$  имеет вид  $\alpha \rightarrow \beta$ ,

где  $\alpha = \xi_1 A \xi_2$ ,  $\beta = \xi_1 \gamma \xi_2$ ,  $A \in N$ ,  $\gamma \in (T \cup N)^+$ ,  $\xi_1, \xi_2 \in (T \cup N)^*$ .

В виде исключения в КЗ-грамматике допускается наличие правила  $S \rightarrow \varepsilon$ , при условии, что  $S$  (начальный символ) не встречается в правых частях правил.

Язык, порождаемый контекстно-зависимой грамматикой, называется *контекстно-зависимым языком*.

# Эквивалентность неукорачивающих и КЗ-грамматик

**Утверждение 1.** Пусть  $L$  — формальный язык. Следующие утверждения эквивалентны.

- 1) существует контекстно-зависимая грамматика  $G_1$ , такая что  $L = L(G_1)$ ;
- 2) существует неукорачивающая грамматика  $G_2$ , такая что  $L = L(G_2)$ .

**Док-во.** Очевидно, что  $(1) \Rightarrow (2)$ : любая контекстно-зависимая грамматика удовлетворяет ограничениям неукорачивающей грамматики (см. определения).

Т.к. каждое неукорачивающее правило можно заменить эквивалентной серией контекстно-зависимых правил, следовательно  $(2) \Rightarrow (1)$ .

Т.о., язык, порождаемый неукорачивающей грамматикой, - контекстно-зависимый. ■

Т.е., неукорачивающие и КЗ-грамматики определяют один и тот же класс языков.

# Классификация грамматик и языков по Хомскому: Тип 2

**Опр.** Грамматика  $G = \langle T, N, P, S \rangle$  называется **контекстно-свободной** (КС), если каждое правило из  $P$  имеет вид  $A \rightarrow \beta$ , где  $A \in N$ ,  $\beta \in (T \cup N)^*$ .

Заметим, что в КС-грамматиках допускаются правила с пустыми правыми частями. Язык, порождаемый контекстно-свободной грамматикой, называется **контекстно-свободным** языком.

Грамматикой **типа 2** будем называть контекстно-свободную грамматику.

КС-грамматика может являться неукорачивающей, т.е. удовлетворять ограничениям неукорачивающей грамматики.

**Утверждение 2.** Для любой КС-грамматики  $G$  существует неукорачивающая КС-грамматика  $G'$ , такая что  $L(G) = L(G')$ .

# Классификация грамматик и языков по Хомскому: Тип 3

**Опр.** Грамматика  $G = \langle T, N, P, S \rangle$  называется **праволинейной**, если каждое правило из  $P$  имеет вид  $A \rightarrow wB$  либо  $A \rightarrow w$ , где  $A, B \in N$ ,  $w \in T^*$ .

**Опр.** Грамматика  $G = \langle T, N, P, S \rangle$  называется **леволинейной**, если каждое правило из  $P$  имеет вид  $A \rightarrow Bw$  либо  $A \rightarrow w$ , где  $A, B \in N$ ,  $w \in T^*$ .

**Утверждение 3.** Пусть  $L$  — формальный язык. Следующие два утверждения эквивалентны:

- 1) существует праволинейная грамматика  $G_1$ , такая что  $L = L(G_1)$ ;
- 2) существует леволинейная грамматика  $G_2$ , такая что  $L = L(G_2)$ .

Т.е., праволинейные и леволинейные грамматики определяют один и тот же класс языков. Такие языки называются **регулярными**.

- Право- и леволинейные грамматики также называют регулярными.
- Регулярная грамматика является грамматикой **типа 3**.

# Автоматные грамматики

**Опр.** *Автоматной* грамматикой называется праволинейная (леволинейная) грамматика, такая, что каждое правило с непустой правой частью имеет вид:

$A \rightarrow a$  либо  $A \rightarrow aB$  (для леволинейной, соответственно,  $A \rightarrow a$  либо  $A \rightarrow Ba$ ),

где  $A, B \in N, a \in T$ .

Автоматная грамматика является более простой формой регулярной грамматики.

Существует алгоритм, позволяющий по регулярной (право- или леволинейной) грамматике построить соответствующую автоматную грамматику.

Таким образом, *любой регулярный язык может быть порожден автоматной грамматикой.*

Существует алгоритм, позволяющий устранить из регулярной (автоматной) грамматики все  $\varepsilon$ -правила (кроме  $S \rightarrow \varepsilon$  в случае, если пустая цепочка принадлежит языку; при этом  $S$  не будет встречаться в правых частях правил).

**Утверждение 4.** Для любой регулярной (автоматной) грамматики  $G$  существует неукорачивающая регулярная (автоматная) грамматика  $G'$ , такая что  $L(G) = L(G')$ .

# Иерархия грамматик Хомского

**Утверждение 5.** Справедливы следующие утверждения:

- 1) любая регулярная грамматика является КС-грамматикой;
- 2) любая неукорачивающая КС-грамматика является КЗ-грамматикой;
- 3) любая неукорачивающая грамматика является грамматикой типа 0.

Утверждение 5 следует непосредственно из определений.

Рассматривая только неукорачивающие регулярные и неукорачивающие КС-грамматики, получаем следующую иерархию классов грамматик:

$$\text{Регулярные неукорачивающие} \subset \text{КС неукорачивающие} \subset \text{КЗ} \subset \text{Тип 0}$$

# Иерархия языков

**Утверждение 6.** Справедливы следующие утверждения:

1) Каждый регулярный язык является КС-языком, **но существуют КС-языки, которые не являются регулярными**, например:

$$L = \{a^n b^n \mid n > 0\};$$

2) Каждый КС-язык является КЗ-языком, **но существуют КЗ-языки, которые не являются КС-языками**, например:

$$L = \{a^n b^n c^n \mid n > 0\};$$

3) Каждый КЗ-язык является языком типа 0 (т. е. рекурсивно перечислимым языком), **но существуют языки типа 0, которые не являются КЗ-языками.**

Из утверждения 6 следует иерархия классов языков:

$$\text{Тип 3 (Регулярные)} \subset \text{Тип 2 (КС)} \subset \text{Тип 1 (КЗ)} \subset \text{Тип 0}$$

# Иерархия языков



Для  $k = 1, 2, 3$  язык типа  $k$  является также и языком типа  $k - 1$  (класс языков типа  $k$  является подклассом класса языков типа  $k - 1$ ).



# Иерархия языков

- **Утверждение 7.** Проблема «Можно ли язык, описанный грамматикой типа  $k$  ( $k = 0, 1, 2$ ), описать грамматикой типа  $k + 1$ ?» является *алгоритмически неразрешимой*.
- Т.е., нет алгоритма, позволяющего по заданному описанию языка  $L$  (например, по грамматике), определить максимальное  $k$ , такое что  $L$  является языком типа  $k$
- НО! В примерах и задачах при ответе на вопрос «Какого типа заданный язык  $L$ ?» будем указывать, если не оговорено иное, максимально возможное  $k$  для заданного языка  $L$ .

# Задача

Даны грамматики  $G_1$  и  $G_2$ :

$G_1$

$S \rightarrow 0A1$   
 $A \rightarrow 0A0$   
 $A \rightarrow \epsilon$

$G_2$ :

$S \rightarrow aSb \mid \epsilon$   
 $S \rightarrow 0A$   
 $A \rightarrow 0B \mid 1$   
 $B \rightarrow 0A$

- 1) Какого типа язык  $L(G_1)$ ?
- 2) Какого типа язык  $L(G_2)$ ?

## Решение

1. Правила грамматики  $G_1$  удовлетворяют ограничениям на правила КС-грамматик, но не удовлетворяют ограничениям регулярных грамматик, т. е. это грамматика типа 2, и по этому  $L(G_1)$  является языком типа 2.

$L(G_1) = \{00^n0^n1 \mid n \geq 0\} = \{0^{(2n+1)}1 \mid n \geq 0\}$  и этот язык является также языком типа 3, поскольку описывается соответствующей регулярной грамматикой:

$S \rightarrow B1; B \rightarrow A0; A \rightarrow A00 \mid 00$

Т.о., максимальное  $k$ , для которого  $L(G_1)$  является языком типа  $k$ , равно 3.

# Задача

$G_2$ :

|     |               |                     |
|-----|---------------|---------------------|
| $S$ | $\rightarrow$ | $aSb \mid \epsilon$ |
| $S$ | $\rightarrow$ | $0A$                |
| $A$ | $\rightarrow$ | $0B \mid 1$         |
| $B$ | $\rightarrow$ | $0A$                |

2. По виду правил  $G_2$  является грамматикой типа 2 и не является грамматикой типа 3. Следовательно,  $L(G_2)$  является языком типа 2.

$$L(G_2) = \{a^n 0^m 1 b^n \mid n, m \geq 0\}$$

Является ли  $L(G_2)$  языком типа 3?

Ответ на этот вопрос отрицательный, так как не существует регулярной грамматики (т. е. грамматики типа 3), порождающей язык  $L(G_2) = \{a^n b^n \mid n \geq 0\}$ .

Т.о., максимальное  $k$ , для которого  $L(G_2)$  является языком типа  $k$ , равно 2.

# Примеры грамматик и языков

## Регулярные

1. Грамматика  $S \rightarrow aS \mid a$  является праволинейной (неукорачивающей) грамматикой и порождает регулярный язык  $\{a^n \mid n > 0\}$ .

Этот язык может быть порожден и леволинейной грамматикой:  $S \rightarrow Sa \mid a$ .

Обе эти грамматики являются автоматными.

2. Грамматика  $S \rightarrow aS \mid \varepsilon$  является праволинейной и порождает регулярный язык  $\{a^n \mid n \geq 0\}$ .

Для любого регулярного языка существует неукорачивающая регулярная грамматика (см. утверждение 4):

$$\begin{aligned} S &\rightarrow aA \mid a \mid \varepsilon \\ A &\rightarrow a \mid aA \end{aligned}$$

Правило с пустой правой частью может применяться только один раз и только на первом шаге вывода; остальные правила таковы, что их правая часть не короче левой, т. е. грамматика неукорачивающая.

# Примеры грамматик и языков

3. Грамматика  $S \rightarrow A\perp \mid B\perp$  левوليнейная;  
 $A \rightarrow a \mid Ba$   
 $B \rightarrow b \mid Bb \mid Ab$

она порождает регулярный язык, состоящий из всех непустых цепочек в алфавите  $\{a, b\}$ , заканчивающихся символом  $\perp$  (маркер конца) и не содержащих подцепочку  $aa$ .

То есть в цепочках этого языка символ  $a$  не может встречаться два раза подряд, хотя бы один символ  $b$  обязательно присутствует между любыми двумя  $a$ .

Данный язык описывается как:  $L = \{\omega\perp \mid \omega \in \{a, b\}^+, aa \not\subset \omega\}$ .

# Примеры грамматик и языков

## Контекстно-свободные

1. Грамматика

$$\begin{aligned} S &\rightarrow aQb \mid acsb \\ Q &\rightarrow cSc \end{aligned}$$

является контекстно-свободной (неукорачивающей) и порождает КС-язык  $\{(ac)^n (cb)^n \mid n > 0\}$ , который, не является регулярным.

2. Грамматика

$$S \rightarrow aSa \mid bSb \mid \varepsilon$$

порождает КС-язык  $\{xx^R, x \in \{a, b\}^*\}$ . Данный язык не является регулярным.

Грамматика не удовлетворяет определению неукорачивающей, но для нее существует эквивалентная неукорачивающая грамматика (см. утверждение 2):

$$\begin{aligned} S &\rightarrow A \mid \varepsilon \\ A &\rightarrow aAa \mid bAb \mid aa \mid bb \end{aligned}$$

# Примеры грамматик и языков

## Неукорачивающие и контекстно-зависимые

1. Грамматика:

$$\begin{array}{ll} S & \rightarrow aSBC \mid abC \\ CB & \rightarrow BC \\ bB & \rightarrow bb \\ bC & \rightarrow bc \\ cC & \rightarrow cc \end{array}$$

неукорачивающая и порождает язык  $\{a^n b^n c^n \mid n > 0\}$ , который является языком типа 1, но не является языком типа 2.

Правило  $CB \rightarrow BC$  не удовлетворяет определению КЗ-грамматики. Заменим его тремя новыми

$CB \rightarrow CD, CD \rightarrow BD, BD \rightarrow BC$ . Получим эквивалентную серию контекстно-зависимых правил, которые меняют местами символы  $C$  и  $B$ . Таким образом, получаем эквивалентную КЗ-грамматику:

$$\begin{array}{ll} S & \rightarrow aSBC \mid abC \\ CB & \rightarrow CD \\ CD & \rightarrow BD \\ BD & \rightarrow BC \\ bB & \rightarrow bb \\ bC & \rightarrow bc \\ cC & \rightarrow cc \end{array}$$

# Примеры грамматик и языков

1. Грамматика типа 0

$$\begin{array}{lcl} S & \rightarrow & SS \\ SS & \rightarrow & \varepsilon \end{array}$$

Второе правило не удовлетворяет ограничениям неукорачивающей грамматики. В данной грамматике существует бесконечно много выводов, однако порождаемый язык конечен и состоит из единственной цепочки:  $\{\varepsilon\}$ .

2. Грамматика

$$\begin{array}{lcl} S & \rightarrow & SS \\ SS & \rightarrow & Sa \mid S \end{array}$$

также не является неукорачивающей и порождает пустой язык, так как ни одна терминальная цепочка не выводится из  $S$ .

Языки  $L_\varepsilon = \{\varepsilon\}$  и  $L_\emptyset = \emptyset$  (пустой язык) могут быть описаны грамматиками типа 3.



# Разбор цепочек

- **Опр.** Цепочка в алфавите  $T$  принадлежит языку, порождаемому грамматикой  $\langle T, N, P, S \rangle$ , только в том случае, если существует ее вывод из начального символа  $S$  этой грамматики.
- Процесс построения такого вывода (а, следовательно, и определения принадлежности цепочки языку) называется *разбором*.
- Построение вывода можно осуществлять и в обратном порядке: в исходной цепочке ищем вхождение в неё правой части некоторого правила и заменяем его на левую часть (делаем свёртку).
- В итоге исходная цепочка «сворачивается» к некоторой сентенциальной форме. Затем идет следующая свертка и т. д., пока не приходим к  $S$ .
- Такой процесс разбора называют также *анализом*.

# Пример

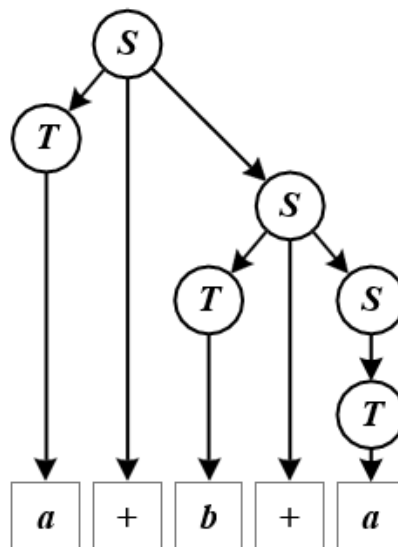
$$G_{expr} = \langle \{a, b, +\}, \{S, T\}, \{S \rightarrow T \mid T + S ; T \rightarrow a \mid b\}, S \rangle$$

Можно построить разные выводы для цепочки  $a + b + a$  :

$$1) S \rightarrow T + S \rightarrow T + T + S \rightarrow T + T + T \rightarrow a + T + T \rightarrow a + b + T \rightarrow a + b + a$$

$$2) S \rightarrow T + S \rightarrow a + S \rightarrow a + T + S \rightarrow a + b + S \rightarrow a + b + T \rightarrow a + b + a$$

$$3) S \rightarrow T + S \rightarrow T + T + S \rightarrow T + T + T \rightarrow T + T + a \rightarrow T + b + a \rightarrow a + b + a$$



Дерево разбора

Регулярные множества, их  
распознавание и порождение

# Регулярные множества и регулярные выражения

- Опр. Пусть  $\Sigma$  - конечный алфавит. Регулярное множество в алфавите  $\Sigma$  определяется следующим образом:
  - 1)  $\emptyset$  – регулярное множество в алфавите  $\Sigma$ .
  - 2)  $\{\varepsilon\}$  – регулярное множество в алфавите  $\Sigma$ .
  - 3)  $\{a\}$  – регулярное множество в алфавите  $\Sigma$  для каждого  $a \in \Sigma$ .
  - 4) Если  $Q$  и  $P$  - регулярные множества в алфавите  $\Sigma$ , то множества  $Q \cup P$ ,  $QP$  и  $P^*$  регулярные.
  - 5) Ничто другое не является регулярным множеством в алфавите  $\Sigma$ .

# Регулярные выражения

- **Опр.** Пусть  $\Sigma$  — конечный алфавит. Определим рекурсивно *регулярное выражение в алфавите  $\Sigma$*  и регулярные множества, которые они обозначают.

*Базис индукции:*

- 1)  $\emptyset$  есть регулярное выражение, обозначающее регулярное множество  $\emptyset$
- 2)  $\varepsilon$  есть регулярное выражение, обозначающее регулярное множество  $\{\varepsilon\}$
- 3)  $a \in \Sigma$  есть регулярное выражение, обозначающее регулярное множество  $\{a\}$

*Индукция:*

Если  $\alpha$  и  $\beta$  — регулярные выражения, обозначающие регулярные множества  $Q$  и  $P$  соответственно, то:

- 1)  $(\alpha + \beta)$  — регулярное выражение, обозначающее  $Q \cup P$ ;
- 2)  $(\alpha\beta)$  — регулярное выражение, обозначающее  $QP$ ;
- 3)  $(\beta)^*$  — регулярное выражение, обозначающее  $P^*$ ;

Никаких других регулярных выражений, кроме тех, что построены в соответствии с описанным определением, нет.

Лемма 1. Если  $\alpha, \beta, \gamma$  - регулярные выражения, то справедливы следующие соотношения:

$$1) \alpha + \beta = \beta + \alpha$$

$$2) \alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma$$

$$3) \alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma$$

$$4) \alpha\varepsilon = \varepsilon\alpha = \alpha$$

$$5) \alpha^* = \alpha + \alpha^*$$

$$6) \alpha + \alpha = \alpha$$

$$7) \emptyset^* = \varepsilon$$

$$8) \alpha(\beta\gamma) = (\alpha\beta)\gamma$$

$$9) (\alpha + \beta)\gamma = \alpha\gamma + \beta\gamma$$

$$10) \emptyset\alpha = \alpha\emptyset = \emptyset$$

$$11) (\alpha^*)^* = \alpha^*$$

$$12) \alpha + \emptyset = \alpha$$

- Док-во. Пусть  $\alpha, \beta$  обозначают множества  $Q$  и  $P$  соответственно. Тогда  $\alpha + \beta$  обозначает  $Q \cup P$ , а  $\beta + \alpha$  –  $P \cup Q$ .

Но  $Q \cup P = P \cup Q$ . Следовательно,  $\alpha + \beta = \beta + \alpha$ .

# Алгебраические законы для регулярных выражений

- Объединение и конкатенация ведут себя как сложение и умножение.
- $+$  является коммутативным и ассоциативным;
- Конкатенация является ассоциативной операцией.
- Конкатенация распределяется по  $+$ .
- Исключение: Конкатенация не коммутативна.

# Единицы и нейтральные элементы в RE

- $\emptyset$  является нейтральным элементом для  $+$ .

$$R + \emptyset = R.$$

- $\varepsilon$  является «единицей» для конкатенации.

$$\varepsilon R = R\varepsilon = R.$$

- $\emptyset$  является «нулем» для конкатенации.

$$\emptyset R = R\emptyset = \emptyset.$$



# Пример преобразования РЕ

Пусть язык задан регулярным выражением:  $L = (0^* + \varepsilon)^*(1 + 1^+)$ .

Имеем:  $L = ((0^+ + \varepsilon) + \varepsilon)^*(1 + 1^+)$

См. определение итерации.

$L = (0^+ + \varepsilon + \varepsilon)^*(1 + 1^+)$

См. (2)

$L = (0^+ + \varepsilon)^*(1 + 1^+)$

См. (6)

$L = (0^*)^*(1 + 1^+)$

См. определение положительной итерации.

$L = 0^*(1 + 1^+)$

См. (11)

$L = 0^*1(\varepsilon + 1^*)$

См. (3)

$L = 0^*11^*$

См. (2,6)

$L = 0^*1^+$

См. определение положительной итерации.

$$L = \{0^n 1^m \mid n \geq 0, m \geq 1\}$$

# Примеры RE

- $\Sigma = \{0, 1\}$
- $L(\mathbf{01}) = \{01\}$ .
- $L(\mathbf{01+0}) = \{01, 0\}$ .
- $L(\mathbf{0(1+0)}) = \{01, 00\}$ .
- $L(\mathbf{0^*}) = \{\epsilon, 0, 00, 000, \dots\}$ .
- $L((\mathbf{0+10})^*(\epsilon+\mathbf{1})) =$  все строки из 0 и 1 без двух последовательных 1.

# Примеры RE

**Примеры** регулярных выражений над алфавитом  $\{a, b\}$ :

$$a + b, (a + b)^*, \quad (aa(ab)^*bb)^*.$$

Соответствующие языки:

$$L(a + b) = \{a\} \cup \{b\} = \{a, b\},$$

$$L((a + b)^*) = \{a, b\}^*,$$

$$L((aa(ab)^*bb)^*) = \{\varepsilon\} \cup \{aa x_1 bbaa x_2 bb \dots x_k bb \mid k \geq 1, \\ x_i \in \{(ab)^n \mid n \geq 0\} \text{ для } i = 1, \dots, k\}.$$

# Уравнения с регулярными коэффициентами

- Рассм. уравнение  $X=aX+b$ , где  $a$  и  $b$  – РВ.

$X=a*b$  – решение уравнения:

$$a*b=aa*b+b$$

$$a*b=(aa*+\varepsilon)b$$

$$a*b=a*b$$

Если множество, определяемое рег. выражением  $a$ , содержит  $\varepsilon$ , то ур-е имеет бесконечно много решений:  $X=a*(b+c)$  для любого РВ  $c$ .

В этом случае берут «наименьшее решение» – *наименьшую неподвижную точку*.

- Опр. Система уравнений с рег. коэффициентами наз. *стандартной системой* с мн-вом неизвестных  $\Delta = \{X_1, X_2, \dots, X_n\}$ , если она имеет вид:

$$X_1 = a_{10}X_1 + a_{12}X_2 + \dots + a_{1n}X_n$$

.....

$$X_n = a_{n0}X_1 + a_{n2}X_2 + \dots + a_{nn}X_n$$

где все  $a_{ij}$  - регулярные выражения в алфавите, не пересекающимся с  $\Delta$ .

Если  $a_{ij} = \emptyset$ , то соотв. слагаемое отсутствует, если  $a_{ij} = \varepsilon$ , то слагаемое равно  $X_j$ .

# Алгоритм решения стандартной системы уравнений с регулярными коэффициентами

- Вход. Стандартная система  $Q$  уравнений с рег. коэффициентами в алфавите  $\Sigma$  и множеством неизвестных  $\Delta = \{X_1, X_2, \dots, X_n\}$ .
- Выход. Решение системы  $Q$  в виде  $X_1 = a_1, X_2 = a_2, \dots, X_n = a_n$ , где  $a_i$  - РВ в алфавите  $\Sigma$ .

- Метод. (см. решение СЛУ методом Гаусса)

Шаг 1.  $i = 1$ .

Шаг 2. Если  $i = n$ , перейти к шагу 4.

Иначе, с помощью тождеств РВ записать ур-е для  $X_i$  в виде  $X_i = \alpha X_i + \beta$ , где  $\alpha$  - РВ в алфавите  $\Sigma$ , а  $\beta$  - РВ вида  $\beta_0 + \beta_i X_{i+1} + \dots + \beta_n X_n$ ;  $\beta_i$  - РВ в алфавите  $\Sigma$ .

Затем, в правых частях уравнений для  $X_{i+1}, \dots, X_n$  заменить  $X_i$  выражением  $\alpha^* \beta$ .

Шаг 3.  $i = i + 1$ . Перейти к шагу 2.

Шаг 4. Записать ур-е для  $X_n$  в виде  $X_n = \alpha X_n + \beta$ , где  $\alpha$  и  $\beta$  - РВ в алфавите  $\Sigma$ .

Перейти к шагу 5 ( $i = n$ ).

Шаг 5. Уравнение для  $X_i$  имеет вид  $X_i = \alpha X_i + \beta$ , где  $\alpha$  и  $\beta$  - РВ в алфавите  $\Sigma$ .

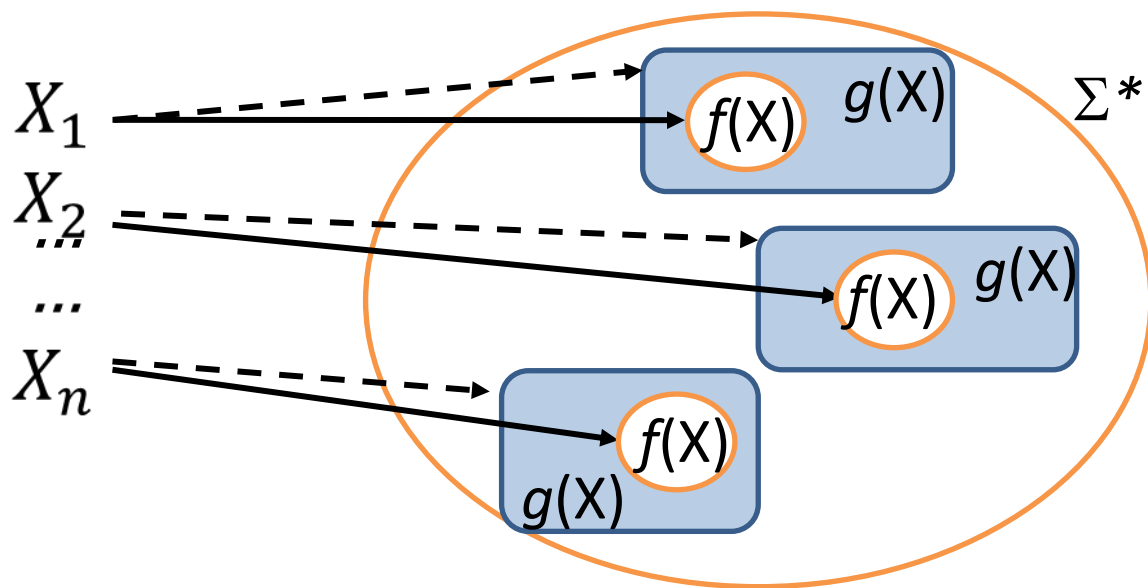
Записать на выходе  $X_i = \alpha^* \beta$  и в ур-я для  $X_{i-1}, \dots, X_1$  подставить  $\alpha^* \beta$  вместо  $X_i$ .

Шаг 6. Если  $i = 1$ , остановиться. В противном случае  $i = i - 1$  и вернуться к шагу 5.

Опр.

Отображение  $f$  множества  $\Delta$  в множество языков в алфавите  $\Sigma$  называют *решением системы*  $Q$ , если после подстановки в каждое уравнение  $f(X)$  вместо  $X$  для каждого  $X \in \Delta$  получаем тождества.

Если для любого другого решения  $g$ :  $(\forall X) f(X) \subseteq g(X)$ , то  $f$  называют *наименьшей неподвижной точкой системы*  $Q$ .





## Пример

- Пусть . Система имеет вид:

$$\begin{aligned}X_1 &= 0X_2 + 1X_1 + \varepsilon \\X_2 &= 0X_3 + 1X_2 \\X_3 &= 0X_1 + 1X_3\end{aligned}$$

- Из первого ур-я получаем  $X_1 = 1^*(0X_2 + \varepsilon)$ , сделаем соотв. подстановки во второе и третье ур-я. Получим:

$$\begin{aligned}X_1 &= 1^*0X_2 + 1^* \\X_2 &= 0X_3 + 1X_2 \\X_3 &= 01^*0X_2 + 01^* + 1X_3\end{aligned}$$

- Второе ур-е представим в виде  $X_2 = 1^*0X_3$ , подставим в третье ур-е. Получим:

$$\begin{aligned}X_1 &= 1^*0X_2 + 1^* \\X_2 &= 1^*0X_3 \\X_3 &= 01^*01^*0X_3 + 01^* + 1X_3\end{aligned}$$

- Найдем выражение для  $X_3$  и подставим его во второе ур-е:

$$\begin{aligned}X_1 &= 1^*0X_2 + 1^* \\X_2 &= 1^*0(01^*01^*0 + 1)^*01^* \\X_3 &= (01^*01^*0 + 1)^*01^*\end{aligned}$$

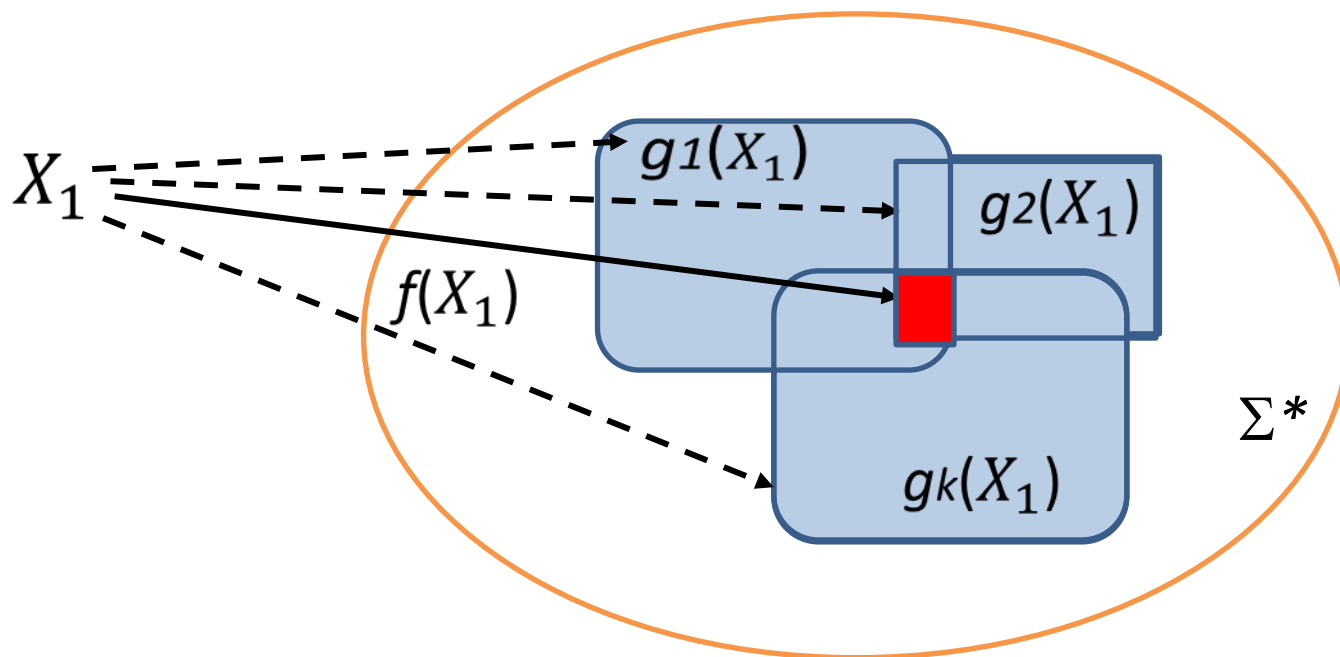
- Решение для  $X_2$  подставим в первое ур-е:

$$\begin{aligned}X_1 &= 1^*01^*0(01^*01^*0 + 1)^*01^* + 1^* \\X_2 &= 1^*0(01^*01^*0 + 1)^*01^* \\X_3 &= (01^*01^*0 + 1)^*01^*\end{aligned}$$

- Лемма 2. Каждая стандартная система уравнений  $Q$  с неизвестными  $\Delta$  обладает единственной наименьшей неподвижной точкой.
- Док-во.

Пусть  $f(X) = \{w | w \in g(X) \text{ для всех решений } g \text{ системы } Q\}$  для всех  $X \in \Delta$ .  
Очевидно, что  $f(X)$  - решение и  $(\forall X) f(X) \subseteq g(X)$ .

Таким образом,  $f$  – единственная наименьшая неподвижная точка системы  $Q$ .



- Теорема. Алгоритм находит наименьшую неподвижную точку стандартной системы уравнений.
- Док-во. После применения шага 5  $(\forall i) X_i = \alpha_i$ , где  $\alpha_i$  - РВ в алфавите  $\Sigma$ .

Отображение  $f$ , для которого  $f(X_i) = \alpha_i$  будет наименьшей неподвижной точкой.

# Регулярные множества и праволинейные грамматики

# Регулярные множества и праволинейные грамматики

- Язык определяется праволинейной грамматикой т.и.т.т., когда он является регулярным множеством.
- Лемма 3. Множества  $\emptyset$  ,  $\{\varepsilon\}$  и  $a$  для всех  $a \in \Sigma$  являются праволинейными языками.
- Док-во.
  - 1)  $G = (\{S\}, \Sigma, P, S)$  – праволинейная грамматика, для которой  $L(G) = \emptyset$ .
  - 2)  $G = (\{S\}, \Sigma, \{S \rightarrow \varepsilon\}, S)$  – праволинейная грамматика, для которой  $L(G) = \{\varepsilon\}$ .
  - 3)  $G_a = (\{S\}, \Sigma, \{S \rightarrow a\}, S)$  – праволинейная грамматика, для которой  $L(G_a) = \{a\}$ .

- Лемма 4. Если  $P$  и  $Q$  – праволинейные языки, то языки

1)  $P \cup Q$ , 2)  $PQ$ , 3)  $P^*$  тоже праволинейные.

- Док-во.

Т.к.  $P$  и  $Q$  – праволинейные, то  $\exists$  праволинейные грамматики

$G_P = (N_1, \Sigma, P_1, S_1)$  и  $G_Q = (N_2, \Sigma, P_2, S_2)$ , для которых

$L(G_P) = P$  и  $L(G_Q) = Q$ . Считаем, что  $N_1 \cap N_2 = \emptyset$ .

1)  $G_3 = (N_1 \cup N_2, \Sigma, P_1 \cup P_2 \cup \{S_3 \rightarrow S_1 | S_2\}, S_3)$  – праволинейная.

$L(G_3) = L(G_P) \cup L(G_Q)$  т.к. для каждого вывода  $S_3 \Rightarrow_{G_3}^+ w$  существует либо вывод  $S_1 \Rightarrow_{G_P}^+ w$ , либо вывод  $S_2 \Rightarrow_{G_Q}^+ w$  и обратно.

Т.к.  $G_3$  - праволинейная грамматика, то  $L(G_3)$  – праволинейный язык.

2)  $G_4 = (N_1 \cup N_2, \Sigma, P_4, S_1)$  – праволинейная.

$P_4$ : (1) Если  $A \rightarrow xB$  есть в  $P_1$ , то  $A \rightarrow xB$  принадлежит  $P_4$ .

(2) Если  $A \rightarrow x$  есть в  $P_1$ , то  $A \rightarrow xS_2$  принадлежит  $P_4$ .

(3) Все правила из  $P_2$  принадлежат  $P_4$ .

Отметим, что если  $S_1 \Rightarrow_{G_P}^+ w$ , то  $S_1 \Rightarrow_{G_4}^+ wS_2$ , а если  $S_2 \Rightarrow_{G_Q}^+ x$ , то  $S_2 \Rightarrow_{G_4}^+ x$ .

Таким образом,  $L(G_P)L(G_Q) \subseteq L(G_4)$ .

Пусть  $S_1 \Rightarrow_{G_4}^+ w$ . Т.к. в  $G_4$  нет правил  $A \rightarrow x$ , попавших из  $P_1$ , то этот вывод можно сделать так:  $S_1 \Rightarrow_{G_4}^+ xS_2 \Rightarrow_{G_4}^+ xu$ , где  $w = xu$  и все правила в выводе  $S_1 \Rightarrow_{G_4}^+ wS_2$  попали в  $P_4$  по (1) и (2).

Следовательно, должны быть выводы  $S_1 \Rightarrow_{G_P}^+ x$  и  $S_2 \Rightarrow_{G_Q}^+ u$ .

Отсюда  $L(G_4) \subseteq L(G_P)L(G_Q)$ .

3) Пусть  $G_5 = (N_1 \cup \{S_5\}, \Sigma, P_5, S_5)$  такая, что  $S_5 \notin N_1$  и

$P_5$ : (1) Если  $A \rightarrow xB$  есть в  $P_1$ , то  $A \rightarrow xB$  принадлежит  $P_5$ .

(2) Если  $A \rightarrow x$  есть в  $P_1$ , то  $A \rightarrow xS_5$  и  $A \rightarrow x$  принадлежат  $P_5$ .

(3)  $S_5 \rightarrow S_1 | \varepsilon$  принадлежат  $P_5$ .

Очевидно, что

$$S_5 \Rightarrow_{G_5}^+ x_1 S_5 \Rightarrow_{G_5}^+ x_1 x_2 S_5 \Rightarrow_{G_5}^+ \dots \Rightarrow_{G_5}^+ x_1 x_2 \dots x_{n-1} x_n$$

т.и.т.т., когда

$$S_1 \Rightarrow_{G_P}^+ x_1, S_1 \Rightarrow_{G_P}^+ x_2, \dots, S_1 \Rightarrow_{G_P}^+ x_n.$$

Отсюда следует, что  $L(G_4) = (L(G_P))^*$ . ■



- Теорема. Язык является регулярным множеством т.и.т.т., когда он праволинейный.
- Док-во.

Необходимость.

Следует из лемм 3 и 4, индукцией по числу шагов построения регулярного множества, где один шаг – это применение одного из правил, определяющих регулярные множества.

### Достаточность.

Пусть  $G = (N, \Sigma, P, S)$  – праволинейная грамматика и  $N = \{A_1, A_2, \dots, A_n\}$ .

Можно построить стандартную систему уравнений с регулярными коэффициентами, неизвестными которой являются нетерминалы из  $N$ .

Уравнение для  $A_i$  будет иметь вид:  $A_i = \alpha_{i0} + \alpha_{i1}A_1 + \dots + \alpha_{in}A_n$ , где

(1)  $\alpha_{i0} = w_1 + \dots + w_k$ , если  $A_i \rightarrow w_1 \mid \dots \mid w_k$  - все правила с левой частью  $A_i$  и правой частью, состоящей только из терминалов (если  $k=0$ , то  $\alpha_{i0} = \emptyset$ ).

(2) Для  $j > 0$   $\alpha_{ij} = x_1 + \dots + x_m$ , если  $A_i \rightarrow x_1A_j \mid \dots \mid x_mA_j$  - все правила с левой частью  $A_i$  и правой частью, оканчивающуюся на  $A_j$  (если  $m=0$ , то  $\alpha_{ij} = \emptyset$ ).

Решая эту систему уравнений получаем решение  $f$  для  $N = \{A_1, A_2, \dots, A_n\}$ .

Для  $S$  получаем РВ  $f(S)$ , которое определяет язык  $L(G)$ .

Но алгоритм строит  $f(S)$  как язык, обозначаемый некоторым РВ.

Таким образом,  $L(G)$  – регулярное множество.

# Пример построения грамматики по РВ

- Рассм. РВ  $(0+1)^*2$ . Построим праволинейную грамматику, определяющую тот же язык.
- Для языков  $\{0\}$ ,  $\{1\}$ ,  $\{2\}$  определим грамматики (согласно леммам):

$$G_0 = (\{S_0\}, \{0, 1, 2\}, \{S_0 \rightarrow 0\}, S_0),$$

$$G_1 = (\{S_1\}, \{0, 1, 2\}, \{S_1 \rightarrow 1\}, S_1),$$

$$G_2 = (\{S_2\}, \{0, 1, 2\}, \{S_2 \rightarrow 2\}, S_2)$$

- Для РВ  $0 + 1$  по грамматикам  $G_0$  и  $G_1$  строим грамматику  $G_3 = (\{S_3, S_0, S_1\}, \{0, 1, 2\}, P_3, S_3)$

- 

$$P_3 = \left\{ \begin{array}{l} S_3 \rightarrow S_0 \\ S_3 \rightarrow S_1 \\ S_0 \rightarrow 0, \\ S_1 \rightarrow 1 \end{array} \right\}.$$

- Для РВ  $(0+1)^*$  по грамматике  $G_3$  строим грамматику  $G_4 = (\{S_4, S_3, S_0, S_1\}, \{0, 1, 2\}, P_4, S_4)$

где  $P_4$ :

$$S_4 \rightarrow 0 \mid 1 \mid 0S_3 \mid 1S_3 \mid \varepsilon$$

$$S_3 \rightarrow 0 \mid 1 \mid 0S_3 \mid 1S_3$$

$$S_0 \rightarrow 0 \mid 0S_3$$

$$S_1 \rightarrow 1 \mid 1S_3$$

# Пример (прод.)

- Для РВ  $(0+1)^*2$ . Построим праволинейную грамматику

$$G_5 = (\{S_4, S_3, S_0, S_1, S_2\}, \{0, 1, 2\}, P_5, S_4),$$

где  $P_5$ :

$$S_4 \rightarrow 0S_2 \mid 1S_2 \mid 0S_3 \mid 1S_3 \mid 2$$
$$S_3 \rightarrow 0S_2 \mid 1S_2 \mid 0S_3 \mid 1S_3$$
$$S_0 \rightarrow 0S_2 \mid 0S_3$$
$$S_1 \rightarrow 1S_2 \mid 1S_3$$
$$S_2 \rightarrow 2$$

определяющую тот же язык.

# Пример построения РВ по праволинейной грамматике

- Пусть дана грамматика, определяемая правилами:
$$\begin{array}{lcl} S & \rightarrow & 0A \mid 1B \mid \varepsilon \\ A & \rightarrow & 0A \mid 1B \mid 2 \\ B & \rightarrow & 0B \mid 1A \end{array}$$
- Рассмотрим систему уравнений:
$$\begin{array}{lcl} S & = & 0A + 1B + \varepsilon \\ A & = & 0A + 1B + 2 \\ B & = & 0B + 1A \end{array}$$
- Выразим  $B=0^*1A$  и подставим его значение в 1-е и 2-е уравнения:
$$\begin{array}{lcl} S & = & 0A + 10^*1A + \varepsilon \\ A & = & 0A + 10^*1A + 2 \\ B & = & 0^*1A \end{array}$$
- Выразим  $A=(0+10^*1)^*2$  и подставим его в первое уравнение:
$$\begin{array}{lcl} S & = & 0(0 + 10^*1)^*2 + 10^*1(0 + 10^*1)^*2 + \varepsilon \\ A & = & (0 + 10^*1)^*2 \\ B & = & 0^*1A \end{array}$$

Выражение для  $S$  можно преобразовать к виду:  $S = (0 + 10^*1)^+2 + \varepsilon$ .

Это РВ будет определять то же регулярное множество (регулярный язык), что и данная праволинейная грамматика

# Конечные автоматы и регулярные множества

# Способы определения класса регулярных множеств

- Класс регулярных множеств – наименьший класс языков, содержащий  $\emptyset$ ,  $\{\epsilon\}$ .  $\{a\}$  для всех  $a$  и замкнутый относительно операций объединения, конкатенации и итерации.
- Регулярное множества – множества, определяемые регулярными выражениями.
- Регулярное множества – языки, порождаемые праволинейными грамматиками.
- Регулярное множества – языки, распознаваемые конечными автоматами.

# Что такое конечный автомат?

- Формальная система.
- Помнит только конечное количество информации.
- Информация представляется его *состояниями*.
- Состояние изменяется по воздействию *входов*.
- Правила, которые говорят, как изменять состояния под воздействием входов, называются *переходами*.



# Допустимые входы

- Дана последовательность входов (*входная строка*).
- Начать в начальном состоянии и следовать по переходу по каждому очередному символу входной строки.
- Вход *принимается*, если вы перенеслись в финальное (принимающее) состояние после чтения всех входных символов.

# Язык автомата

- Множество строк, принимаемых автоматом  $A$ , являются *языком* автомата  $A$ .
- Обозначается  $L(A)$ .
- Различные множества финальных состояний -> Разные языки.

# Детерминированный конечный автомат

□ Формализм для определения языков, состоящий из:

Конечного множества *состояний* (обозн. обычно  $Q$ ).

*Входной алфавит* ( $\Sigma$ , обычно).

*Функция переходов* ( $\delta$ , обычно).

*Начальное состояние* ( $q_0$ , в  $Q$ , обычно).

*Финальные состояния* ( $F \subseteq Q$ , обычно).

□ “Финальный” и “принимающий” является синонимами.

$$A = (Q, \Sigma, \delta, q_0, F)$$

$$\delta: Q \times \Sigma \rightarrow Q$$

# Функция переходов

- Имеет два аргумента: состояние и входной символ.
- $\delta(q, a)$  = состояние, в которое КДА переходит, если он в состоянии  $q$  получает на вход символ  $a$ .
- **Замечание:** всегда есть следующее состояние – добавим *мёртвое состояние* если нет переходов (Пример далее).
- Форма задания: таблица переходов, граф.
- Функцию переходов можно доопределить для слов:  
$$\delta^*(q, a) = \delta(q, a) \text{ если } |a|=1.$$
$$\delta^*(q, aw) = \delta(\delta^*(q, w), a)$$

## Пример: Расширенная функция переходов

| 0 | 1 |   |
|---|---|---|
| A | A | B |
| B | A | C |
| C | C | C |

$$\delta^*(B, 011) = \delta(\delta^*(B, 01), 1) = \delta(\delta(\delta(B, 0), 1), 1) =$$

$$\delta(\delta(A, 1), 1) = \delta(B, 1) = C$$

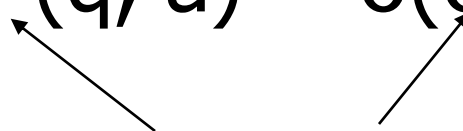
# Дельта с крышкой

□ Мы не будем различать обозначения дельта и дельта со звездочкой

□ Причина:

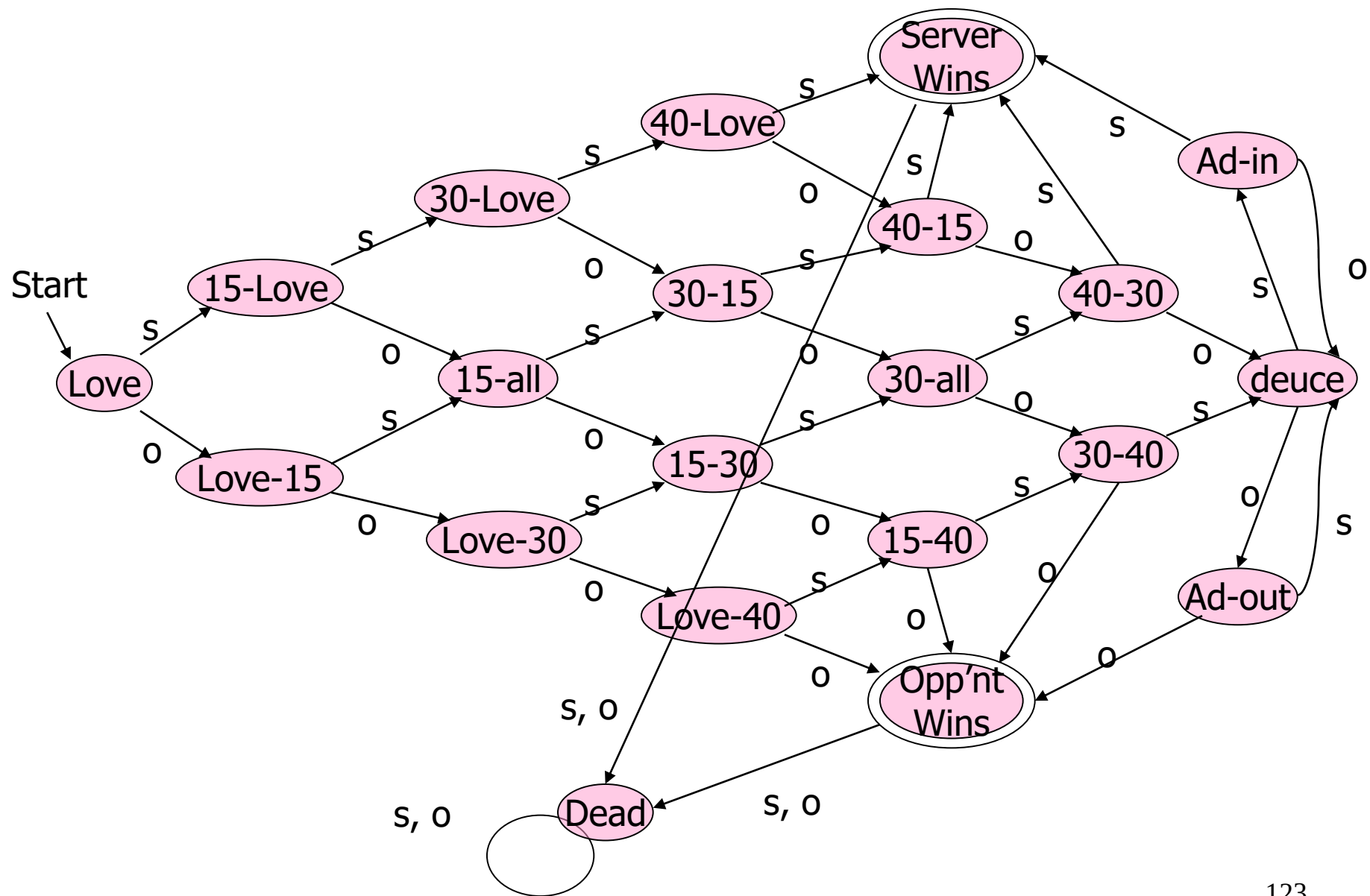
□  $\delta^*(q, a) = \delta(\delta^*(q, \epsilon), a) = \delta(q, a)$

Расширенная дельта



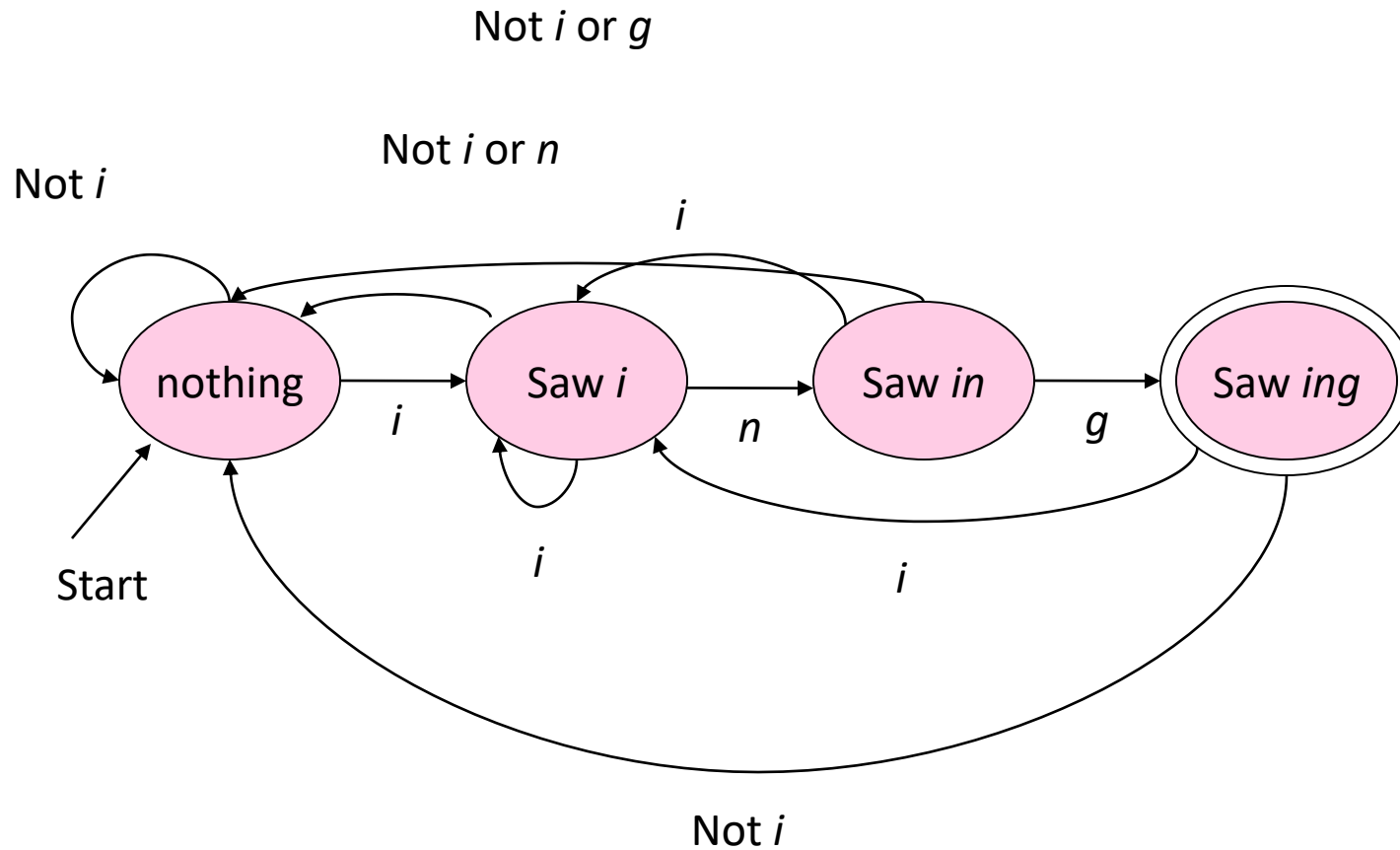
# Представление КДА графом

- Вершины = состояния.
- Дуги представляют функцию переходов.
  - Дуга из состояния  $p$  в состояние  $q$  помечается всеми теми входными символами, по которым происходит переход из  $p$  в  $q$ .
- Стрелка помеченная как "Start" указывает на начальное состояние.
- Финальные состояния обозначаются двойной окружностью.

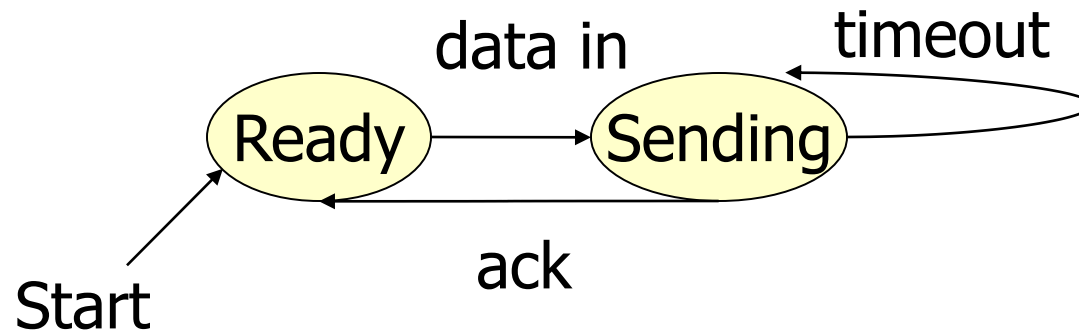




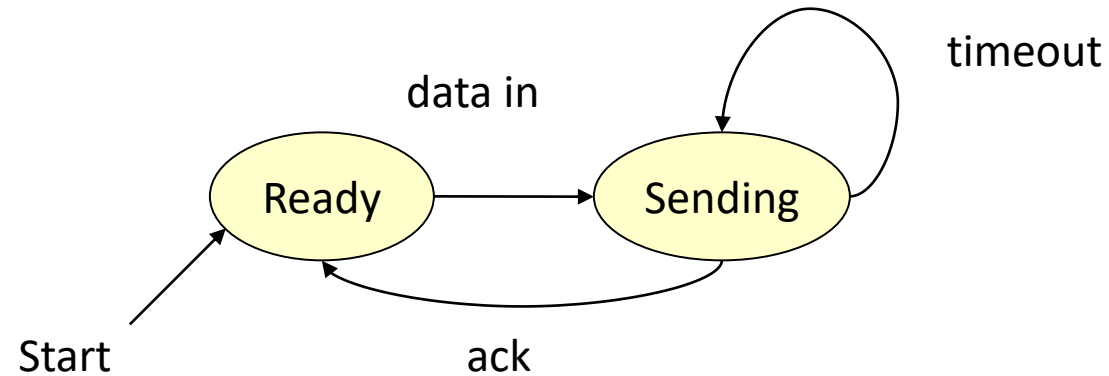
# Пример: Распознавание строк, оканчивающихся на “ing”



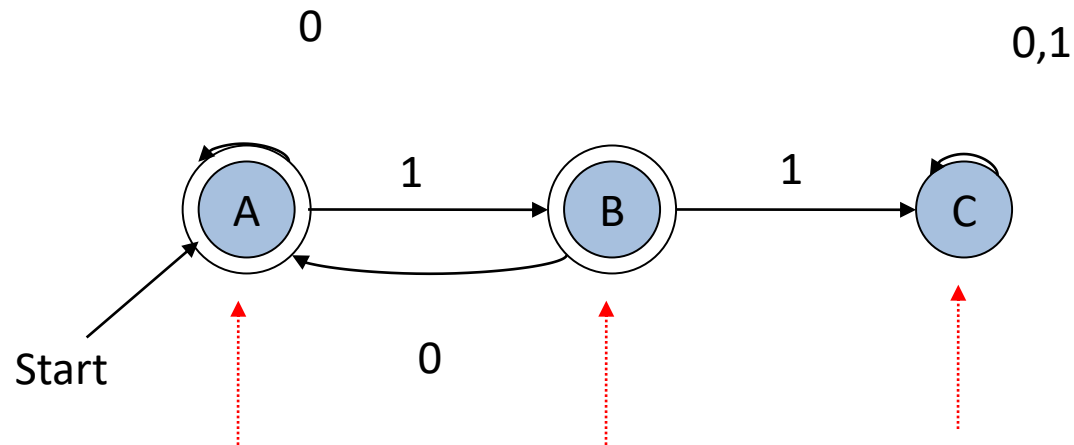
# Пример: Протокол для посылки данных



# Пример: Протокол для пересылки данных



# Пример: Строки без 11



Строка еще  
не имеет 11,  
Не окан-  
чивается 1.

Строка еще  
не имеет 11,  
но окан-  
чивается  
одной 1.

Видны две  
последова-  
тельные  
1.

# Альтернативное представление: Таблица переходов

Финальные  
состояния  
помечаются  
звёздочкой

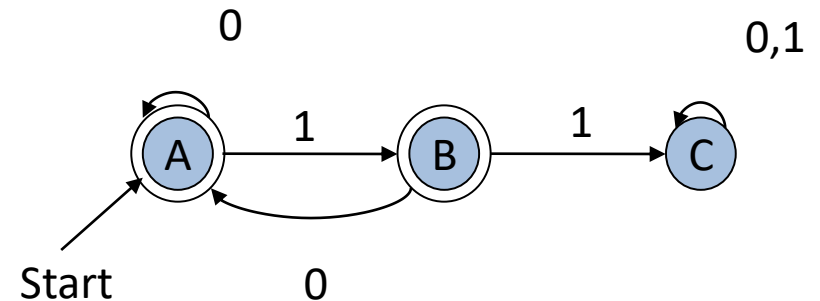
→ \* A  
Стрелка для \* B  
начального  
состояния C

|   | 0 | 1 |
|---|---|---|
| A | A | B |
| B | A | C |
| C | C | C |

Строки = состояния

Каждый элемент  
есть  $\delta$  для строки и  
столбца.

Колонки =  
ВХОДНЫЕ СИМВОЛЫ



# Соглашение: Строки и Символы

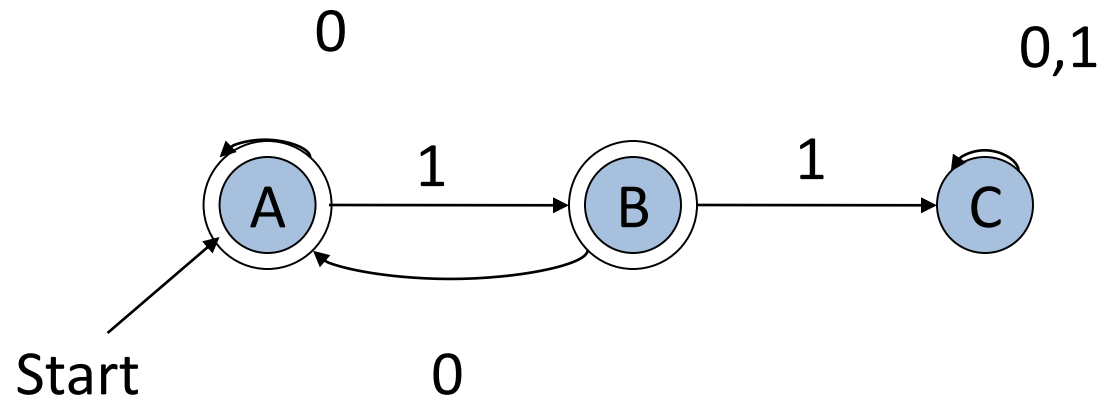
- ...  $w, x, y, z$  - строки.
- $a, b, c, \dots$  - одиночные входные СИМВОЛЫ.

# Язык КДА

- Автоматы всех видов определяют языки.
- Если  $A$  - автомат,  $L(A)$  – его язык.
- Для КДА  $A$ ,  $L(A)$  есть множество строк, помечающих пути из начального состояния в финальное.
- **Формально:**  $L(A)$  = множество строк  $w$  таких, что  $\delta(q_0, w)$  есть в  $F$ .

# Пример: Строки в языке

Строка 101 есть в языке КДА, приведенного ниже.  
Start в A.

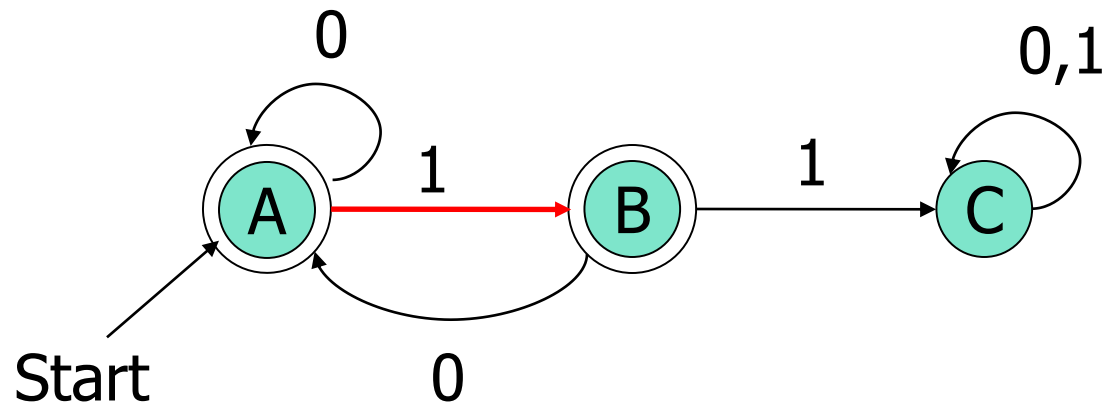




# Пример: Строки в языке

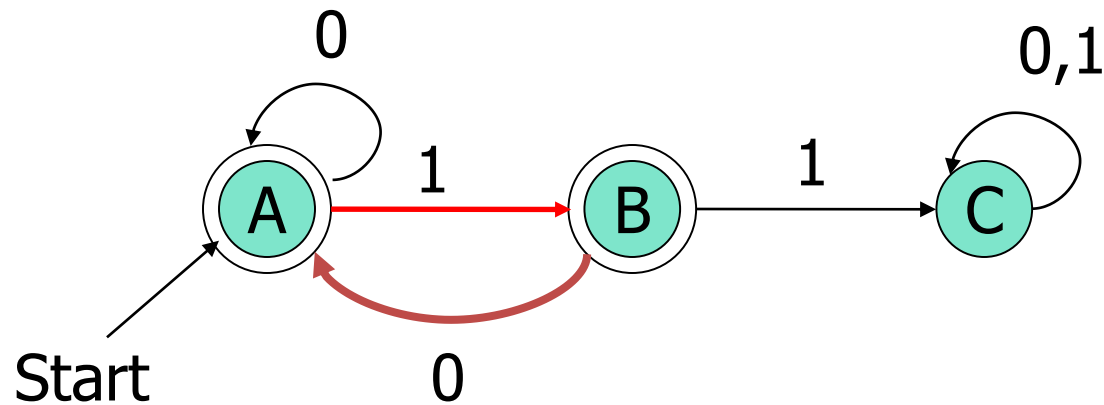
Строка 101 есть в языке КДА, приведенного ниже.

Следуем по дуге, помеченной 1.



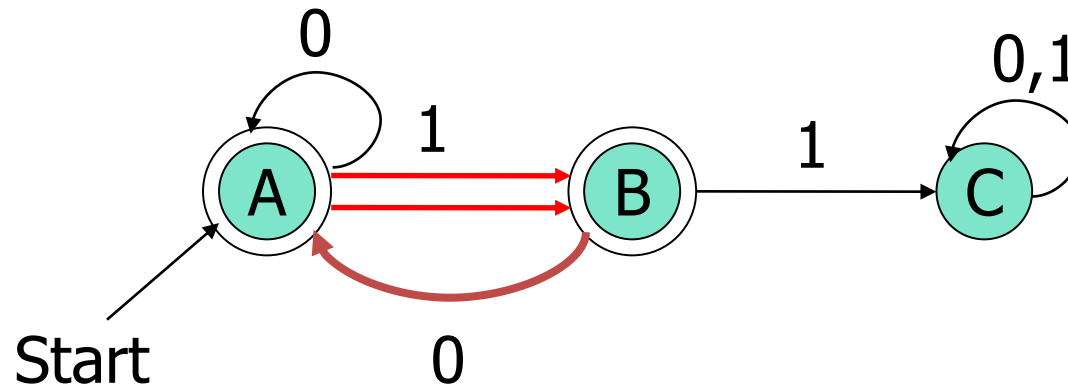
# Пример: Строки в языке

Строка 101 есть в языке КДА, приведенного ниже.  
Затем по дуге, помеченной 0 из текущего состояния В.



# Пример: Строки в языке

Строка 101 есть в языке КДА, приведенного ниже.  
Наконец, по дуге, помеченной 1 из текущего состояния A.  
Результатом является принимающее состояние,  
так что 101 есть в языке.



## Пример – Заключение

□ Языком для нашего КДА является:

$\{w \mid w \text{ есть в } \{0,1\}^* \text{ и } w \text{ не имеет двух последовательных } 1\}$

# Доказательство равенства множеств

- Часто нам нужно доказать, что два описания множеств определяют фактически одно и то же множество
- В нашем случае одно множество "язык КДА ", а другое "множество строк из 0 и 1 без двух последовательных 1."

# Доказательство

- В общем, чтобы доказать  $S = T$ , нам нужно доказать две части:  $S \subseteq T$  и  $T \subseteq S$ . То есть:

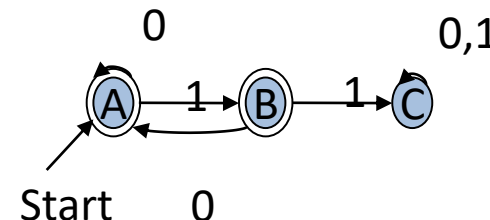
Если  $w$  есть в  $S$ , то  $w$  есть в  $T$ .

Если  $w$  есть в  $T$ , то  $w$  есть в  $S$ .

- В нашем случае:
- $S = L(A)$ , т.е., « $S$  - есть язык нашего автомата», и
- $T$  = «в словах языка нет двух последовательных единиц».

## Часть 1: $S \subseteq T$

- **Доказать:** если  $w$  принимается автоматом то  $w$  не имеет последовательных 1.
- Доказательство индукцией по длине  $w$ .
- **Важно:** Расширим индуктивную гипотезу чтобы быть более детальными по сравнению с утверждением, которое мы доказываем.



# Индуктивная гипотеза

Если  $\delta(A, w) = A$ , то  $w$  не имеет последовательных 1 и не заканчивается на 1.

Если  $\delta(A, w) = B$ , то  $w$  не имеет последовательных 1 и заканчивается единственной 1.

□ **Базис:**  $|w| = 0$ ; т.е.,  $w = \epsilon$ .

□ (1) выполняется, т.к.  $\epsilon$  не имеет 1 совсем.

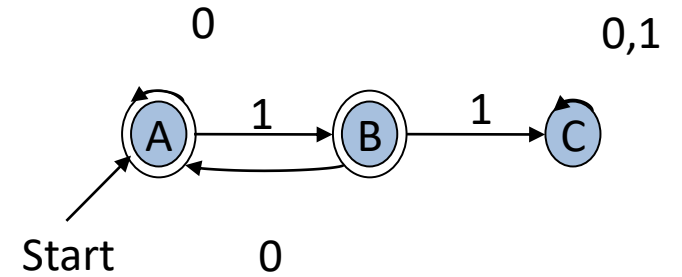
□ (2) выполняется, т.к.  $\delta(A, \epsilon)$  не есть  $B$ .

Важно:

Если "if" часть в "if..then" есть false, то утверждение имеет значение true.

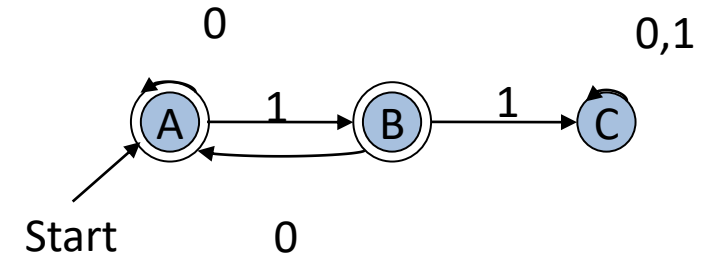


# Индуктивный шаг



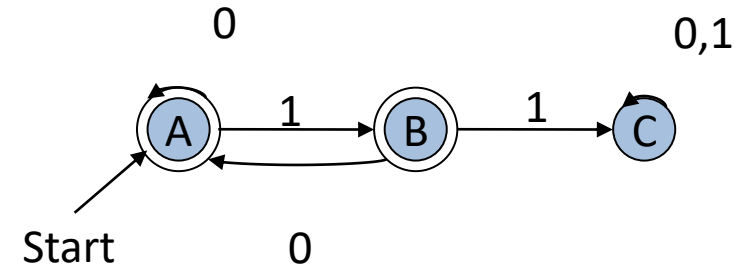
- Пусть (1) и (2) выполняются для строк короче, чем  $w$ , где  $|w|$  по крайней мере 1.
- Т.к.  $w$  не пуста, можно записать  $w = xa$ , где  $a$  - есть последний символ в  $w$ , и  $x$  – предшествующая строка.
- По предположению индукции, пусть  $IN$  верна для  $x$ .

## Индуктивный шаг– (2)



- Нужно доказать (1) и (2) для  $w = xa$ .
- (1) для  $w$  имеем: Если  $\delta(A, w) = A$ , то  $w$  не имеет последовательных 1 и не заканчивается на 1.
- Т.к.  $\delta(A, w) = A$ ,  $\delta(A, x)$  должно быть A или B, и  $a$  должно быть 0 (см. КДА).
- По IH,  $x$  не имеет 11.
- Таким образом,  $w$  не имеет последовательных 1 и не заканчивается на 1.

## Индуктивный шаг– (3)



□ Теперь докажем (2) для  $w = xa$ :

Если  $\delta(A, w) = B$ , то  $w$  не имеет последовательных 1 и заканчивается на 1.

□ Т.к.  $\delta(A, w) = B$ ,  $\delta(A, x)$  должно быть  $A$ , и  $a$  должно быть 1 (см. КДА).

□ По IH,  $x$  не имеет последовательных 1 и не заканчивается на 1.

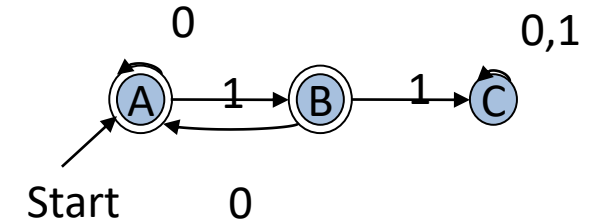
□ Т.о.,  $w$  не имеет последовательных 1 и заканчивается на 1.

## Часть 2: $T \subseteq S$

□ Теперь мы должны доказать : Если  $w$  не имеет 11, то  $w$  принимается



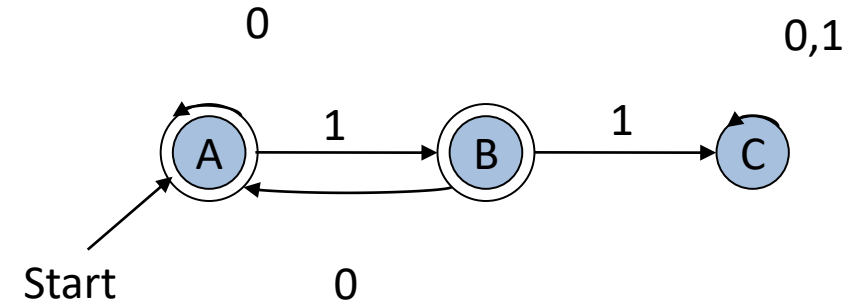
□ *От противного* : Если  $w$  не принимается



то  $w$  имеет 11.

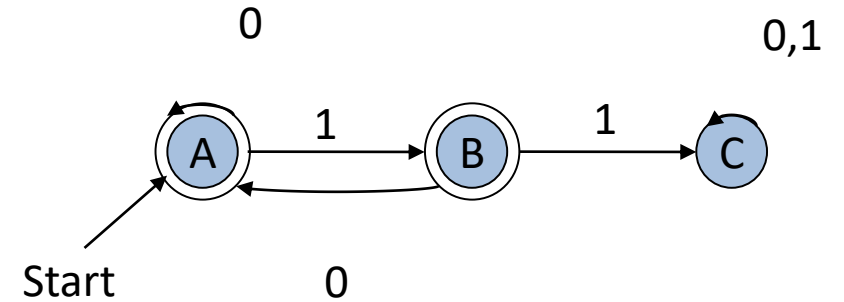
Идея: противным для "если  $X$  то  $Y$ " является эквивалентное утверждение "если не  $Y$  то не  $X$ ."

# От противного



- Так как есть единственный переход от каждого состояния, по каждому входному символу, каждое  $w$  приводит КДА точно в одно состояние.
- Единственный случай, когда  $w$  не принимается будет, если он перейдет в  $C$ .

## От противного – (2)



- Единственный способ попасть в C [формально:  $\delta(A, w) = C$ ] будет, если  $w = x1y$ , где  $x$  приводит в B, и  $y$  есть хвост  $w$ , который последует после первого попадания в C.
- Если  $\delta(A, x) = B$  то, конечно,  $x = z1$  для некоторого  $z$ .
- Т.о.,  $w = z11y$  и имеет 11.

# Недетерминированный конечный автомат

Недетерминизм

Конструкция подмножеств  
ε-переходы

# Недетерминизм

- *Недетерминированный конечный автомат* может находиться в нескольких состояниях одновременно.
- Переходы из состояния по входному символу могут быть в любое множество состояний.
- Начать работу в начальном состоянии.
- Принять, если какая-либо последовательность переходов приводит в заключительное состояние.
- **Интуитивно**: НКА всегда “угадывает правильно.”

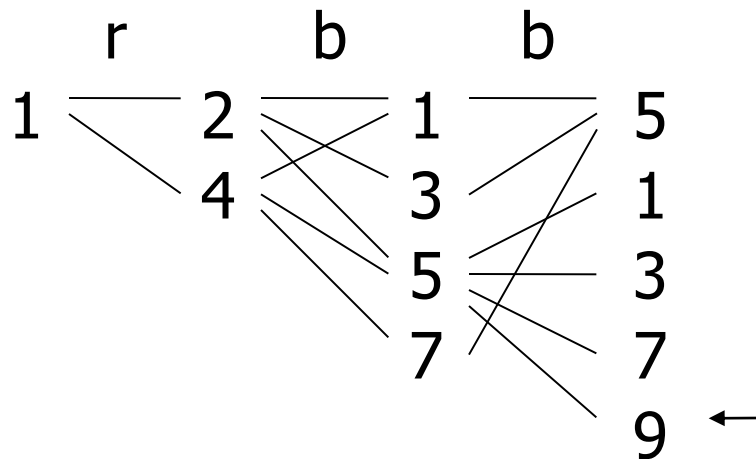


# Пример: Передвижения на шахматной доске

- Состояния = квадраты.
- Входы =  $r$  (перейти в смежный красный квадрат) и  $b$  (перейти в смежный чёрный квадрат).
- Начальное состояние, конечное состояние в противоположных углах.

# Пример: шахматная доска – (2)

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |



|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| * 9 | 6,8     | 5       |

Принять, т.к. финальное состояние достигнуто

# Формальное определение НКА

- Конечное множество состояний, обычно  $Q$ .
- Входной алфавит, обычно  $\Sigma$ .
- Функция переходов, обычно  $\delta$ .
- Начальное состояние в  $Q$ , обычно  $q_0$ .
- Множество конечных состояний  $F \subseteq Q$ .

$$A = (Q, \Sigma, \delta, q_0, F)$$

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

# Функция переходов НКА

- $\delta(q, a)$  есть множество состояний.
- Расширим функцию на строки следующим образом:
- **Базис**:  $\delta(q, \epsilon) = \{q\}$
- **Индукция**:  $\delta(q, wa) = \text{объединение } \delta(p, a) \text{ по всем}$   
состояниям  $p$  в  $\delta(q, w)$   
 $p \rightarrow \delta(p, a) \rightarrow \delta(q', w)$   
 $\dots \rightarrow \delta(q'', w)$   
 $\rightarrow \delta(p', a) \rightarrow \delta(q''', w)$

$$\delta(q, wa) = \bigcup_{p \in \delta(q, w)} \delta(p, a)$$

# Язык НКА

- Строка  $w$  принимается НКА если  $\delta(q_0, w)$  содержит, по крайней мере, одно финальное состояние.
- Язык НКА – это множество всех принимаемых строк.

## Пример: Язык НКА

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

- Для нашей доски и НКА мы видели, что строка 'rbb' принимается.
- Если вход состоит только из b, множество принимаемых состояний изменяется между  $\{5\}$  и  $\{1,3,7,9\}$ , так что только строки четной длины, из b принимаются.
- Что относительно строк по крайней мере с одним r?

# Эквивалентность КДА и НКА

- КДА может быть преобразован в НКА который принимает тот же язык.
- Если  $\delta_D(q, a) = p$ , то пусть НКА имеет  $\delta_N(q, a) = \{p\}$ .
- Тогда НКА всегда находится в множестве, состоящем точно из одного состояния – состояния КДА, в котором он находится после чтения того же входа.

## Эквивалентность – (2)

- Удивительно, но для любого НКА существует КДА, принимающий тот же язык.
- Доказательство базируется на *конструкции подмножеств*.
- Число состояний КДА может быть экспоненциальным по отношению к числу состояний НКА.
- Т.о., НКА принимают точно те же регулярные языки.



# Конструкция подмножеств

- По данному НКА с состояниями  $Q$ , входами  $\Sigma$ , функцией переходов  $\delta_N$ , начальным состоянием  $q_0$ , и финальными состояниями  $F$ , построим эквивалентный КДА  $u$  которого:
  - Состояния  $2^Q$  (Подмножества множества  $Q$ ).
  - Входы  $\Sigma$ .
  - Начальное состояние  $\{q_0\}$ .
  - Финальные состояния = все те, в которых есть состояния из  $F$ .

# Замечание

- Состояния КДА имеют *имена* , которые обозначают множества состояний НКА.
- Но как говорит определение КДА, выражение подобное  $\{p,q\}$  должно пониматься как один символ, а не множество.
- **Аналогия**: Класс объектов, значения которых являются множества объектов другого класса.

## Конструкция подмножеств – (2)

□ Функция переходов  $\delta_D$  определяется так:

$\delta_D(\{q_1, \dots, q_k\}, a)$  есть объединение по всем  $i = 1, \dots, k$  для  $\delta_N(q_i, a)$ :

$$\delta_D(\{q_1, \dots, q_k\}, a) = \bigcup_{i=1}^k \delta_N(q_i, a)$$

□ **Пример:** Мы построим КДА эквивалентный нашему «клеточному» НКА.

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| * 9 | 6,8     | 5       |

|       | r     | b   |
|-------|-------|-----|
| → {1} | {2,4} | {5} |
| {2,4} |       |     |
| {5}   |       |     |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

**NB!** То, что мы здесь делаем – это *ленивая* форма построения КДА, когда мы строим новое состояние только когда это необходимо.

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| *9  | 6,8     | 5       |

|           | r         | b         |
|-----------|-----------|-----------|
| → {1}     | {2,4}     | {5}       |
| {2,4}     | {2,4,6,8} | {1,3,5,7} |
| {5}       |           |           |
| {2,4,6,8} |           |           |
| {1,3,5,7} |           |           |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| *9  | 6,8     | 5       |

|             | r         | b         |
|-------------|-----------|-----------|
| → {1}       | {2,4}     | {5}       |
| {2,4}       | {2,4,6,8} | {1,3,5,7} |
| {5}         | {2,4,6,8} | {1,3,7,9} |
| {2,4,6,8}   |           |           |
| {1,3,5,7}   |           |           |
| * {1,3,7,9} |           |           |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| *9  | 6,8     | 5       |

|               | r         | b           |
|---------------|-----------|-------------|
| → {1}         | {2,4}     | {5}         |
| {2,4}         | {2,4,6,8} | {1,3,5,7}   |
| {5}           | {2,4,6,8} | {1,3,7,9}   |
| {2,4,6,8}     | {2,4,6,8} | {1,3,5,7,9} |
| {1,3,5,7}     |           |             |
| * {1,3,7,9}   |           |             |
| * {1,3,5,7,9} |           |             |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| *9  | 6,8     | 5       |

|               | r         | b           |
|---------------|-----------|-------------|
| → {1}         | {2,4}     | {5}         |
| {2,4}         | {2,4,6,8} | {1,3,5,7}   |
| {5}           | {2,4,6,8} | {1,3,7,9}   |
| {2,4,6,8}     | {2,4,6,8} | {1,3,5,7,9} |
| {1,3,5,7}     | {2,4,6,8} | {1,3,5,7,9} |
| * {1,3,7,9}   |           |             |
| * {1,3,5,7,9} |           |             |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |



# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| * 9 | 6,8     | 5       |

|               | r         | b           |
|---------------|-----------|-------------|
| → {1}         | {2,4}     | {5}         |
| {2,4}         | {2,4,6,8} | {1,3,5,7}   |
| {5}           | {2,4,6,8} | {1,3,7,9}   |
| {2,4,6,8}     | {2,4,6,8} | {1,3,5,7,9} |
| {1,3,5,7}     | {2,4,6,8} | {1,3,5,7,9} |
| * {1,3,7,9}   | {2,4,6,8} | {5}         |
| * {1,3,5,7,9} |           |             |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

# Пример: Конструкция подмножеств

|     | r       | b       |
|-----|---------|---------|
| → 1 | 2,4     | 5       |
| 2   | 4,6     | 1,3,5   |
| 3   | 2,6     | 5       |
| 4   | 2,8     | 1,5,7   |
| 5   | 2,4,6,8 | 1,3,7,9 |
| 6   | 2,8     | 3,5,9   |
| 7   | 4,8     | 5       |
| 8   | 4,6     | 5,7,9   |
| * 9 | 6,8     | 5       |

|               | r         | b           |
|---------------|-----------|-------------|
| → {1}         | {2,4}     | {5}         |
| {2,4}         | {2,4,6,8} | {1,3,5,7}   |
| {5}           | {2,4,6,8} | {1,3,7,9}   |
| {2,4,6,8}     | {2,4,6,8} | {1,3,5,7,9} |
| {1,3,5,7}     | {2,4,6,8} | {1,3,5,7,9} |
| * {1,3,7,9}   | {2,4,6,8} | {5}         |
| * {1,3,5,7,9} | {2,4,6,8} | {1,3,5,7,9} |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

# Доказательство эквивалентности: конструкция подмножеств

- Доказательство:
- Покажем индукцией по  $|w|$ , что
$$\delta_N(q_0, w) = \delta_D(\{q_0\}, w)$$
- **Базис:**  $w = \epsilon$ :  $\delta_N(q_0, \epsilon) = \delta_D(\{q_0\}, \epsilon) = \{q_0\}$ .

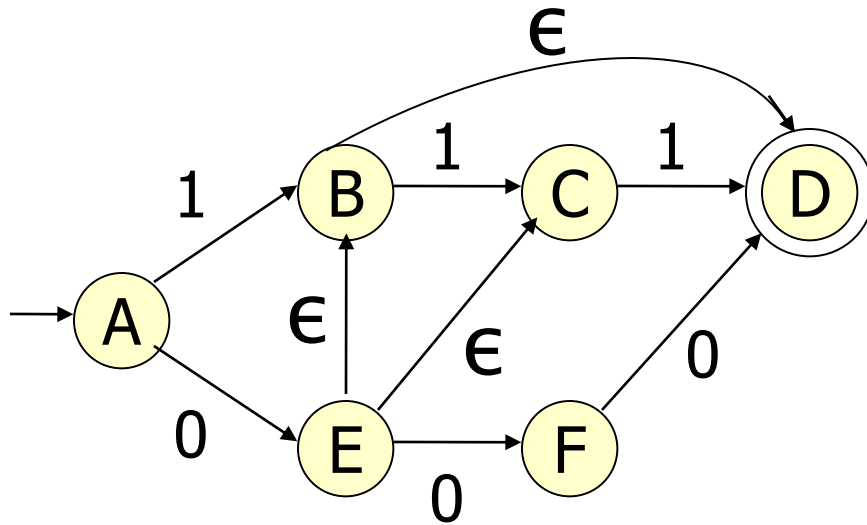
# Индукция

- Пусть  $IH$  выполняется для строк короче, чем для  $w$ .
- Пусть  $w = xa$ ;  $IH$  выполняется для  $x$ .
- Пусть  $\delta_N(q_0, x) = \delta_D(\{q_0\}, x) = S$ .
- Пусть  $T = \text{объединение } \delta_N(p, a) \text{ по всем состояниям } p \text{ в } S$ .
- Тогда  $\delta_N(q_0, w) = \delta_D(\{q_0\}, w) = T$ .

# НКА с $\epsilon$ -переходами

- Мы можем допустить переходы между состояниями по входу  $\epsilon$ .
- Эти переходы происходят спонтанно, без оглядки на входную строку.
- Иногда это удобно, но принимаются те же регулярные языки, что принимаются КДА и НКА.

## Пример: $\epsilon$ -НКА



|               |   | 0           | 1           | $\epsilon$  |
|---------------|---|-------------|-------------|-------------|
| $\rightarrow$ | A | {E}         | {B}         | $\emptyset$ |
|               | B | $\emptyset$ | {C}         | {D}         |
|               | C | $\emptyset$ | {D}         | $\emptyset$ |
| *             | D | $\emptyset$ | $\emptyset$ | $\emptyset$ |
|               | E | {F}         | $\emptyset$ | {B, C}      |
|               | F | {D}         | $\emptyset$ | $\emptyset$ |

$$A = (Q, \Sigma, \delta_E, q_0, F), \quad \delta_E: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

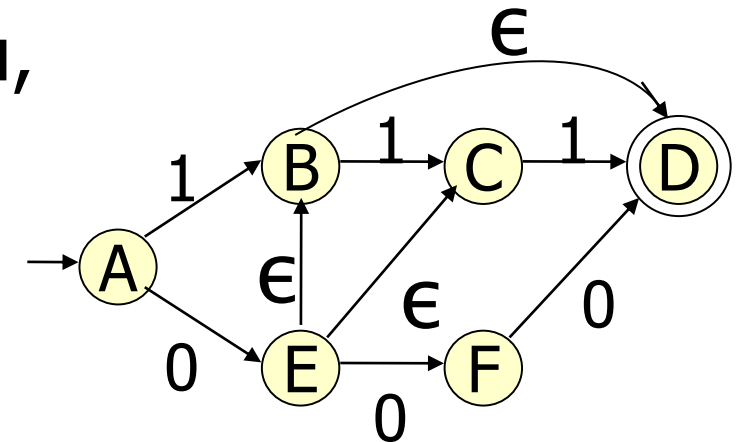
# Замыкание для состояний

□  $CL(q)$  = множество состояний, которые мы можем достичь из состояния  $q$ , следуя по дугам, помеченным  $\epsilon$ .

□ **Пример:**  $CL(A) = \{A\}$ ;  
 $CL(E) = \{B, C, D, E\}$ .

□ Замыкание множества состояний = объединение множеств замыканий по каждому из состояний множества.

□  $CL(\{B, C, D, E\}) = CL(B, D) \cup CL(C) \cup CL(D) \cup CL(E) =$   
 $= \{D\} \cup \emptyset \cup \emptyset \cup \{B, C, D, E\} = \{B, C, D, E\}$

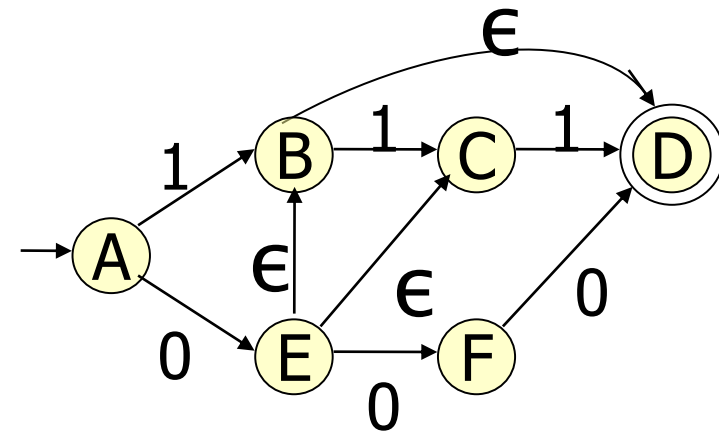


# Расширенная дельта

- **Интуитивно:**  $\hat{\delta}(q, w)$  – множество состояний, которые можно достичь из  $q$ , следуя по пути, помеченным  $w$ . [NB! :  $\hat{\delta}(q, \epsilon) \neq \delta(q, \epsilon)$  ]
- **Базис:**  $\hat{\delta}(q, \epsilon) = CL(q)$ .
- **Индукция:**  $\hat{\delta}(q, xa)$  вычисляется так:  
Начать с  $\hat{\delta}(q, x) = S$ .  
Возьмём объединение  $CL(\delta(p, a))$   
по всем  $p$  из  $S$ .



# Пример: Расширение дельты

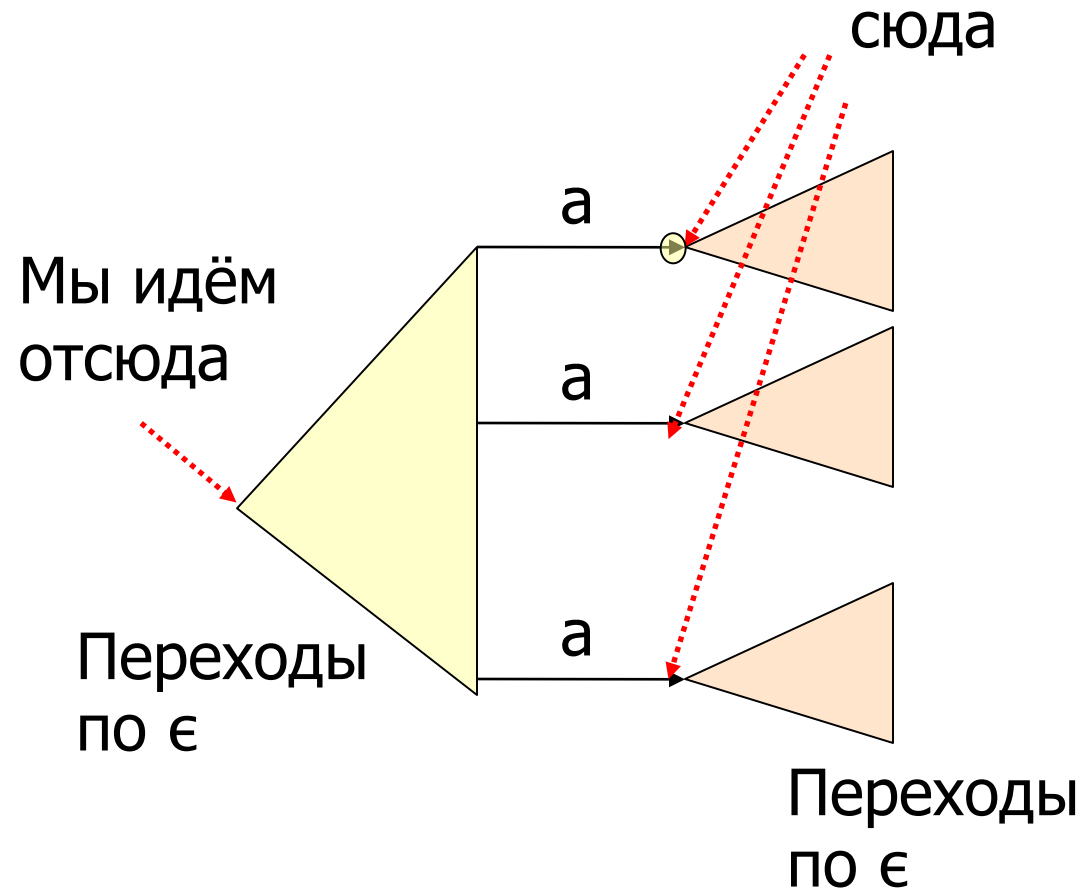


- $\hat{\delta}(A, \epsilon) = CL(A) = \{A\}.$
- $\hat{\delta}(A, 0) = CL(\{E\}) = \{B, C, D, E\}.$
- $\hat{\delta}(A, 01) = CL(\{C, D\}) = \{C, D\}.$
- **Языком**  $\epsilon$ -НКА является множество строк  $w$  такое, что  $\hat{\delta}(q_0, w)$  содержит финальное состояние.
- $A = (Q, \Sigma, \delta_E, q_0, F), \quad \delta_E: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
- $L(A) = \{w: \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$

# Эквивалентность НКА, $\epsilon$ -НКА

- Каждый НКА является  $\epsilon$ -НКА.
  - Он только не имеет  $\epsilon$ -переходов.
- Доказательство обратного требует от нас взять  $\epsilon$ -НКА и построить НКА, который принимает тот же язык.
- Мы сделаем это путем комбинирования  $\epsilon$ -переходов со следующим переходом по реальному входу.

# Иллюстрация удаления $\epsilon$ -переходов



Поскольку мы не выполняем замыкания состояний после перехода по «а», мы должны сделать дополнительные конечные состояния - те, из которых можно достичь конечного состояния только по  $\epsilon$ .

## Эквивалентность – (2)

- Возьмем  $\epsilon$ -НКА с состояниями  $Q$ , входами  $\Sigma$ , начальным состоянием  $q_0$ , финальными состояниями  $F$ , и функцией переходов  $\delta_E$ .
- Построим “обычный” НКА с состояниями  $Q$ , входами  $\Sigma$ , начальным состоянием  $q_0$ , финальными состояниями  $F'$ , и функцией переходов  $\delta_N$ .

## Эквивалентность – (3)

- Будем вычислять  $\delta_N(q, a)$  следующим образом:

Пусть  $S = CL(q)$ .

$\delta_N(q, a)$  есть объединение  $\delta_E(p, a)$  по всем  $p$  в  $S$ :

$$\delta_N(p, a) = \bigcup_{p \in S} \delta_E(p, a)$$

- $F' =$  множество состояний  $q$  таких, что  $CL(q)$  содержит состояние из  $F$ .

## Эквивалентность – (4)

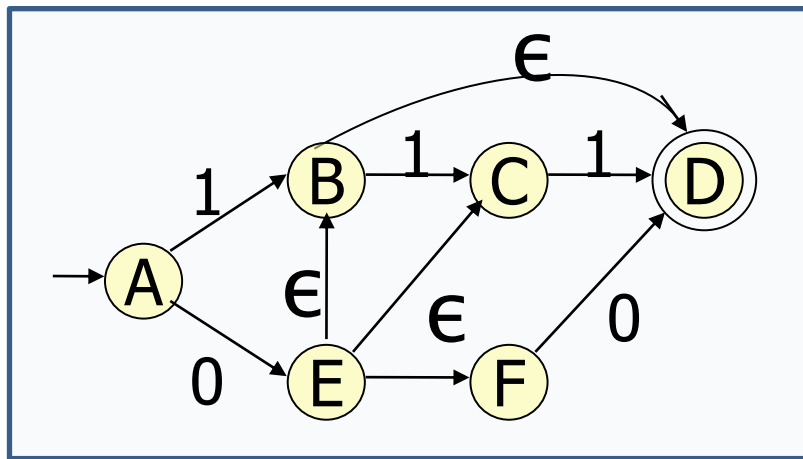
□ Доказательство индукцией по  $|w|$  того, что

$$CL(\delta_N(q_0, w)) = \overset{\wedge}{\delta}_E(q_0, w). \quad (1)$$

Идея доказательства: НКА на любом входе  $w$  переходит в тот же набор состояний, в который переходит  $\epsilon$ -НКА на том же входе, используя, где возможно,  $\epsilon$ -переходы, кроме как в конце - после чтения всех реальных входов  $w$ . Однако, состояние  $p$  в  $\overset{\wedge}{\delta}_E(q_0, w)$  является конечным состоянием, если в  $\epsilon$ -НКА из  $p$  можно достичь конечного состояния, следуя только по  $\epsilon$ . Таким образом, тот факт, что равенство (1) выполняется, означает, что  $w$  принимается или не принимается обоими автоматами; то есть они принимают один и тот же язык.

□ Т.о.,  $\epsilon$ -НКА принимает  $w$  т.и т.т., когда его принимает “обычный” НКА.

# Пример: от $\epsilon$ -НКА к НКА



$\epsilon$ -НКА

|                 | 0           | 1           | $\epsilon$  |
|-----------------|-------------|-------------|-------------|
| $\rightarrow$ A | {E}         | {B}         | $\emptyset$ |
| B               | $\emptyset$ | {C}         | {D}         |
| C               | $\emptyset$ | {D}         | $\emptyset$ |
| * D             | $\emptyset$ | $\emptyset$ | $\emptyset$ |
| E               | {F}         | $\emptyset$ | {B, C}      |
| F               | {D}         | $\emptyset$ | $\emptyset$ |

Т.к. замыкания В и Е включают финальное состояние D.

|                 | 0           | 1           |
|-----------------|-------------|-------------|
| $\rightarrow$ A | {E}         | {B}         |
| * B             | $\emptyset$ | {C}         |
| * C             | $\emptyset$ | {D}         |
| * D             | $\emptyset$ | $\emptyset$ |
| E               | {F}         | {C, D}      |
| F               | {D}         | $\emptyset$ |

Т.к. замыкание Е включает В и С, которые имеют переходы по 1 к С и D.

НЕ меняется, т.к. В, С, D не имеют переходов по 0.

Интересующие замыкания:

$$CL(B) = \{B, D\};$$

$$CL(E) = \{B, C, D, E\}$$

# Заключение

- Теорема. КДА, НКА, и  $\epsilon$ -НКА все принимают в точности одно и то же множество языков: регулярные языки.
- По этой причине эти языки ещё называют *автоматными*.
- Типы НКА проще строить и они могут иметь экспоненциально меньше состояний, чем КДА.
- Но только КДА может быть реализован!



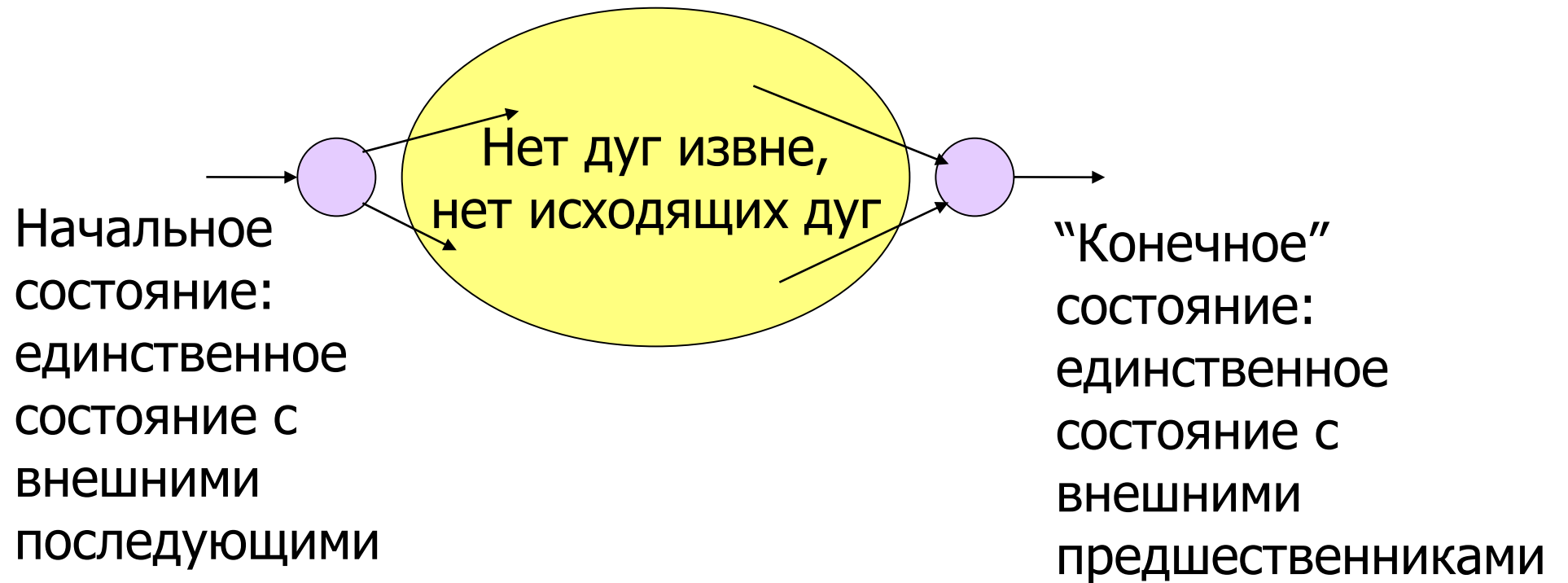
# Эквивалентность РВ и конечных автоматов

- **Теорема**. Регулярные выражения и конечные автоматы представляют один и тот класс регулярных языков.
- Нужно показать, что для каждого РВ существует конечный автомат, который принимает тот же самый язык, что определяет РВ.
  - Для доказательства возьмём наиболее мощный тип автомата:  $\epsilon$ -НКА.
- Также, нужно показать, что для каждого конечного автомата существует РВ, определяющее тот же язык, что КА
  - Для доказательства выберем наиболее ограничительный тип: КДА.

# Преобразование РВ в $\epsilon$ -НКА

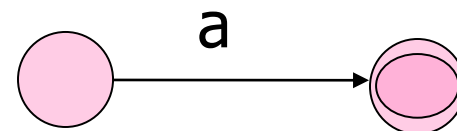
- Докажем индукцией по числу операторов (+, конкатенация, \*) в РВ.
- Мы всегда строим автомат специального вида (см. следующий слайд).

# Вид построенного $\epsilon$ -НКА

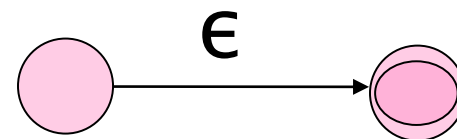


# От РВ к $\epsilon$ -НКА: **Базис**

□ Symbol **a**:



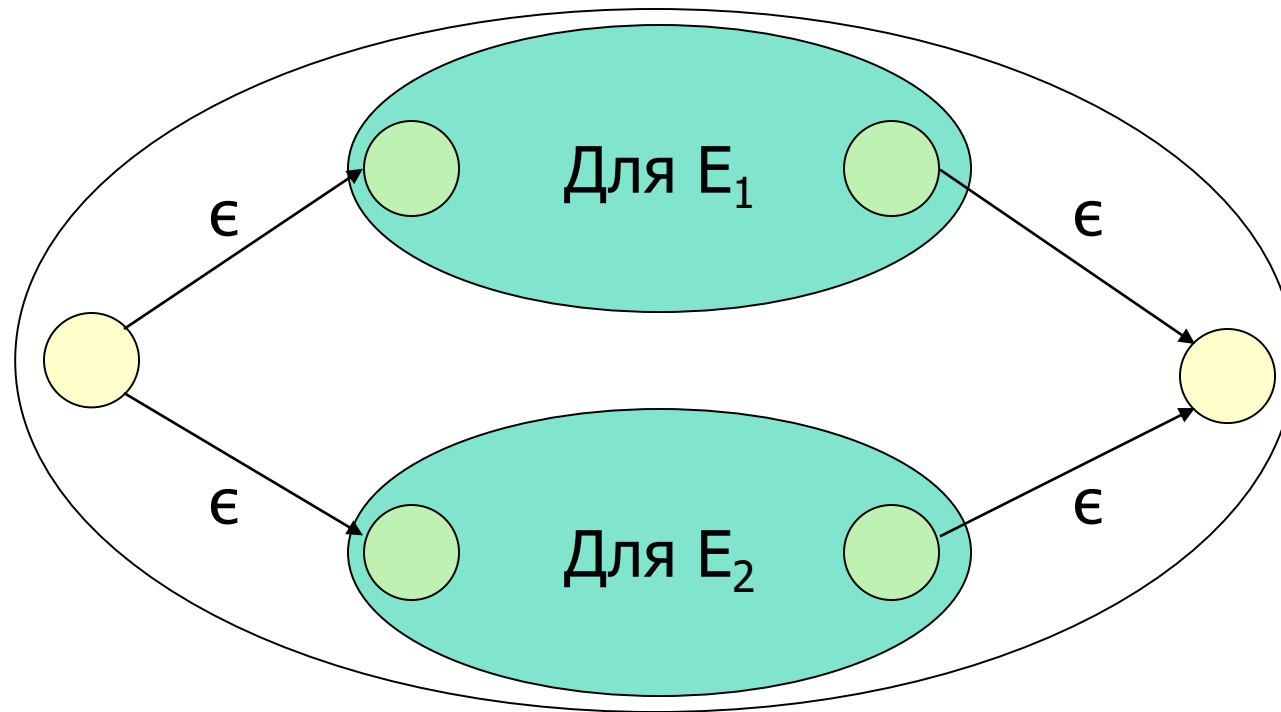
□  $\epsilon$ :



□  $\emptyset$ :

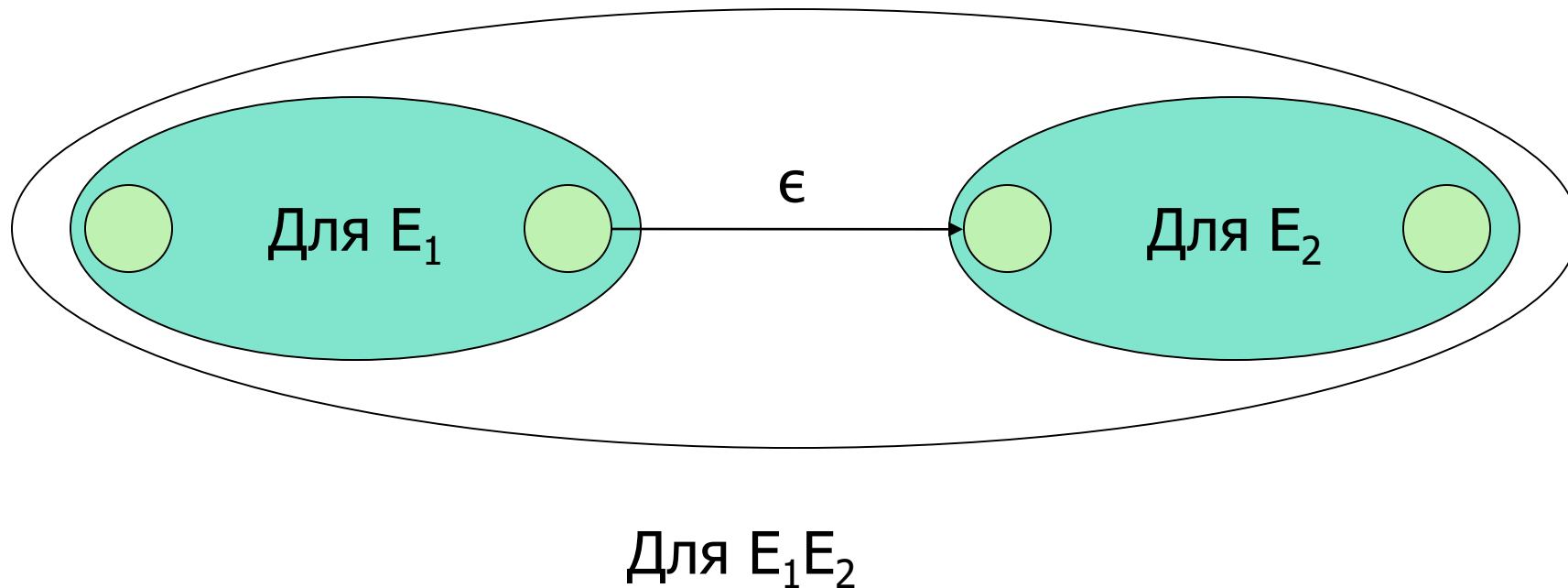


# От РВ к $\epsilon$ -НКА: Индукция 1 – Объединение

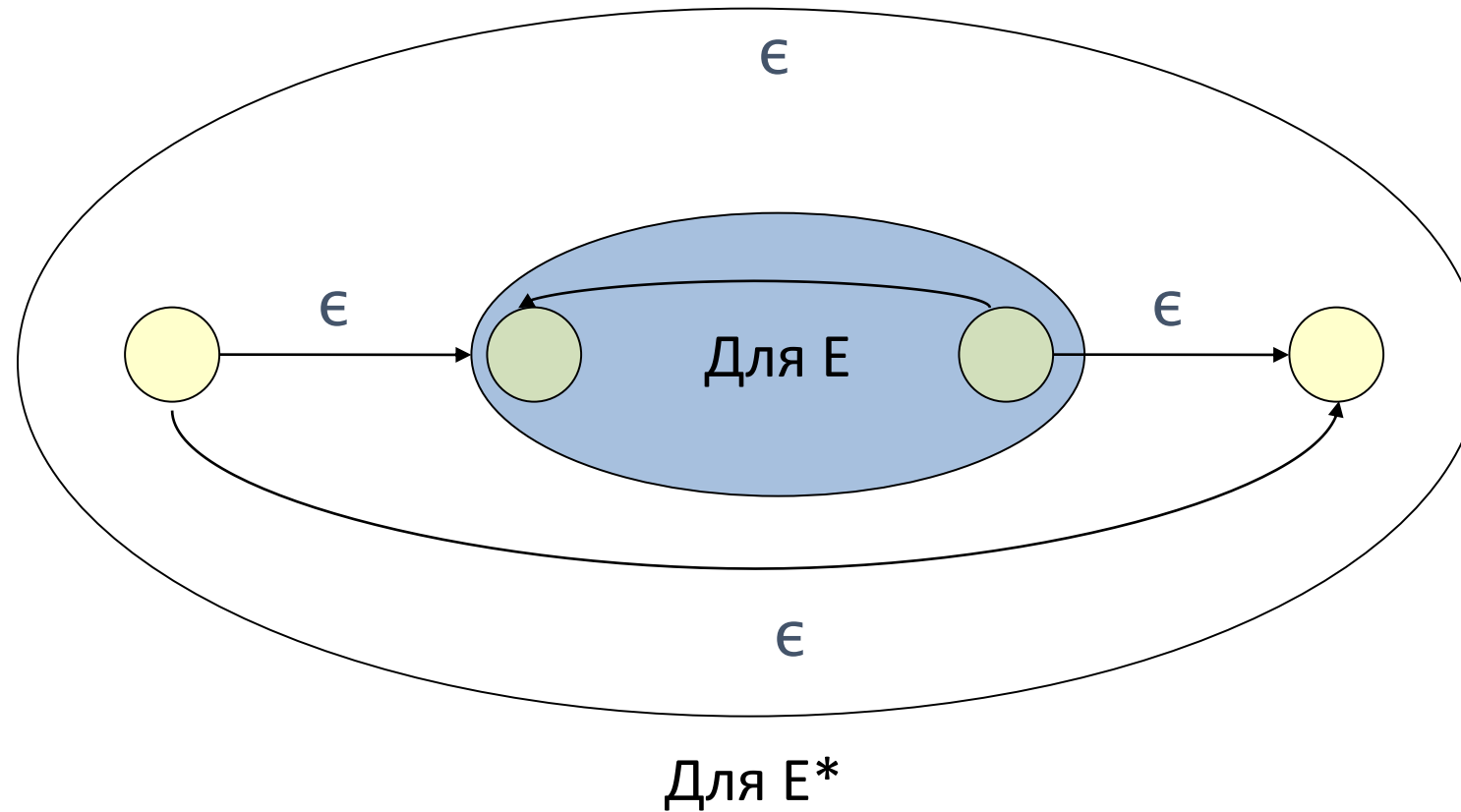


Для  $E_1 \cup E_2$

# От РВ к $\epsilon$ -НКА: Индукция 2 – Конкатенация

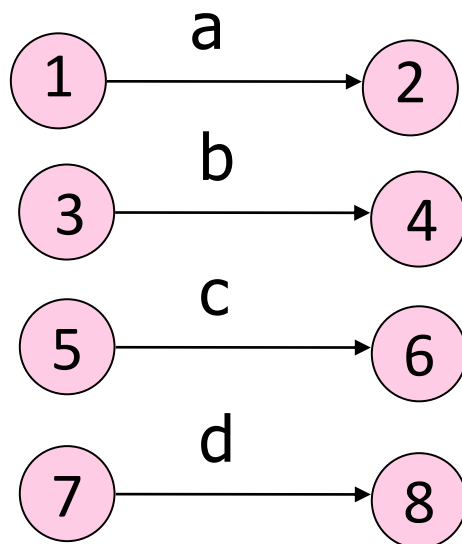


# От РВ к $\epsilon$ -НКА: Индукция 3 – Замыкание



# Пример построения КА по РВ

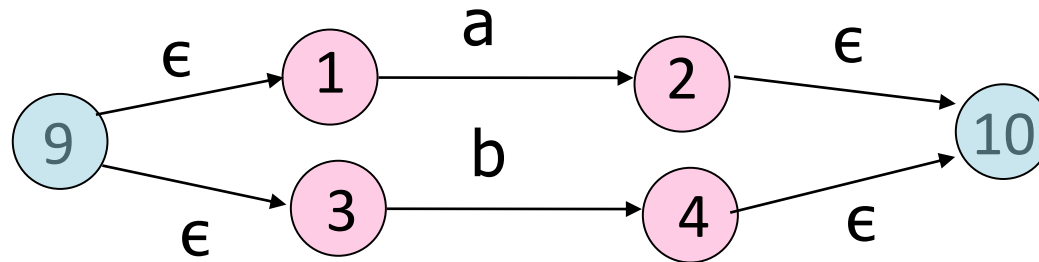
- Регулярное выражение:  $(a+b)(cd)^*$
- 1) Автоматы для РВ  $a, b, c, d$



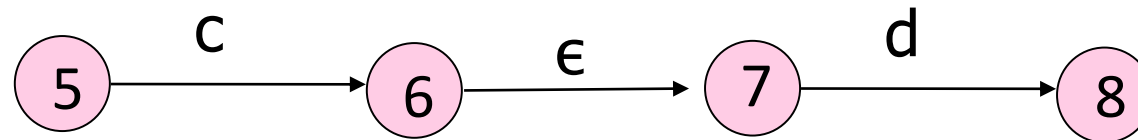


# Пример построения КА по РВ (2)

- Регулярное выражение:  $(a+b)(cd)^*$
- 2) Автомат для РВ  $a+b$

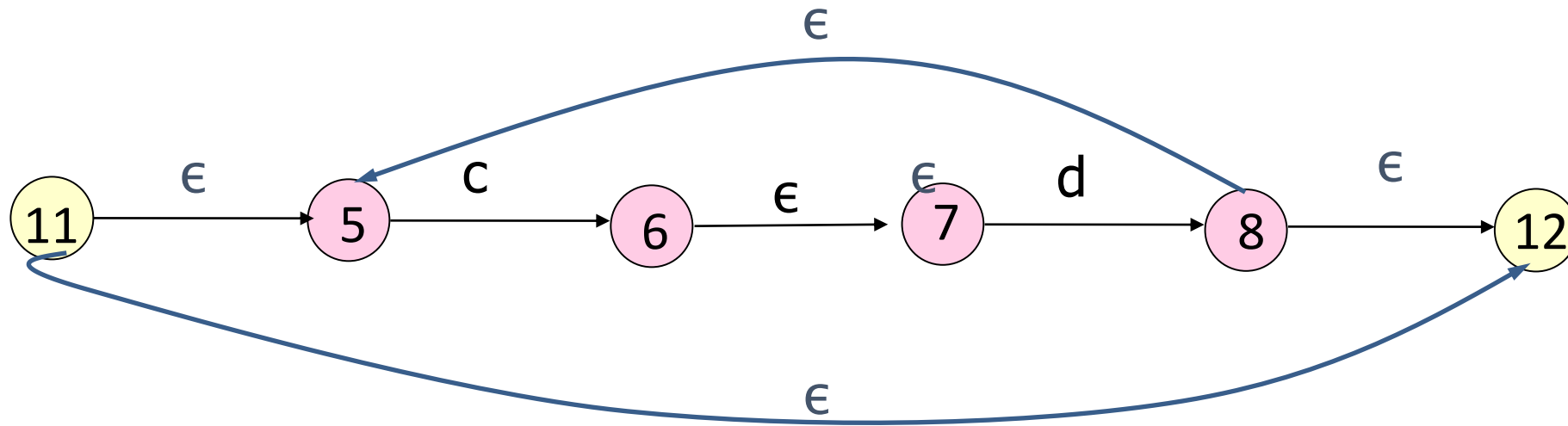


- 3) Автомат для РВ  $cd$



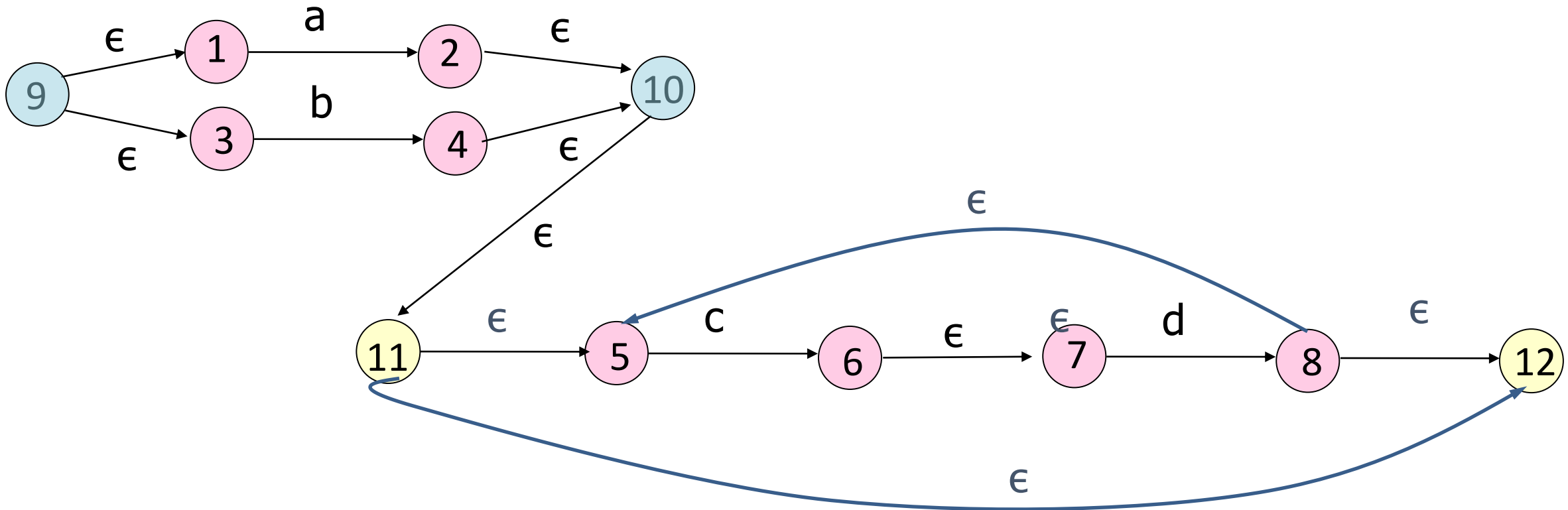
# Пример построения КА по РВ (3)

- Регулярное выражение:  $(a+b)(cd)^*$
- 4) Автомат для РВ  $(cd)^*$



# Пример построения КА по РВ (4)

- Регулярное выражение:  $(a+b)(cd)^*$
- 5) Автомат для РВ  $(a+b)(cd)^*$



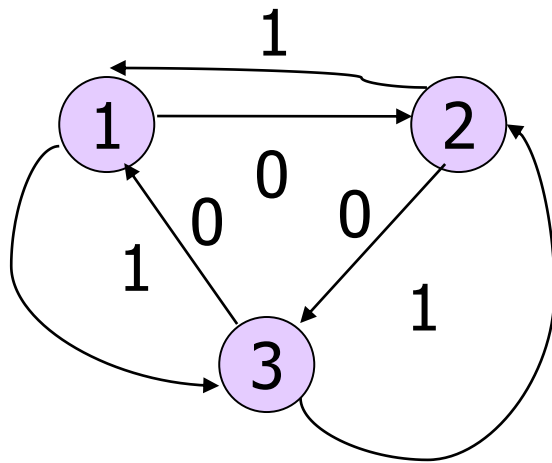
# От-ДКА-к-РВ

- Необычный вид индукции.
- Пусть состояния КДА именуются  $1, 2, \dots, n$ .
- Индукция проводится по  $k$ , максимальному числу состояний, по которым нам позволено проходить вдоль пути.

# к-Пути

- К-путь – это путь в графе КДА, который **не** проходит **через** состояния с номерами больше, чем  $k$ .
- Выбор конечной вершины не ограничивается; ей может быть любое состояние.
- $n$ -пути являются неограниченными ( $n$  – число состояний автомата).
- РВ есть объединение РВ для  $n$ -путей от начального состояния к каждому конечному состоянию.

# Пример: k-пути



3-пути от 2 к 3:  
РВ для меток = ??

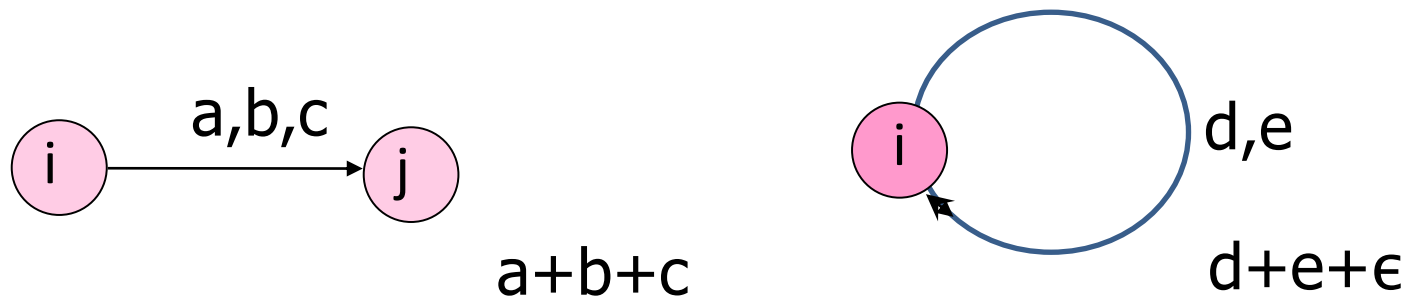
0-пути от 2 к 3:  
РВ для меток = **0**.

1-пути от 2 к 3:  
РВ для меток = **0+11**.

2-пути от 2 к 3:  
РВ для меток =  
**(10)\*0+1(01)\*1**

# От-КДА-к-РВ

- **Базис**:  $k = 0$  (нет промежуточных состояний); только метки дуги или (если  $i=j$ )  $\epsilon$ .
- Составляем РВ:  $a+b+\dots + (\text{м.б.}) \in$
- **Индукция**: строим РВ для путей, позволяющих проход через состояние  $k$  от путей, позволяющих проход через состояния с номерами от 1 до  $k-1$ .



# Индукция по k-пути

- Пусть  $R_{ij}^k$  будет РВ для множества меток k-путей от состояния  $i$  до состояния  $j$ .
- **Базис:**  $k=0$ .  $R_{ij}^0$  = сумма меток дуг от  $i$  к  $j$ .
  - $\emptyset$  если таких дуг нет.
  - но добавим  $\epsilon$  если  $i=j$ .

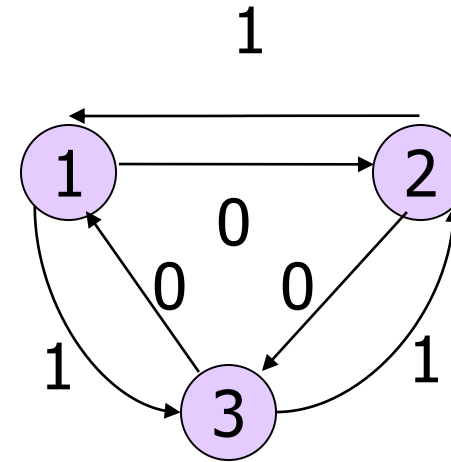


# Пример: Базис

$$\square R_{12}^0 = \mathbf{0}.$$

$$\square R_{11}^0 = \emptyset + \epsilon = \epsilon.$$

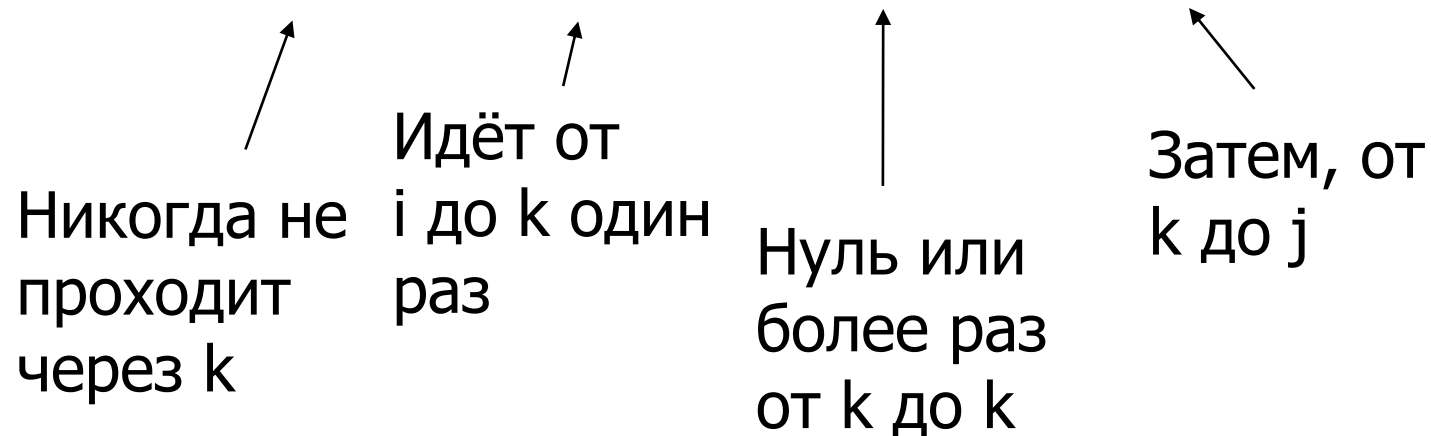
См. алгебраические  
тождества для РВ:  
 $\emptyset$  плюс "что-то" =  
этому "что-то".



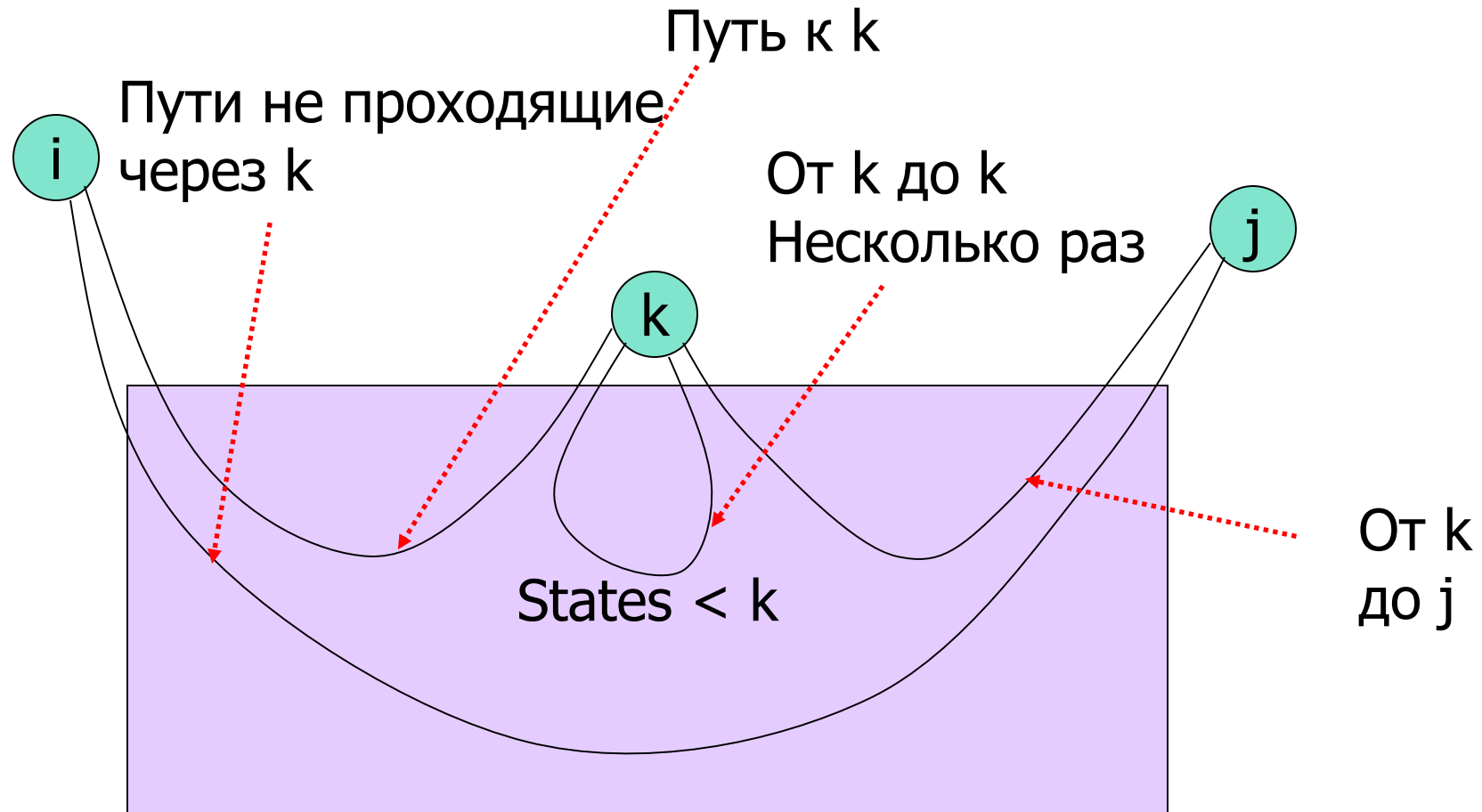
# k-пути Индуктивный переход

- k-путь от  $i$  до  $j$  - это путь, который либо: никогда не проходит через состояние  $k$ , либо идет через  $k$  один или более раз.

$$R_{ij}^k = R_{ij}^{k-1} + R_{ik}^{k-1}(R_{kk}^{k-1})^* R_{kj}^{k-1}.$$



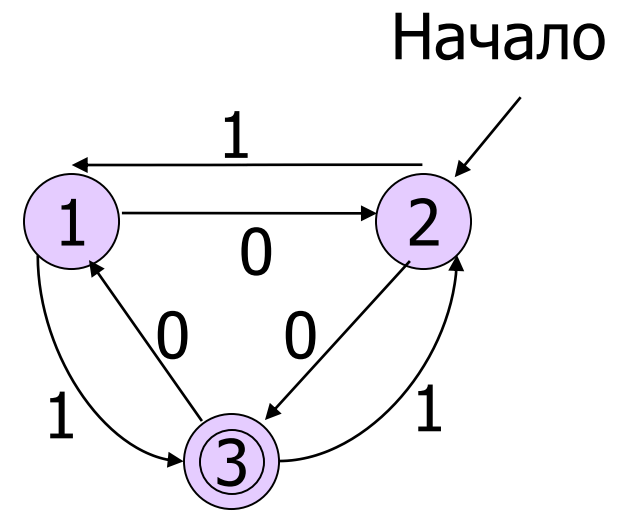
# Иллюстрация Индукции



# Заключительный шаг

- РВ с тем же языком, что и у КДА, есть сумма (объединение)  $R_{ij}^n$ , где:
  - $n$  – число состояний; т.е., пути не ограничены.
  - $i$  – начальное состояние.
  - $j$  – одно из заключительных состояний.

# Пример



$$\square R_{23}^3 = R_{23}^2 + R_{23}^2(R_{33}^2)^*R_{33}^2 = R_{23}^2(R_{33}^2)^*$$

$$\square R_{23}^2 = (\mathbf{10})^*\mathbf{0} + \mathbf{1}(\mathbf{01})^*\mathbf{1}$$

$$\square R_{33}^2 = \epsilon + \mathbf{0}(\mathbf{01})^*(\mathbf{1} + \mathbf{00}) + \mathbf{1}(\mathbf{10})^*(\mathbf{0} + \mathbf{11})$$

$$\square R_{23}^3 = [(\mathbf{10})^*\mathbf{0} + \mathbf{1}(\mathbf{01})^*\mathbf{1}] [\epsilon + (\mathbf{0}(\mathbf{01})^*(\mathbf{1} + \mathbf{00}) + \mathbf{1}(\mathbf{10})^*(\mathbf{0} + \mathbf{11}))]^*$$

# Промежуточные итоги

- Каждый из рассмотренных трёх типов автоматов (КДА, НКА,  $\epsilon$ -НКА), а также регулярные выражения, определяют одно и то же множество языков: регулярные языки.

РВ  $\rightarrow$   $\epsilon$ -НКА  $\rightarrow$  НКА  $\rightarrow$  ДКА  $\rightarrow$  РВ

# Эквивалентность автоматных и праволинейных языков

- Лемма. Если  $L=L(A)$  для некоторого конечного автомата  $A$ , то  $L=L(G)$  для некоторой праволинейной грамматики  $G$ .
- Док-во. Пусть  $A = (Q, \Sigma, \delta, q_0, F)$  – КДА. Определим грамматику  $G = (Q, \Sigma, P, q_0)$ , где  $P$  имеет вид:

Если  $\delta(q, a) = r$ , то  $P$  содержит правило  $q \rightarrow ar$ .

Если  $p \in F$ , то  $P$  содержит правило  $p \rightarrow \varepsilon$ .

Каждый шаг вывода в грамматике  $G$  имитирует такт работы автомата  $A$ .

Индукция по  $i$  – длине вывода.

**Базис.**  $i = 0$ .  $q \Rightarrow \varepsilon$  т.и.т.т., когда  $(q, \varepsilon) \vdash^0 (q, \varepsilon)$ .

**ИН:**  $s \Rightarrow^i x$  т.и.т.т., когда  $(s, x) \vdash^{i-1} (r, \varepsilon)$  для некоторого  $r \in F$ .

**Шаг индукции.** Пусть  $w = ax$ , где  $|x| = i$ .

Тогда  $q \Rightarrow^{i+1} w$  равносильно тому, что  $q \Rightarrow as \Rightarrow^i ax$  для некоторого  $s$ .

Но  $q \Rightarrow as$  равносильно  $\delta(q, a) = s$  или  $(q, a) \vdash^1 (s, \varepsilon)$ .

Это означает, что  $(q, ax) \vdash^1 (s, x)$ .

По индукции  $s \Rightarrow^i x$  т.и.т.т., когда  $(s, x) \vdash^{i-1} (r, \varepsilon)$  для некоторого  $r \in F$ .

Следовательно,  $q \Rightarrow^{i+1} w$  равносильно  $(q, ax) \vdash^1 (s, x) \vdash^{i-1} (r, \varepsilon)$  или  $(q, w) \vdash^i (r, \varepsilon)$  для некоторого  $r \in F$ .

Ч. т. д.



- Лемма. Если  $L=L(G)$  для некоторой праволинейной грамматики  $G$ , то  $L=L(A)$  для некоторого конечного автомата  $A$ .
- Док-во. Пусть  $G = (Q, \Sigma, P, S)$  – праволинейная грамматика. Построим автомат  $A = (N \cup \{q_f\}, \Sigma, \delta, S, F)$ , где  $\delta$  определено как:

Если  $A \rightarrow aB \in P$ , то  $\delta(A, a) = B$  для  $A, B \in N$  и  $a \in \Sigma$ .

Если  $A \rightarrow a \in P$ , то  $\delta(A, a) = q_f$  для  $A \in N$  и  $a \in \Sigma$ .

$F = \{S, q_f\}$ , если в  $P$  есть  $S \rightarrow \varepsilon$  и  $F = \{q_f\}$  – в противном случае.

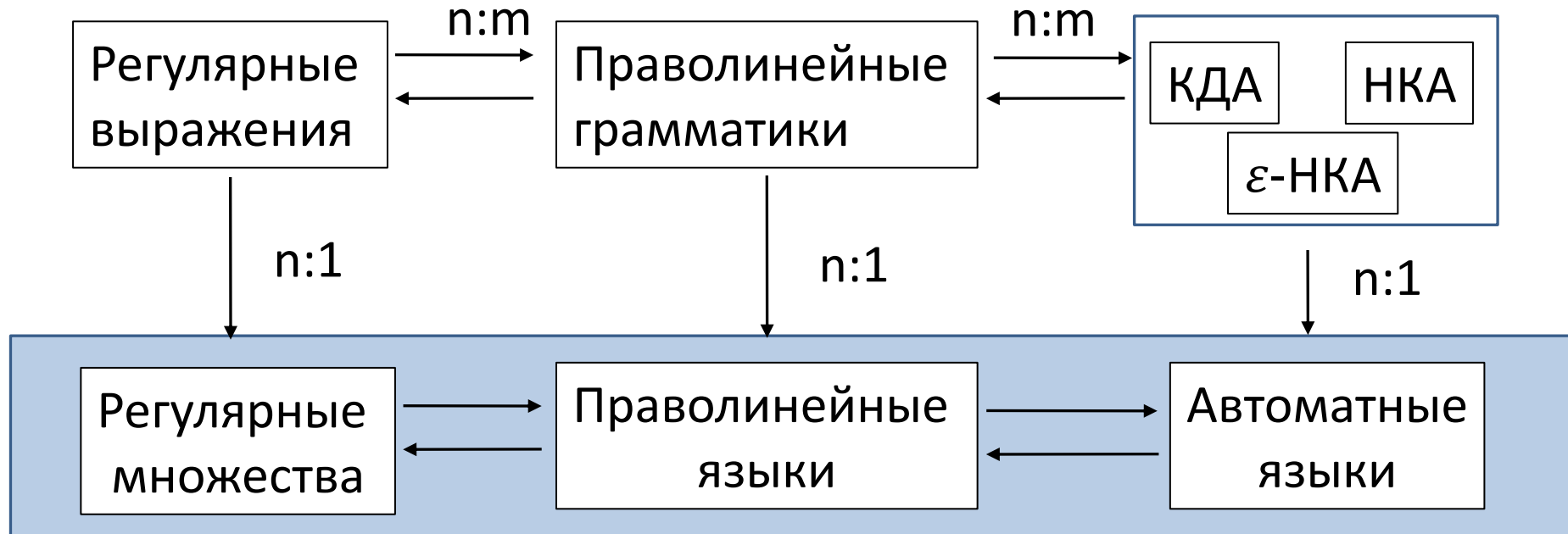
Очевидно, что построенный автомат определяет тот же язык, что и исходная праволинейная грамматика.

- Теорема. Язык праволинейный тогда и только тогда, когда он автоматный.
- Доказательство следует из двух предыдущих лемм.

# Заключение

- Каждый из рассмотренных трёх типов автоматов (КДА, НКА,  $\epsilon$ -НКА), а также регулярные выражения, определяют одно и то же множество языков: регулярные языки.
  - Теорема. Утверждения:
    - L – регулярный язык (регулярное множество),
    - L – праволинейный язык,
    - L – конечно-автоматный язык,
    - L – недетерминированный конечно-автоматный язык,
    - L обозначается (определяется) регулярным выражением
- эквивалентны.

РВ  $\rightarrow$   $\epsilon$ -НКА  $\rightarrow$  НКА  $\rightarrow$  ДКА  $\rightarrow$  РВ



Регулярные языки

# Свойства регулярных языков

# Свойства класса языков

- *Класс языков* - это множество языков.
- Пример: регулярные языки.
- Классы языков имеют два вида важных свойств:
  1. Разрешимые свойства.
  2. Свойства замкнутости.

# Свойства замкнутости

- *Свойство замкнутости* класса языков говорит, что выполнение некоторой операции над языками в классе (например, объединение) даёт в результате язык из того же класса.
- *Пример*: регулярные языки, очевидно, замкнуты относительно операций объединения, конкатенации и замыкания Клини (данные операции называются регулярными).
  - Использовать для доказательства представление регулярных языков регулярными выражениями.

# Представление языков

□ Представление может быть формальным и неформальным.

**Пример:** (формальный): представление языка РВ или КДА, определяющим соотв. язык.

**Пример:** (неформальный): логическое или описательное утверждение о его строках:

□  $\{0^n 1^n \mid n - \text{неотрицательное число}\}$

□ “Множество строк, состоящих из некоторого числа 0-ей, за которыми следует такое же число 1.”



# Разрешимое свойство

- *Разрешимое свойство* для класса языков – это алгоритм, который по формальному описанию языка (например, КДА) определяет выполнение или невыполнение некоторого свойства.
- *Пример*: Является ли язык  $L$  пустым?

# Почему важны разрешимые свойства?

□ Рассмотрим КДА, представляющий некоторый протокол.

**Пример:** “Завершается ли протокол?” = “Является ли язык конечным?”

**Пример:** “Может ли протокол быть неверным?” = “Является ли язык непустым?”

□ Сделать финальным состоянием “состояние ошибки”.

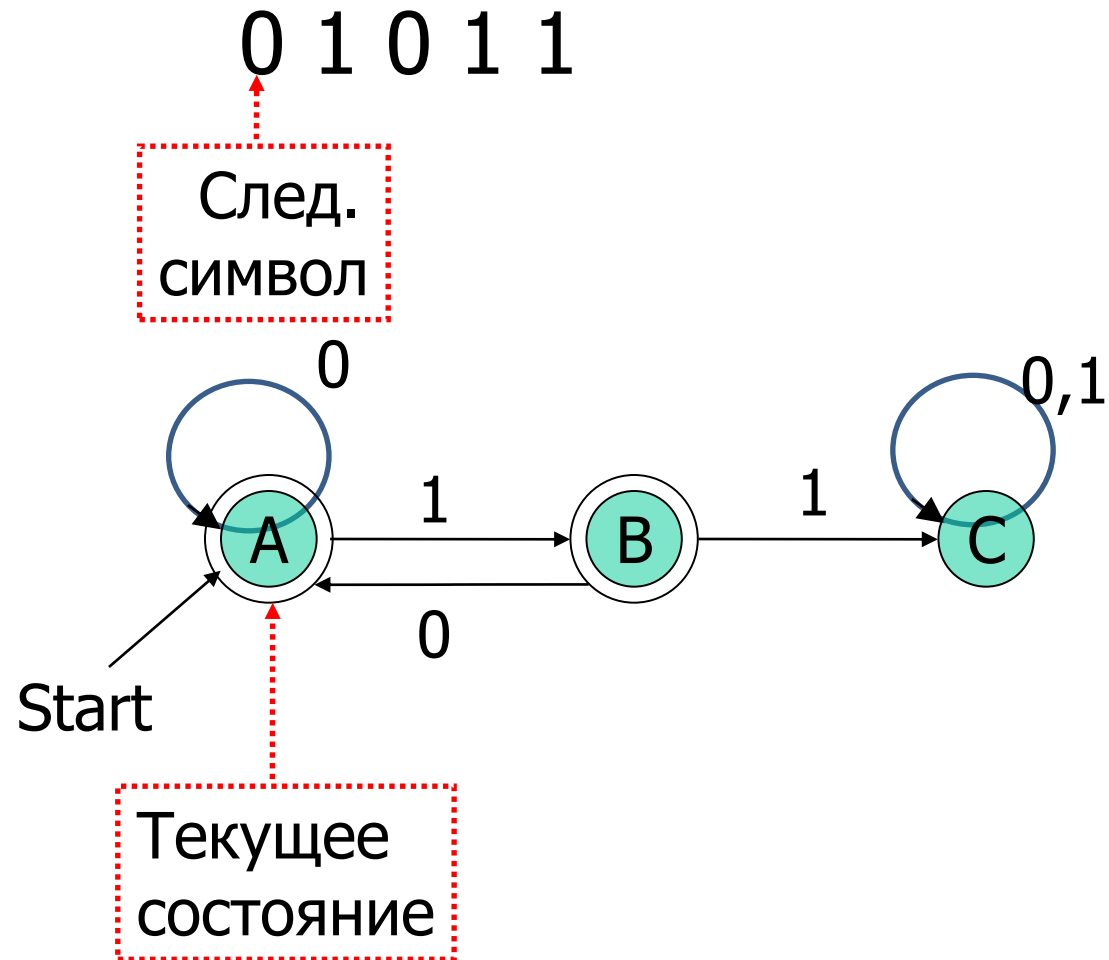
## Почему важны разрешимые свойства – (2)

- Нам хотелось бы иметь “наименьшее” представление языка, т.е., КДА с минимальным числом состояний или самое короткое РВ.
- Можем ли мы определить, “Являются ли два языка одним и тем же?”
  - Т.е., определяют ли два КДА один и тот же язык?

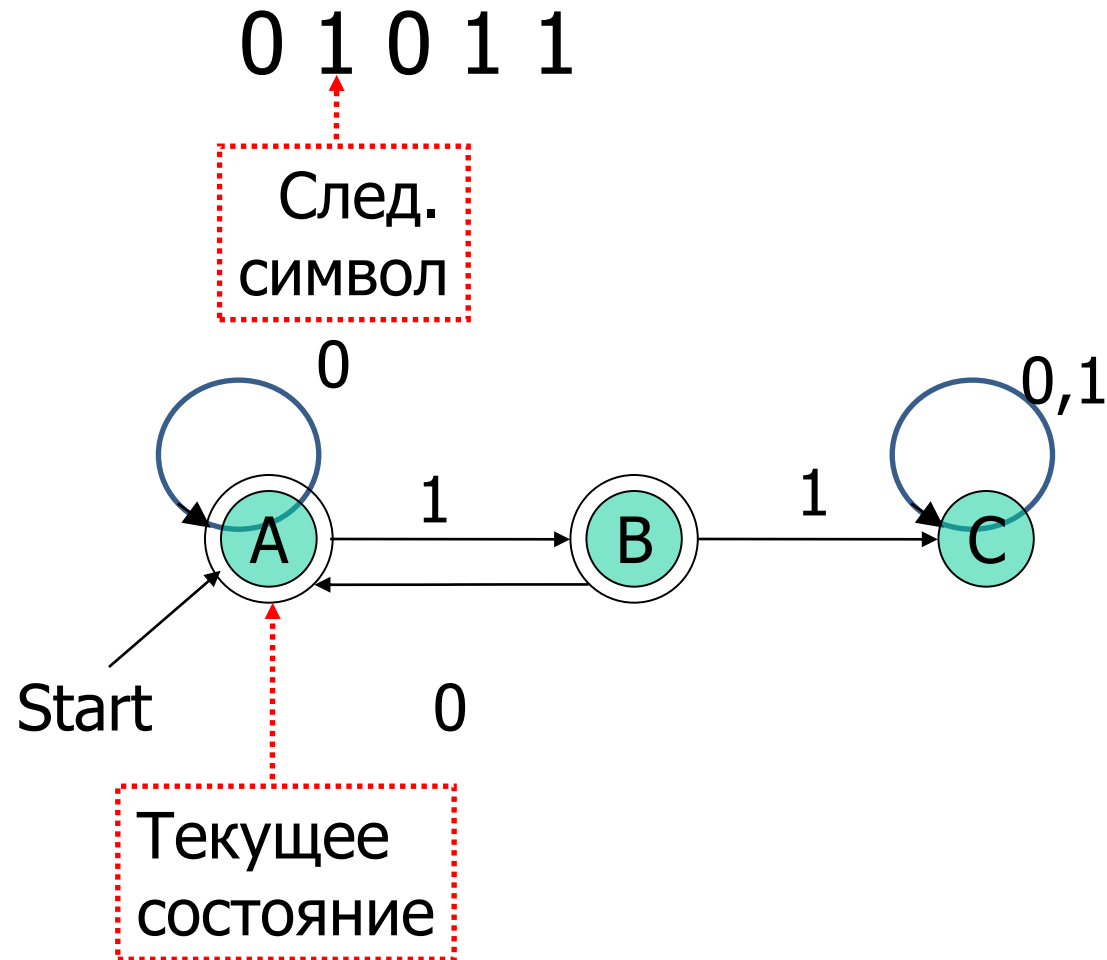
# Проблема принадлежности строки языку

- Нашей первой разрешимой проблемой для регулярных языков будет ответ на вопрос: “находится ли строка  $w$  в регулярном языке  $L$ ?”
- Предположим, что  $L$  представлен КДА  $A$ .
- Смоделируем работу  $A$  на последовательности входных символов  $w$ .

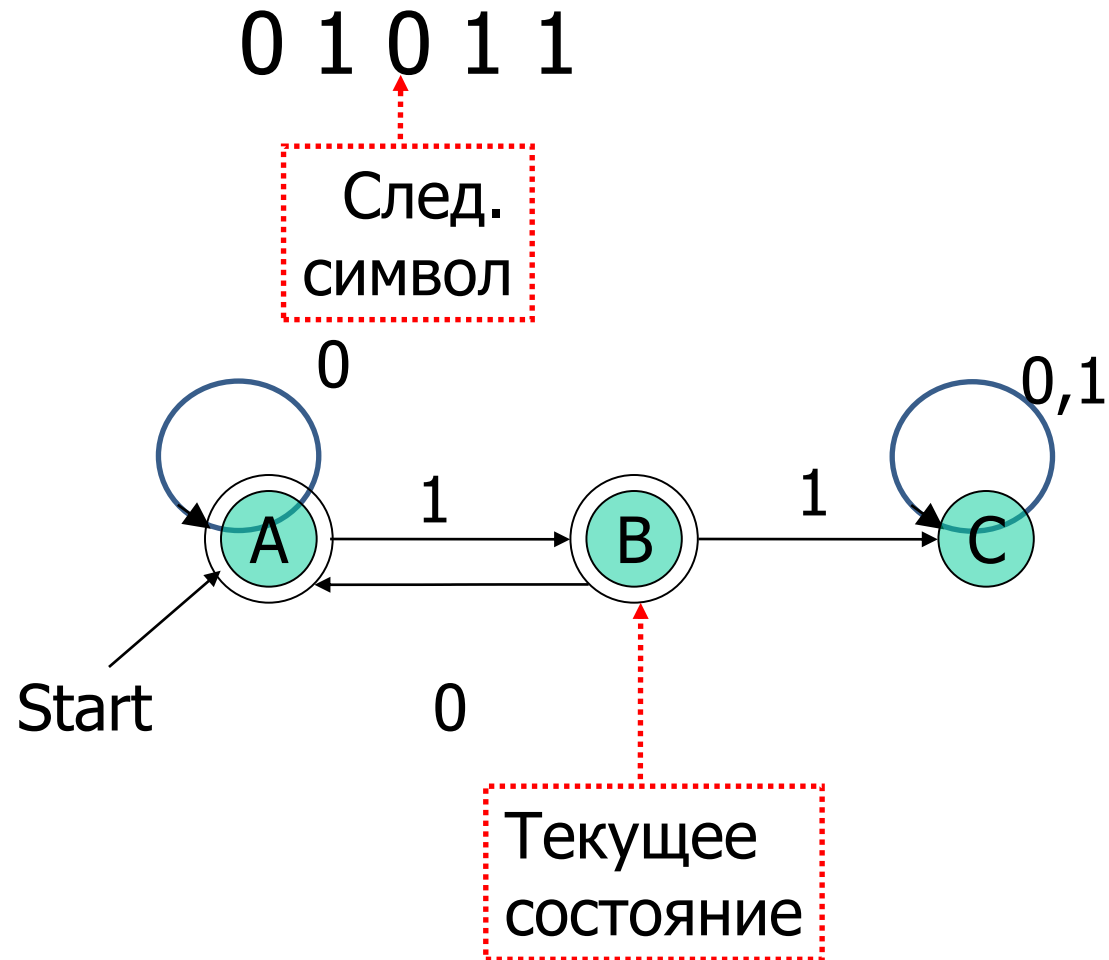
# Пример: проверка принадлежности



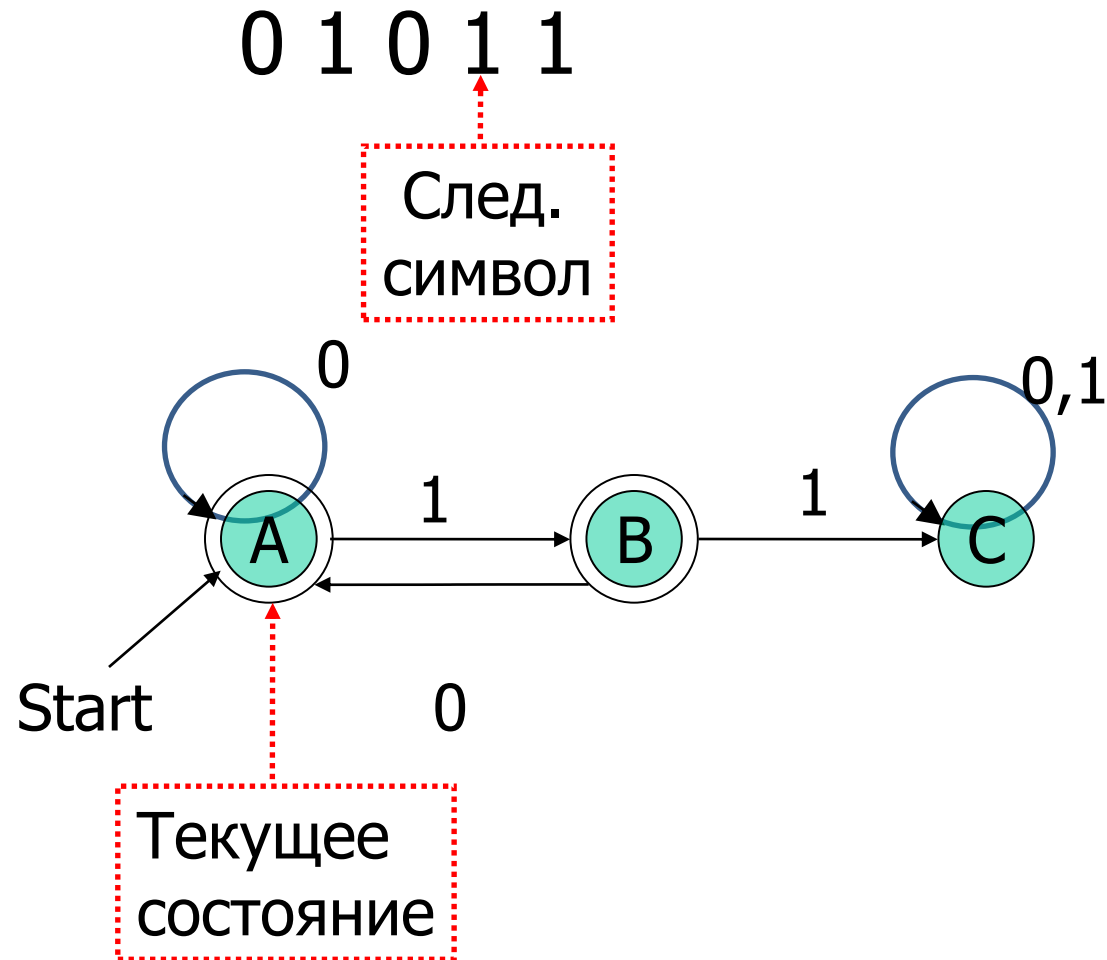
# Пример: проверка принадлежности



## Пример: проверка принадлежности

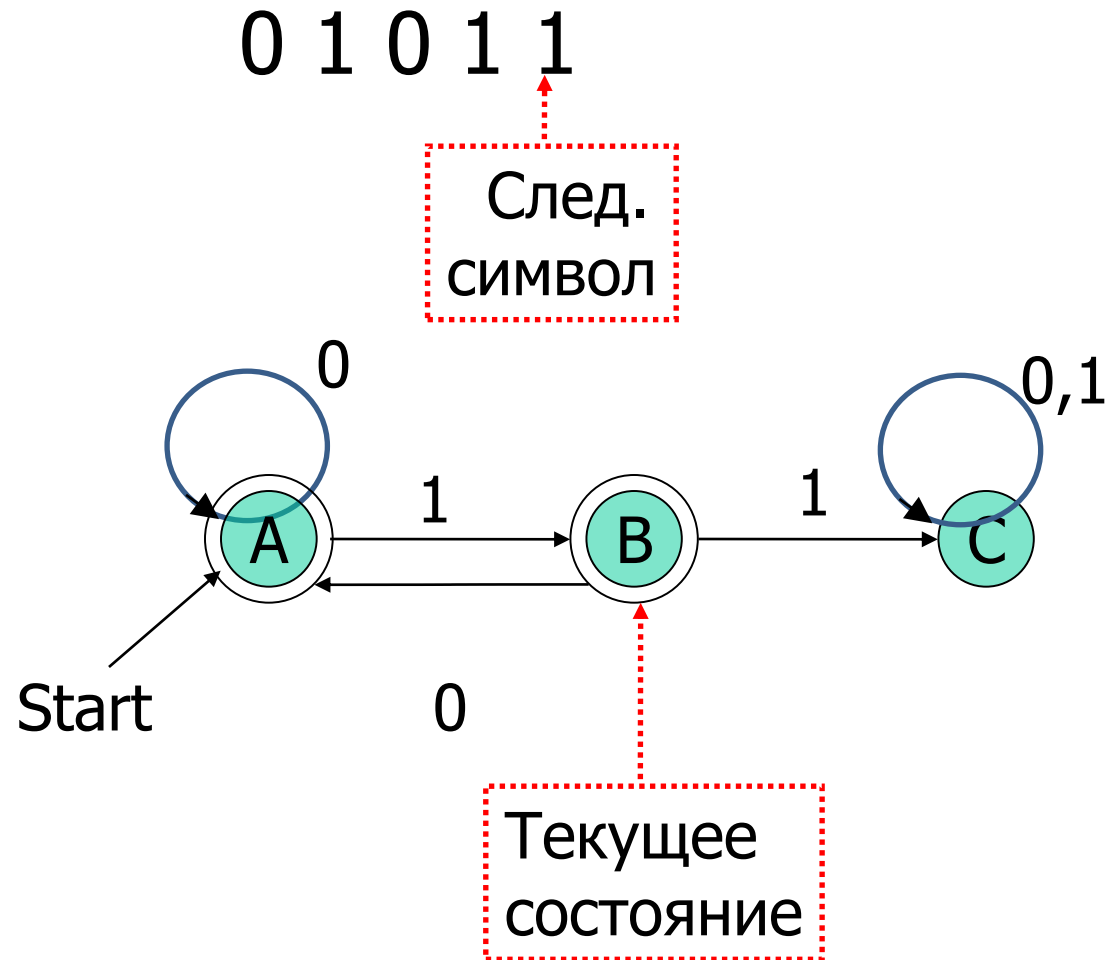


# Пример: проверка принадлежности

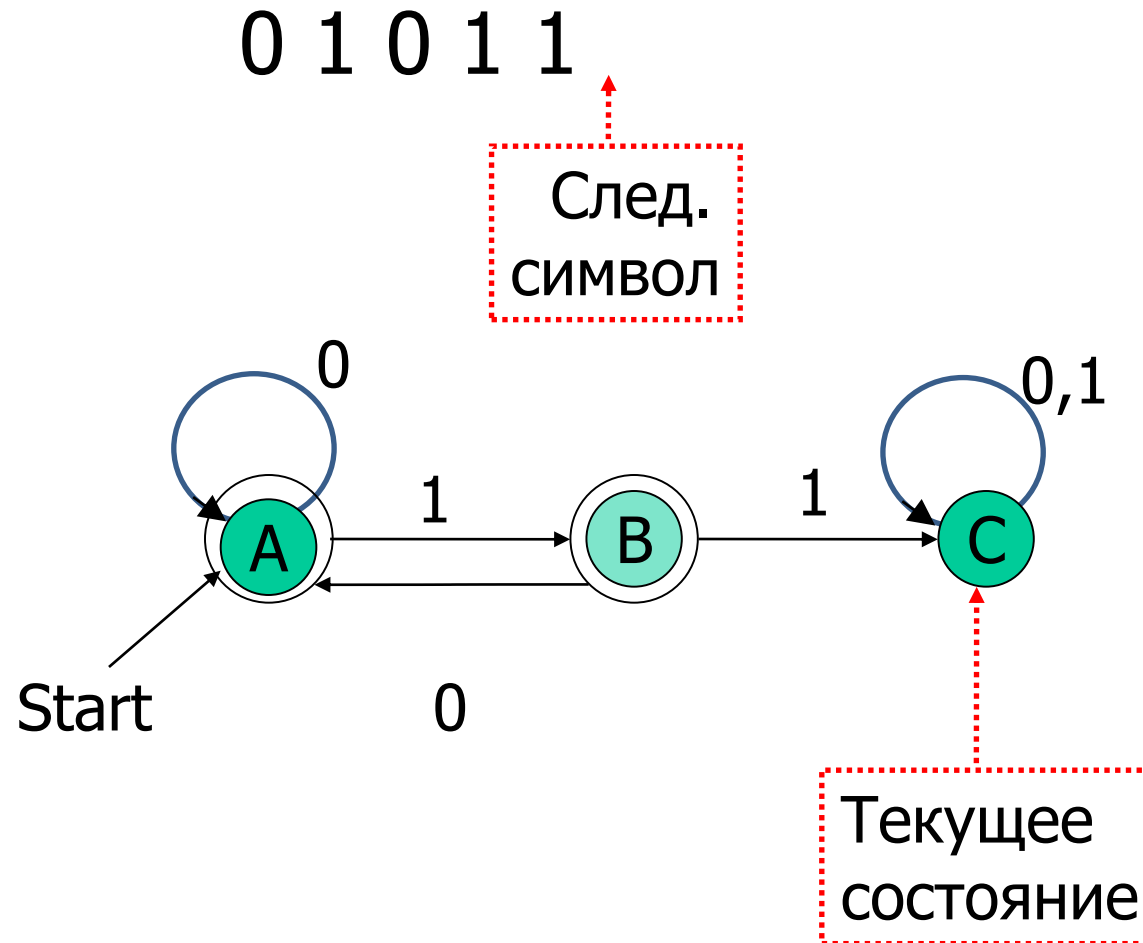




## Пример: проверка принадлежности



## Пример: проверка принадлежности



# Проблема принадлежности слова языку

- Таким образом, *проблема принадлежности произвольной строки (над определенным алфавитом) регулярному языку разрешима.*

# Проблема пустоты

- Для данного регулярного языка определить, содержит ли он хотя бы одну строку ?
- Пусть представлением языка является КДА.
- Вычислим множество состояний, достижимых из начального.
- Если хотя бы одно из конечных состояний достижимо, то ответ “да”, иначе - “нет”.
- *Проблема пустоты* для регулярных языков *разрешима*.

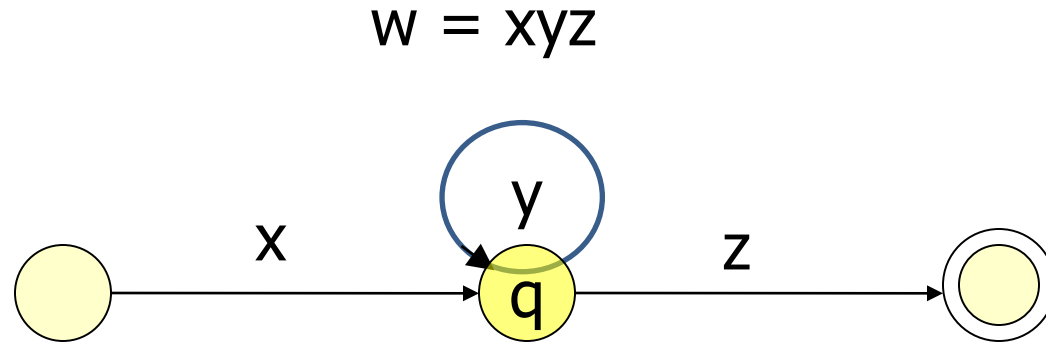
# Проблема бесконечности

- Является данный регулярный язык бесконечным?
- Рассмотрим КДА для языка.
- **Ключевая идея**: если КДА имеет  $n$  состояний и язык содержит какую либо строку длины  $n$  или более, то язык бесконечный.
- В противном случае, язык, безусловно, конечный.
  - Можно ограничиться строками длины  $n$  или меньше.

# Доказательство ключевой идеи

- Если КДА с  $n$  состояниями принимает строку  $w$  длины  $n$  или более, то должно быть состояние, в котором он окажется дважды на пути, помеченном  $w$  от начального состояния к конечному.
- Потому что существует, по крайней мере,  $n+1$  состояний вдоль всего пути.

## Доказательство – (2)



Тогда  $xy^iz$  является языком для всех  $i \geq 0$ .

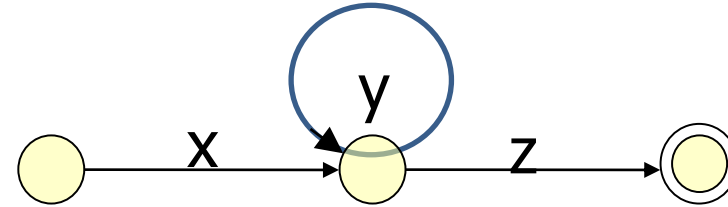
Т.к.  $y$  не есть  $\epsilon$ , мы имеем бесконечное число строк в  $L$ .

# Бесконечность – продолжение

- Пока мы не имеем алгоритма.
- Существует бесконечное число строк длины  $> n$ , и мы не можем проверить их все.
- **Вторая ключевая идея**: если существует строка длины  $\geq n$  ( $n$  = числу состояний) в  $L$ , то в  $L$  существует строка длины между  $n$  и  $2n-1$ .



# Доказательство 2-ой ключевой идеи



- Вспомним:
- $y$  - это первый цикл в пути.
- Тогда:  $|xy| \leq n$ ; в частности,  $1 \leq |y| \leq n$ .
- То есть, одно состояние среди первых  $n+1$  повторяется.
- Т.о., если  $w$  имеет длину  $2n$  или более, то существует более короткая строка в  $L$ , которая имеет длину, по крайней мере,  $n$ .
- Продолжим сокращение длины путём удаления циклов, пока не достигнем длины в интервале  $[n, 2n-1]$ .

# Завершение доказательства алгоритма

- Для доказательства бесконечности регулярного языка построим для него КДА. Пусть он имеет  $n$  состояний
- Протестируем на принадлежность языку всех строк длины между  $n$  и  $2n-1$ .
  - Если какая-либо из них принимается, то язык бесконечен, иначе – конечен.
- Ужасный алгоритм. Если мы имеем  $k$  входных символов и  $n$  состояний, то число строк для проверки составит  $k^{2n}$ .
- **Лучше**: найти циклы между начальным и конечным состояниями. Для этого есть алгоритм сложности порядка  $kn$ .

# Поиск циклов

1. Исключим состояния, недостижимые из начального состояния.
2. Устраним состояния, из которых не достигаются конечные состояния.
3. Проверим наличие циклов в оставшемся графе переходов.

## Поиск циклов— (2)

- Простым, но менее эффективным способом поиска циклов является поиск вперед от заданного узла  $N$ .
- Если из  $N$  можно достичь  $N$ , то цикл есть.
- Сделайте это, начиная с каждого узла.
- Если есть цикл, то язык для данного КДА бесконечный. Если циклов нет, то конечный.
- *Проблема бесконечности регулярного языка разрешима.*

# Лемма о накачке

- Мы, почти случайно, доказали утверждение, которое полезно для доказательства того, что определенные языки не являются регулярными.
- Назовем соответствующее утверждение *леммой о накачке для регулярных языков*.

# Формулировка леммы о накачке

**Лемма.** Для каждого регулярного языка  $L$   
существует целое  $n$ , такое, что  
для каждой строки  $w$  из  $L$  длины  $\geq n$   
мы можем записать ее как  $w = xyz$  так, что:

Число  
состояний  
КДА для  $L$

Метки вдоль  
первого цикла пути,  
помеченного  $w$

1.  $|xy| \leq n$ .
2.  $|y| > 0$ .
3. Для всех  $i \geq 0$ ,  $xy^iz$  также принадлежит  $L$ .

## Пример: Применение леммы о накачке

- Рассмотрим язык  $\{0^k 1^k \mid k \geq 1\}$ . Мы говорили, что он не регулярный.
- Предположим, что это так. Тогда для данного языка должно существовать ассоциированное с леммой число  $n$ .
- Пусть  $w = 0^n 1^n$ . Мы можем записать  $w = xyz$ , где  $|xy| \leq n$ , т.е.  $x$  и  $y$  состоят из 0-ей, и  $y \neq \epsilon$ .
- Но тогда  $xyyz$  тоже должна быть в  $L$ , и эта строка имеет нулей больше, чем единиц !

# Проблема: Эквивалентность

- По данным регулярным языкам  $L$  и  $M$  определить  $L = M$ ?
- Алгоритм доказательства существования разрешающей процедуры базируется на построении КДА - *произведения* автоматов для  $L$  и  $M$ .



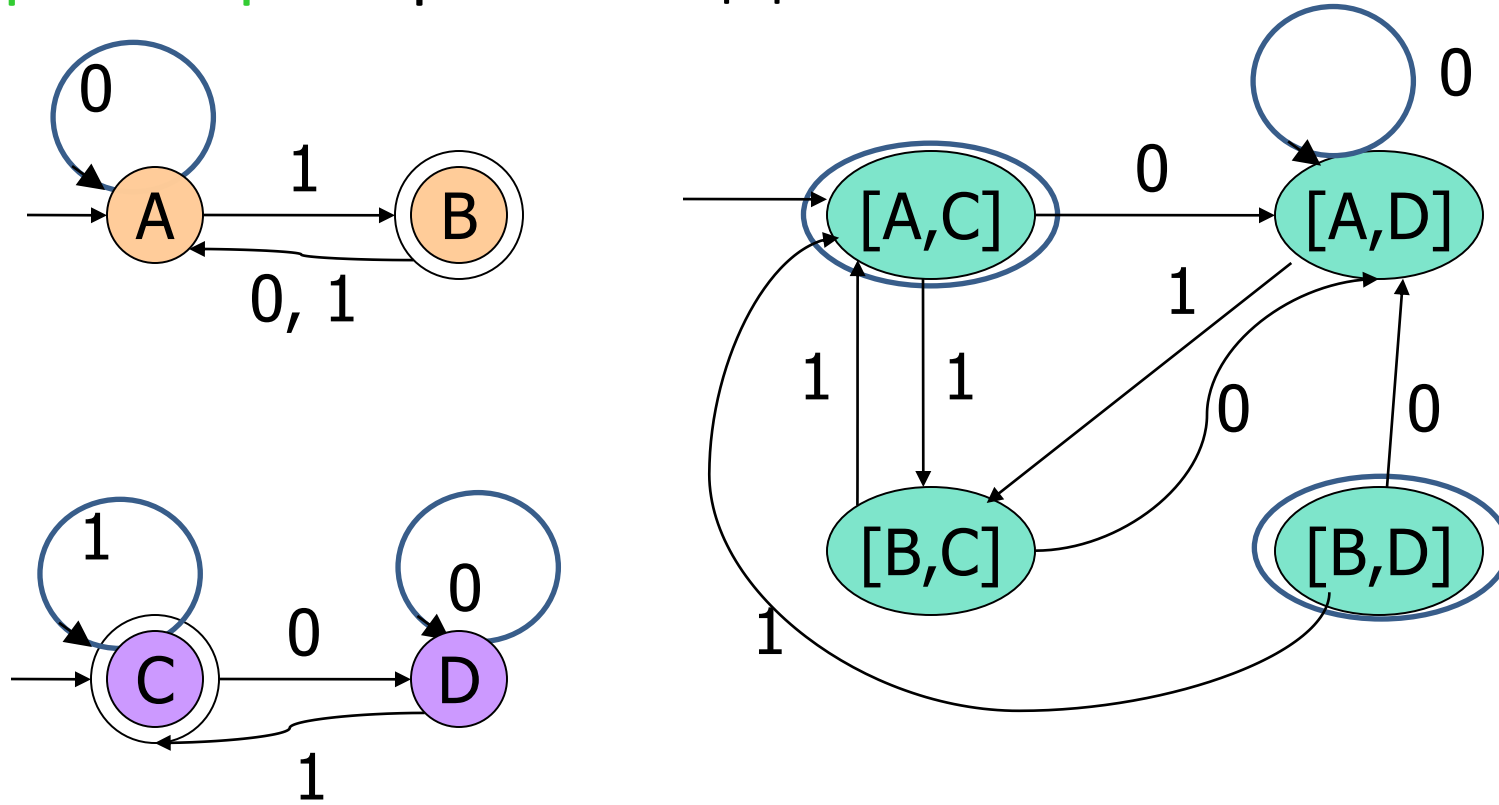
# Построение произведения автоматов L и M

- Пусть КДА L и M имеют множества состояний Q и R, соответственно.
- $L = (Q, \Sigma, \delta_L, q_0, F_L)$ ,  $M = (R, \Sigma, \delta_M, r_0, F_R)$
- Автомат-произведение имеет множество состояний  $Q \times R$ .
  - т.е., пары  $[q, r]$ , где  $q \in Q, r \in R$ .
- Начальное состояние =  $[q_0, r_0]$  (начальные состояния КДА для L, M).

## Произведение автоматов – прод.

- **Переходы:**  $\delta([q,r], a) = [\delta_L(q,a), \delta_M(r,a)]$ 
  - $\delta_L, \delta_M$  функции переходов для КДА  $L$  и  $M$ .
  - То есть, мы моделируем исходные КДА в компонентах двух состояний автомата-произведения.
- Определим заключительные состояния автомата - произведения из тех состояний  $[q, r]$ , в которых точно одно из  $q$  и  $r$  есть финальное состояние соответствующего автомата.

## Пример: Произведение автоматов

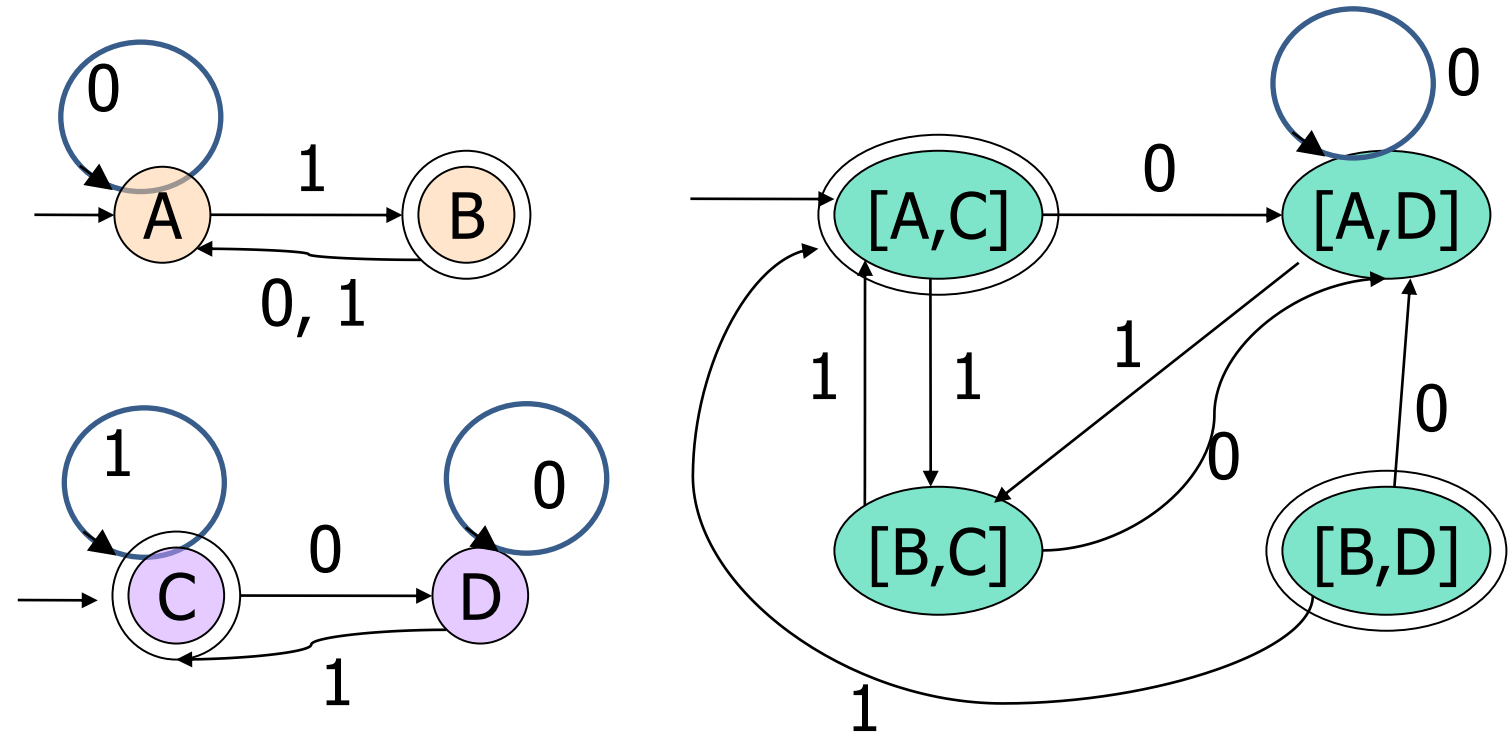


# Алгоритм установления эквивалентности

- Заключительные состояния автомата-произведения - те состояний  $[q, r]$ , в которых точно одно из  $q$  и  $r$  есть финальное состояние соответствующего автомата.
- Т.о., автомат-произведение принимает  $w$  т.и.т.т., когда  $w$  в точности принадлежит языку для одного из  $L$  или  $M$ .
- $L = M$  тогда и только тогда, когда язык для автомата произведения пуст.

## Пример: Эквивалентность

- $[A,C]$  – финальное состояние, т.к. в оригинальных автоматах,  $C$  – финальное, а  $A$  – нет.
- Аналогично,  $[B,D]$  – финальное состояние, т.к.  $B$  – финальное, а  $D$  – нет.



Мы видим, что два исходных автомата не эквивалентны. Очевидно, что  $[B,D]$  недостижимо из начального состояния; это так, потому что нет строк, которые оранжевый автомат принимает, а фиолетовый не принимает.

Однако  $[A,C]$  – тоже финальное состояние и, очевидно, что оно достижимо из начального состояния, т.к. оно и есть начальное состояние. Т.е. пустая строка различает два автомата. Пустая строка принимается фиолетовым автоматом, но не принимается оранжевым.

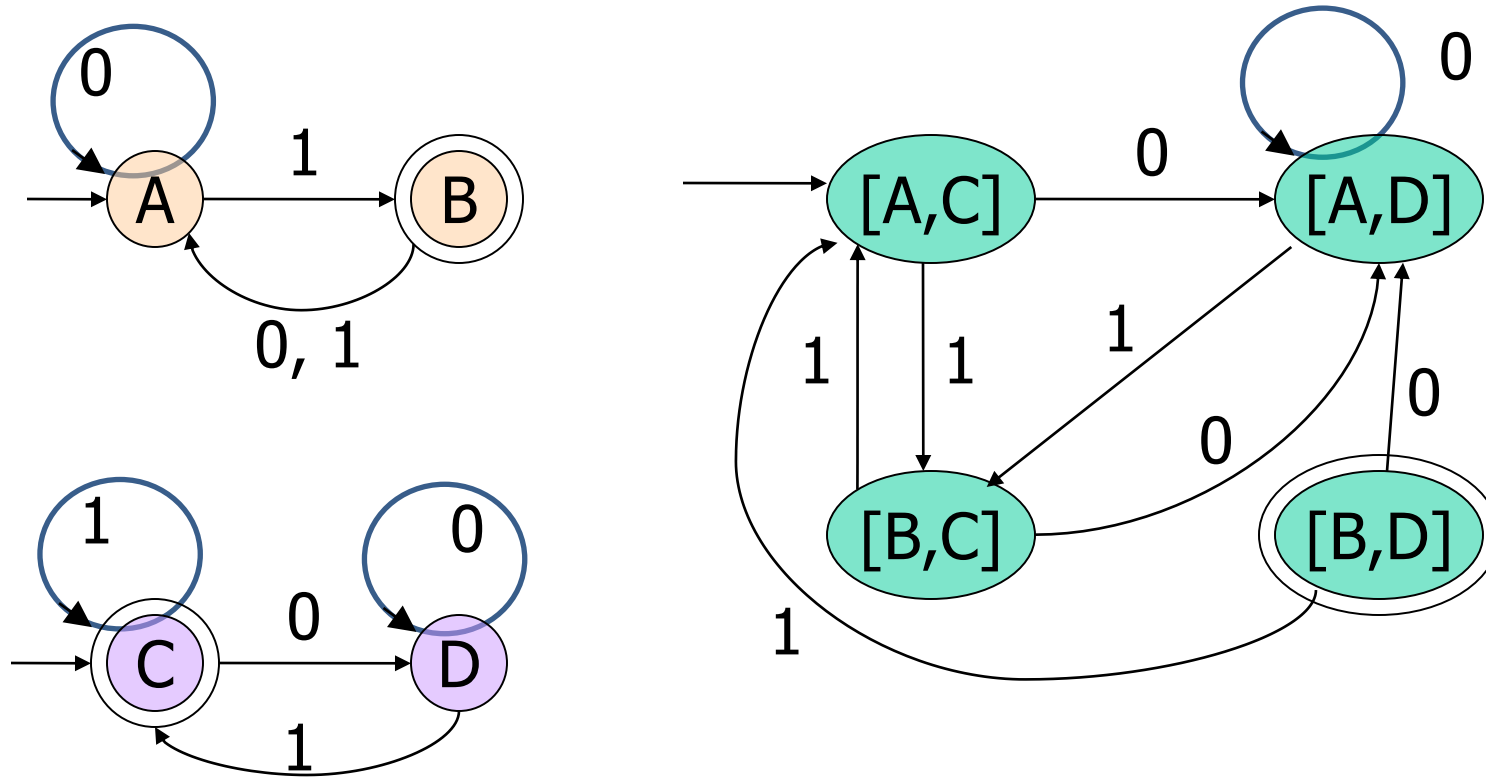
# Проблема вложения языков

- По данным регулярным языкам  $L$  и  $M$  определить,  $L \subseteq M$ ?
- Алгоритм также использует конструкцию произведения автоматов.
- Как нужно определить финальное состояние  $[q, r]$  автомата-произведения, чтобы его язык был пуст т.и.т.т., когда  $L \subseteq M$ ?

**Ответ:**  $q$  - финальное;  $r$  - нет .

- То есть,  $L$  не содержится в  $M$  т.и.т.т., когда существует некоторая строка  $w$  такая, что  $w \in L$  но  $w \notin M$ . Такая строка могла бы перевести КДА для  $L$  в финальное состояние, но не смогла бы перевести КДА для  $M$  в финальное состояние.
- Так что вопрос вложения – это тот же самый вопрос о существовании строки, переводящей автомат-произведение в состояние  $[q, r]$  где  $q$  –заключительное,  $r$  - нет.

## Пример: вложение языков



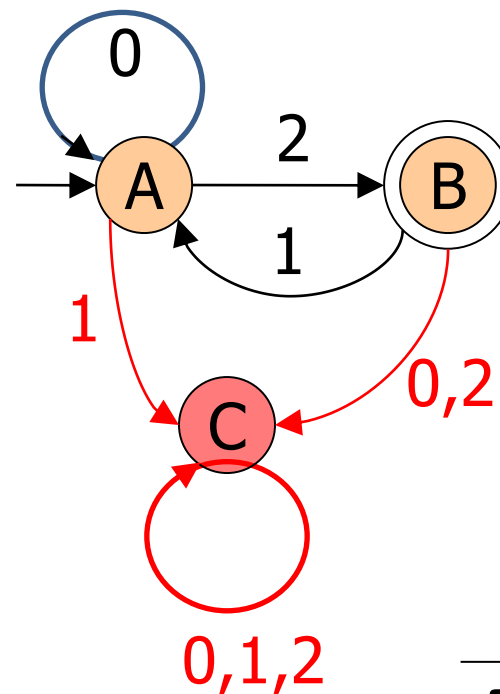
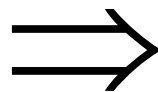
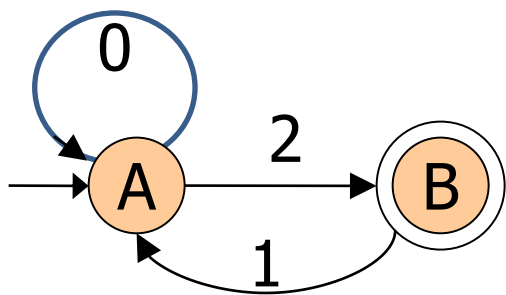
Замечание: только одно финальное состояние недостижимо, язык для автомата произведения пуст, т.о. вложение выполняется.

# Минимальный КДА для регулярных языков

- В принципе, т.к. мы можем протестировать КДА на эквивалентность, то мы можем по данному КДА  $A$  найти КДА с наименьшим числом состояний принимающий  $L(A)$ .
- Можно проверить все КДА на эквивалентность с  $A$ .
- Но этот алгоритм, конечно, никуда не годится.



# Доопределение КДА до полного

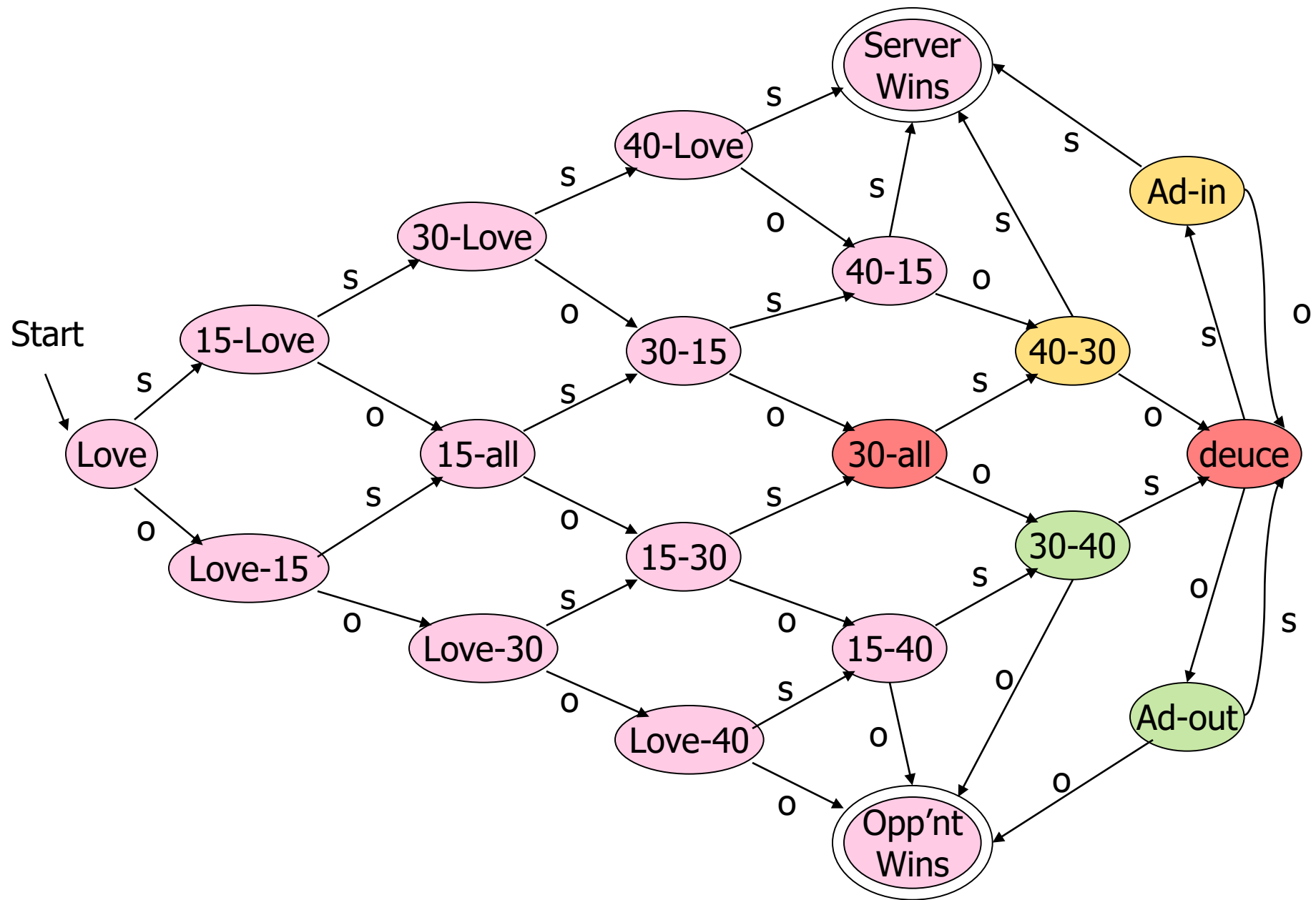


|   | 0 | 1 | 2 |
|---|---|---|---|
| A | A | - | B |
| B | - | A | - |

|   | 0 | 1 | 2 |
|---|---|---|---|
| A | A | C | B |
| B | C | A | C |
| C | C | C | C |

## Эффективная минимизация числа состояний

- Построим таблицу со всеми парами состояний.
- Если мы найдем строку, которая *различает* два состояния (переводит точно одно состояние из двух в заключительное), то пометим эту пару.
- В противном случае состояния в паре будут *неразличимыми*, и они могут быть объединены в одно состояние.
- Алгоритм является рекурсивным по длине кратчайших различающих строк.



## Минимизация числа состояний – (2)

- **Базис**: Пометим пары с точно одним финальным состоянием.
- **Индукция**: пометим  $[q, r]$ , если для некоторого входного символа  $a$ ,  $[\delta(q, a), \delta(r, a)]$  является помеченной.
- После того, как больше нельзя пометить ни одно состояние, непомеченные пары состояний объявляем эквивалентными и объединяем в одно состояние.

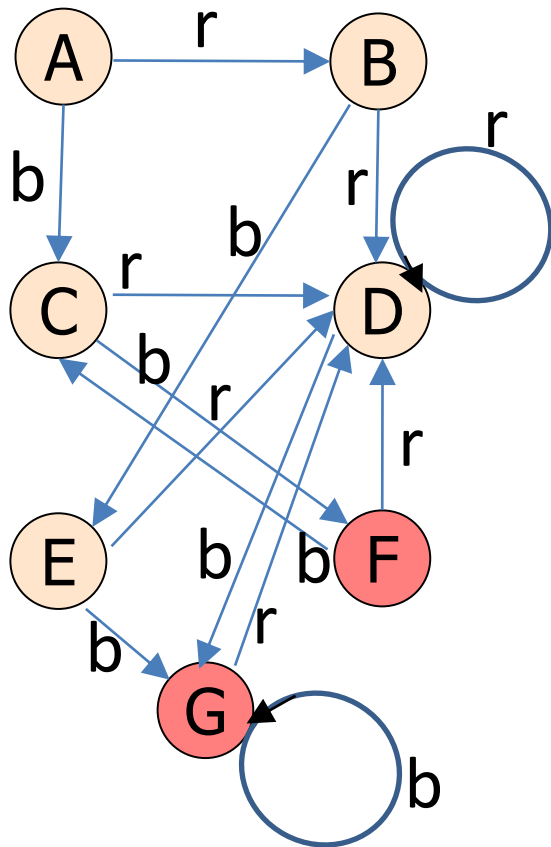
## Транзитивность “Неразличимости”

- Если состояние  $p$  неразлично от  $q$ , и  $q$  неразлично от  $r$ , то  $p$  неразлично от  $r$ .
- **Доказательство:** Результат (принятие или нет) входа  $w$  для  $p$  и  $q$  будет один и тот же, и результат для  $q$  и  $r$  на  $w$  тот же, тогда аналогичный результат будет для  $p$  и  $r$ .

# Построение минимального КДА

- Предположим, что  $q_1, \dots, q_k$  - неразличимые состояния.
- Заменим их одним их *представляющим* состоянием  $q$ .
- Тогда  $\delta(q_1, a), \dots, \delta(q_k, a)$  есть все неразличимые состояния.
  - **NB!**: в противном случае, нам следовало бы пометить по крайней мере на одну пару состояний больше.
- Пусть  $\delta(q, a)$  = состояние - представитель для этой группы.

# Пример: минимизация числа состояний



→

|    | r | b |
|----|---|---|
| A  | B | C |
| B  | D | E |
| C  | D | F |
| D  | D | G |
| E  | D | G |
| *F | D | C |
| *G | D | G |

|   | G | F | E | D | C | B |
|---|---|---|---|---|---|---|
| A | x | x |   |   |   |   |
| B | x | x |   |   |   |   |
| C | x | x |   |   |   |   |
| D | x | x |   |   |   |   |
| E | x | x |   |   |   |   |
| F |   |   |   |   |   |   |
| G |   |   |   |   |   |   |

Начнём с пометки пар с одним из заключительных состояний F или G.

## Пример – продолжение

|     | r | b |
|-----|---|---|
| → A | B | C |
|     | B | D |
|     | D | E |
|     | D | F |
|     | D | G |
|     | D | G |
| *   | F | D |
| *   | G | D |

|   | G | F | E | D | C | B |
|---|---|---|---|---|---|---|
| A | X | X |   |   |   |   |
| B | X | X |   |   |   |   |
| C | X | X |   |   |   |   |
| D | X | X |   |   |   |   |
| E | X | X |   |   |   |   |
| F |   |   |   |   |   |   |

Вход r бесполезен,  
т.к. пара [B, D]  
непомечена.



## Пример – продолжение

|     |   |   |
|-----|---|---|
|     | r | b |
| → A | B | C |
|     | B | D |
|     | D | E |
|     | D | F |
|     | D | G |
|     | D | G |
| *   | F | D |
| *   | G | D |

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
|   | G | F | E | D | C | B |
| A | X | X | X | X | X |   |
| B | X | X | X | X | X |   |
| C | X | X |   |   |   |   |
| D | X | X |   |   |   |   |
| E | X | X |   |   |   |   |
| F | X |   |   |   |   |   |

Но вход b различает состояния из  $\{A, B, F\}$  от состояний из  $\{C, D, E, G\}$ . Например,  $[A, C]$  помечается, т.к.  $[C, F]$  помечено.

## Пример – продолжение

|     | r | b |
|-----|---|---|
| → A | B | C |
| B   | D | E |
| C   | D | F |
| D   | D | G |
| E   | D | G |
| * F | D | C |
| * G | D | G |

|   | G | F | E | D | C | B |
|---|---|---|---|---|---|---|
| A | X | X | X | X | X |   |
| B | X | X | X | X | X |   |
| C | X | X | X | X |   |   |
| D | X | X |   |   |   |   |
| E | X | X |   |   |   |   |
| F | X |   |   |   |   |   |

[C, D] и [C, E] помечаются,  
т.к. есть переход по b в  
Помеченную пару [F, G].

## Пример – продолжение

|   | r   | b |
|---|-----|---|
| → | A B | C |
|   | B D | E |
|   | C D | F |
|   | D D | G |
|   | E D | G |
| * | F D | C |
| * | G D | G |

[A, B] помечается, т.к. имеются переходы по r в помеченную пару [B, D].

|   | G | F | E | D | C | B |
|---|---|---|---|---|---|---|
| A | X | X | X | X | X | X |
| B | X | X | X | X | X |   |
| C | X | X | X | X |   |   |
| D | X | X |   |   |   |   |
| E | X | X |   |   |   |   |
| F | X |   |   |   |   |   |

[D, E] никогда не будут помечены, т.к. по обоим входам они переходят в одно и то же состояние.

## Пример — заключение

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | r | b |   | r | b |   | G | F | E | D | C | B |   |
| → | A | B | C | → | A | B | C | A | X | X | X | X | X |
|   | B | D | E |   | B | H | H | B | X | X | X | X |   |
|   | C | D | F |   | C | H | F | C | X | X | X | X |   |
|   | D | D | G |   | H | H | G | D | X | X |   |   |   |
|   | E | D | G |   |   |   |   | E | X | X |   |   |   |
| * | F | D | C | * | F | H | C | F | X |   |   |   |   |
| * | G | D | G | * | G | H | G |   |   |   |   |   |   |

Заменим D и E на H.

Результат — КДА с минимальным  
числом состояний.

# Удаление недостижимых состояний

- К сожалению, комбинирование неразличимых состояний может оставить нас с недостижимыми состояниями в минимальном КДА.
- Таким образом, до или после, нужно удалить состояния, которые недостижимы из начального состояния.

# Удаление недостижимых состояний

- **Опр.** Состояние  $q$  конечного автомата  $M = (Q, \Sigma, \delta, q_0, F)$  называется недостижимым, если не найдется цепочки  $\alpha \in \Sigma^*$ , приводящей автомат из начального состояния  $q_0$  в состояние  $q$ .

Т.е.,  $\nexists \alpha \in \Sigma^*: \delta(q_0, \alpha) = q$

- **Теорема.** Для любого КДА  $M$  найдется эквивалентный КДА автомат  $M'$  без недостижимых состояний.

**Доказательство.** Пусть  $M = (Q, \Sigma, \delta, q_0, F)$ .

Построим  $M' = (Q', \Sigma, \delta', q_0, F')$ , где

$Q'$  – множество состояний, достижимых из  $q_0$ ,

$\delta' = \delta|_{Q'}$  (ограничение  $\delta$  на  $Q'$ )

$F' = F \cap Q'$ .

Очевидно, что автомат  $M'$  допускает тот же язык, что и автомат  $M$ .

# Ключевой момент

- Пусть мы скомбинировали (отождествили) все возможные состояния данного КДА, получили новый эквивалентный автомат с меньшим числом состояний.
- Может ли существовать другой эквивалентный КДА с меньшим числом состояний?
- Нет. Доказательство включает в себя минимизацию КДА, который мы получили, с гипотетически лучшим КДА.

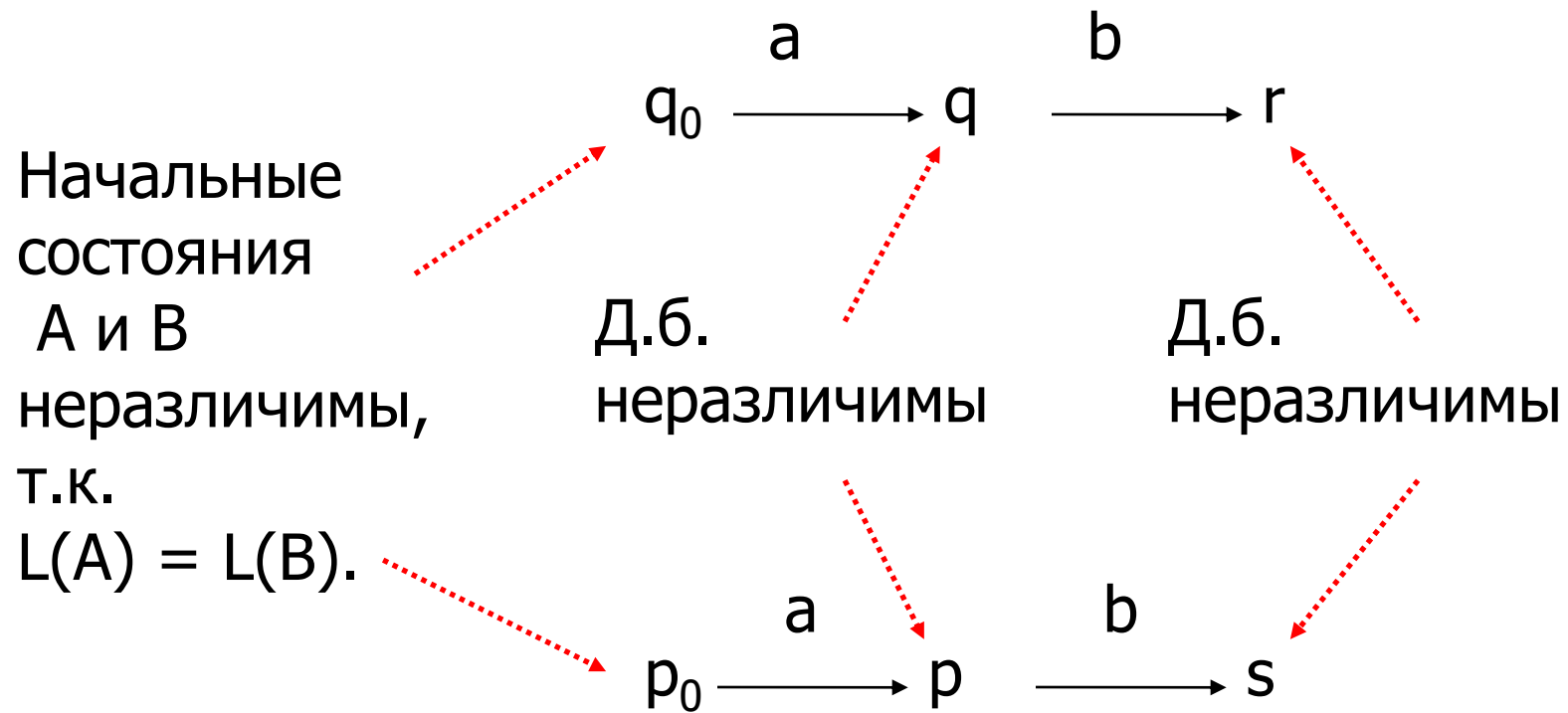


- **Теорема.** (О минимальном автомате)
- Для любого КДА автомата  $M = (Q, \Sigma, \delta, F)$  найдется эквивалентный КДА  $A = (Q, \Sigma, \delta, F)$  без неразличимых состояний, допускающий тот же язык. Причём, если  $M$  –автомат без недостижимых состояний, то не существует КДА, допускающего  $L(M)$  и имеющего состояний меньше, чем у автомата  $A$ .

## Доказательство: Нет другого «меньшего» автомата

- Пусть  $A$  – наш минимизированный КДА; пусть  $B$  – его «меньший» эквивалент.
- Предположим, что  $A$  эквивалентен  $B$ .
- Рассмотрим объединённый комбинированный автомат с состояниями из  $A$  и  $B$  (неважно какое состояние начальное, но заключительные состояния – это те, что были в  $A$  и  $B$ ).
- Используем “различимость” в ее контрапозитивной форме:
  - То есть, если  $w$  различает  $\delta(q, a)$  от  $\delta(r, a)$ , то, конечно,  $aw$  различает  $q$  от  $r$ .
  - Таким образом, если состояния  $q$  и  $r$  **не**различимы, то их последователи по некоторому  $a$ :  $\delta(q, a)$  и  $\delta(r, a)$  - также неразличимы.

# Наследование неразличимости



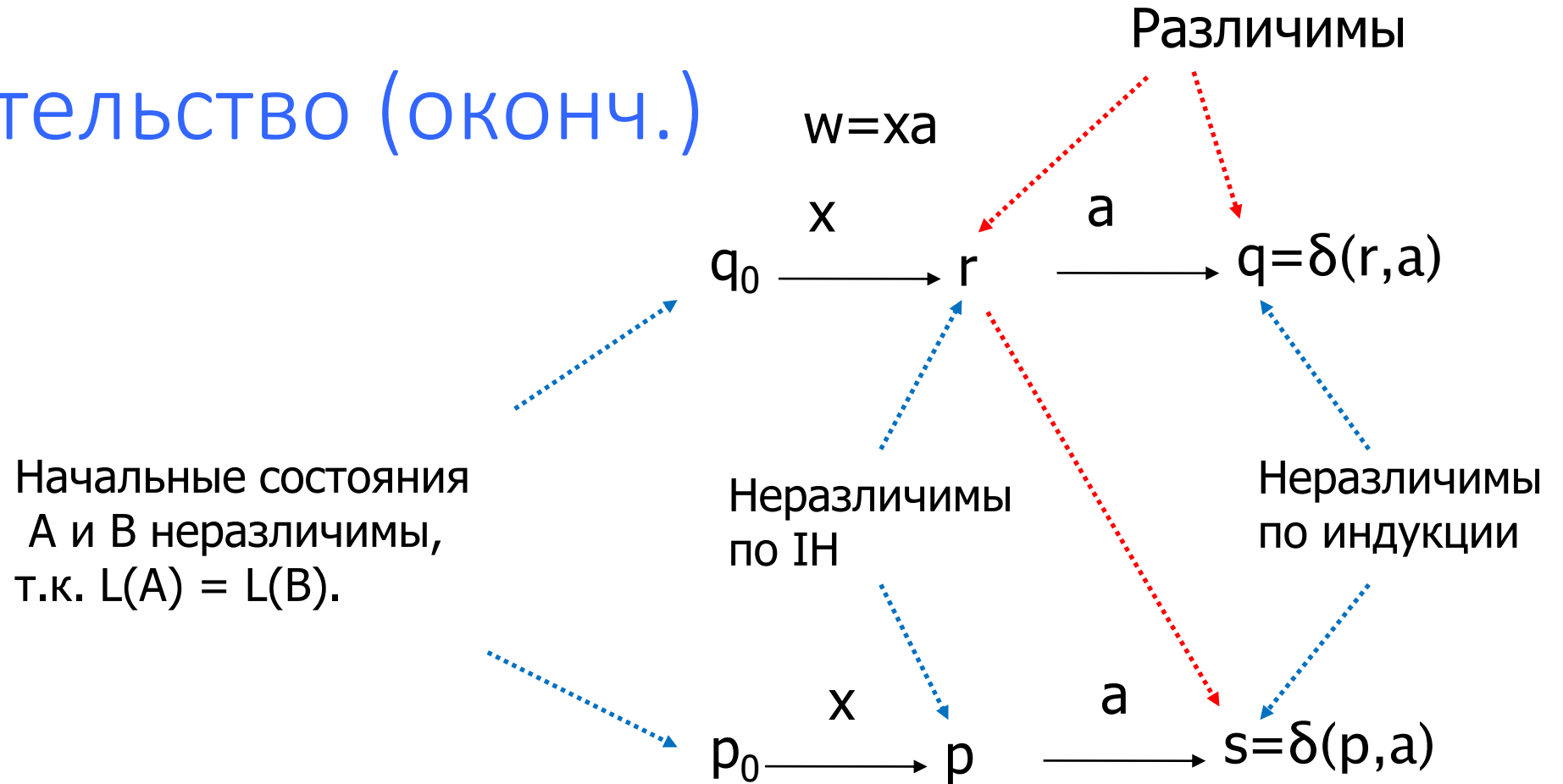
# Индуктивная гипотеза

- Каждое состояние  $q$  автомата  $A$  неразлично с некоторым состоянием автомата  $B$ .
- Индукция проводится по длине кратчайшей строки, переводящей начальное состояние автомата  $A$  в  $q$ .

# Доказательство

- **Базис:** Начальные состояния  $A$  и  $B$  неразличимы, т.к.  $L(A) = L(B)$ .
- **ИН:** Пусть строка длины  $n < |w|$  переводит  $A$  в некоторое состояние  $r$ , которое неразлично с некоторым состоянием  $p$  автомата  $B$ .
- **Индукция:** Предположим, что  $w = xa$  есть кратчайшая строка, переводящая автомат  $A$  в состояние  $q$ .
- Согласно ИН, строка  $x$  переводит  $A$  в некоторое состояние  $r$ , которое неразлично с некоторым состоянием  $p$  автомата  $B$ .
- Тогда  $\delta_A(r, a) = q$  неразлично с  $\delta_B(p, a)$ .

# Доказательство (оконч.)



Начальные состояния  
A и B неразличимы,  
т.к.  $L(A) = L(B)$ .

- Однако, два состояния автомата A не могут быть неразличимы с тем же состоянием в B, или они были бы неразличимы одно от другого.
  - Это нарушает транзитивность “неразличимости.”
- Таким образом, B имеет, по крайней мере, столько же состояний, что и A.

# Разрешимые свойства для регулярных языков - ИТОГИ

- Следующие проблемы разрешимы для регулярных языков:
- Проблема принадлежности языку:  $w \in L$ ?
- Проблема пустоты:  $L = \emptyset$ ?
- Проблема эквивалентности:  $L = M$ ?
- Проблема вложения языков:  $L \subseteq M$ ?
- Проблема бесконечности языка:  $|L| = \infty$ ?

# Свойства замыкания для регулярных языков

Объединение, Пересечение, Разность,  
Конкатенация, Замыкание Клини,  
Гомоморфизм, Инверсный гомоморфизм



# Свойства замкнутости

- *Свойство замкнутости* класса языков говорит, что выполнение некоторой операции над языками в классе (например, объединение) даёт в результате язык из того же класса.
- *Пример*: регулярные языки, очевидно, замкнуты относительно операций объединения, конкатенации и замыкания Клини (данные операции называются регулярными).
  - Использовать для доказательства представление регулярных языков регулярными выражениями.

# Замкнутость класса регулярных языков относительно регулярных операций

- **Теорема**. Класс леволинейных языков замкнут относительно регулярных операций.

- **Док-во**.

Пусть  $L$  и  $L'$  - леволинейные языки, определенные с помощью грамматик  $G = (N, \Sigma, P, S)$  и  $G' = (N', \Sigma', P', S')$  соответственно.

Пусть  $N \cap N' = \emptyset$

1. Язык  $L \cup L'$  определяется грамматикой

$$G_1 = (N \cup N' \cup \{S_0\}, \Sigma \cup \Sigma', P \cup P' \cup \{S_0 \rightarrow S, S_0 \rightarrow S'\}, S_0)$$

## Док-во, прод.

2. Заменим в  $P'$  правила вида  $A \rightarrow \alpha$ , где  $A \in N'$ ,  $\alpha \in \Sigma'^*$  на правила  $A \rightarrow S\alpha$ . Полученное множество правил обозн. через  $P_1$ .

Грамматика  $G_2 = (N \cup N', \Sigma \cup \Sigma', P \cup P_1, S')$  порождает язык  $LL'$ .

Если  $\beta \in L'$ , то  $S' \Rightarrow_{G'} \beta$ . Но тогда  $S' \Rightarrow_{G_2} S\beta$ .

Таким образом,  $\forall \alpha \in L \ S' \Rightarrow_{G_2} S\beta \Rightarrow_{G_2} \alpha\beta$ . Все конкатенации из  $LL'$  можно вывести в  $G_2$  из  $S'$ .

Никакие другие слова вывести из  $S'$  нельзя.

# Док-во, финал

3. Заменим в  $P'$  правила вида  $A \rightarrow \alpha$ , на  $A \rightarrow S' \alpha$ .

Тогда грамматика  $G_3 = (N' \cup \{S_0\}, \Sigma', P' \cup \{S_0 \rightarrow \varepsilon, S_0 \rightarrow S'\}, S_0)$  порождает язык  $L'^*$ .

В  $G_3$  возможны выводы:

$$\begin{aligned} S_0 &\Rightarrow_{G_3} \varepsilon \\ S_0 &\Rightarrow_{G_3} S' \\ S_0 &\Rightarrow_{G_3}^* \alpha \quad \forall \alpha \in L' \\ S_0 &\Rightarrow_{G_3}^* S' \alpha \quad \forall \alpha \in L' \end{aligned}$$

Следовательно, из  $S_0$  можно вывести любое слово составленное из слов языка  $L'$ , т.е.  $L(G_3) = L'^*$

# Замыкание по объединению

□ **Теорема**. Если  $L$  и  $M$  регулярные языки, то  $L \cup M$  будет регулярным языком .

**Доказательство**: Пусть  $L$  и  $M$  регулярные языки для регулярных выражений  $R$  и  $S$ , соответственно.

□ Тогда  $R+S$  есть РВ, языком для которого является  $L \cup M$  (по определению «+»).

# Замыкание по конкатенации и операции Клини

□ Та же идея:

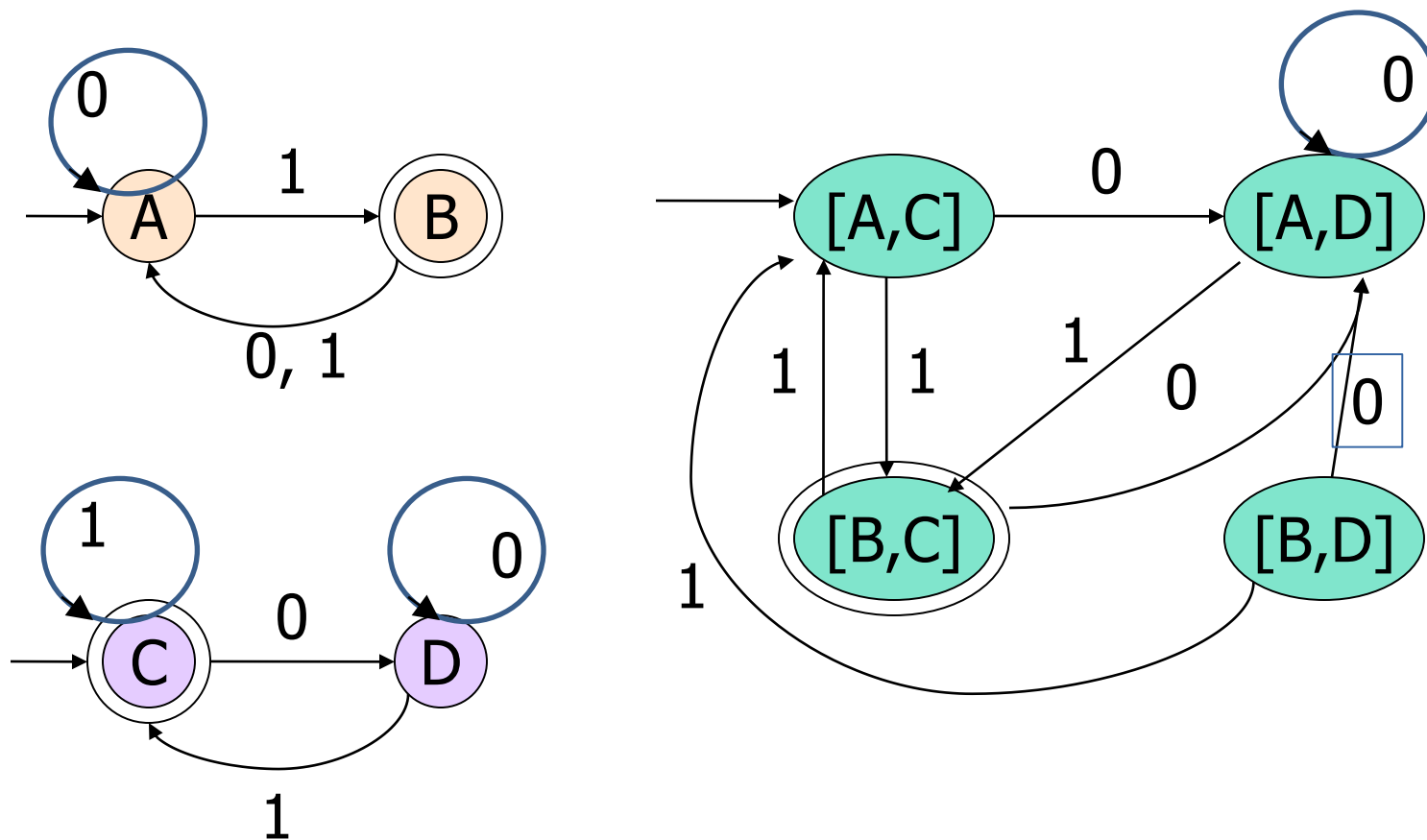
□  $RS$  —  $PB$ , языком для которого является язык  $LM$ .

□  $R^*$  —  $PB$ , языком для которого является язык  $L^*$ .

# Замыкание по пересечению

- **Теорема**. Если  $L$  и  $M$  – регулярные языки, то таковым будет язык  $L \cap M$ .
- **Доказательство**: Пусть  $A$  и  $B$  – КДА, определяющие языки  $L$  и  $M$ , соответственно.
- Построим автомат  $C$  – произведение автоматов  $A$  и  $B$ .
- Сделаем финальными состояниями  $C$  пары, состоящие из финальных состояний  $A$  и  $B$ .

# Пример: КДА-произведение для пересечения





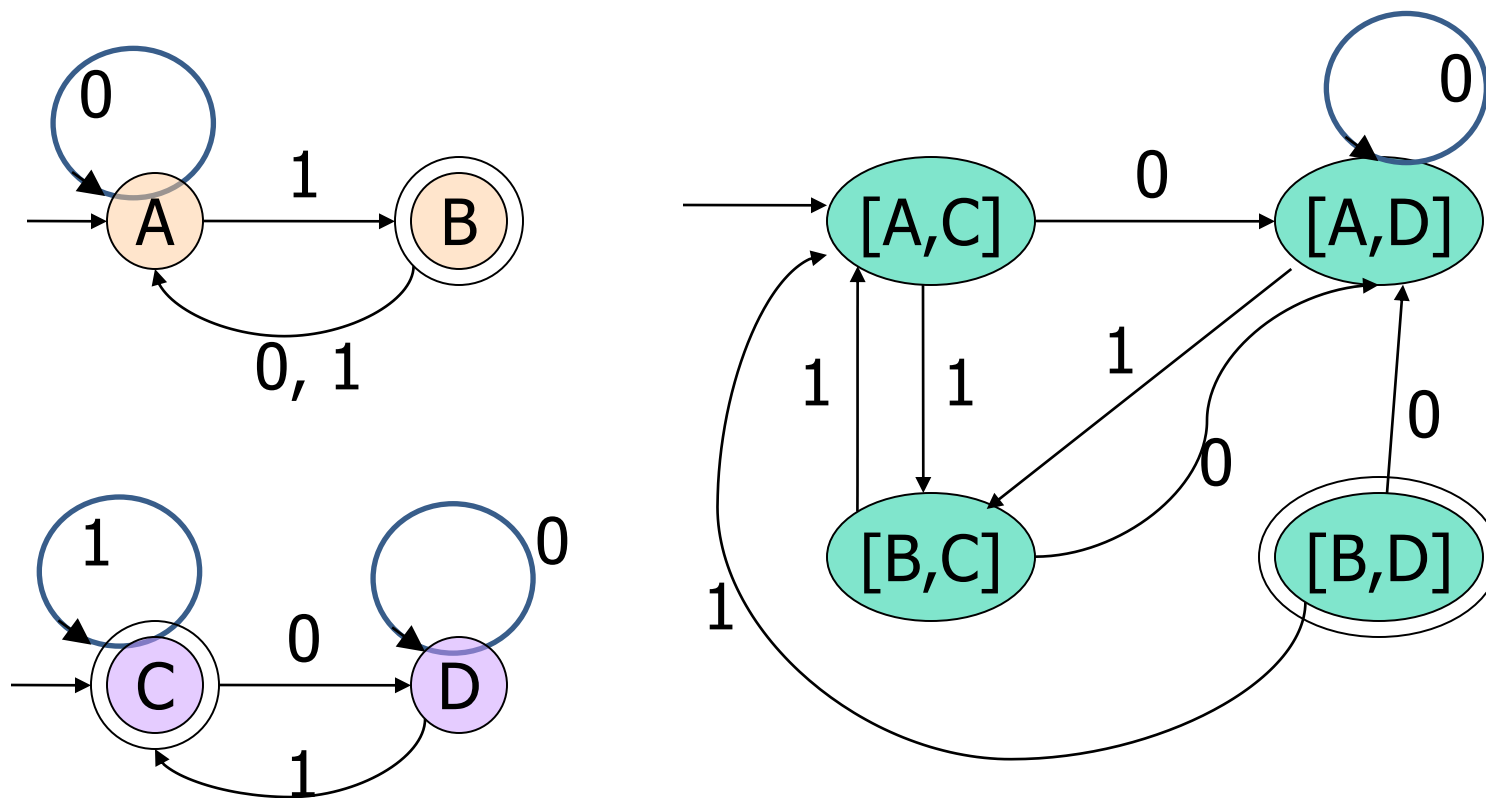
## Пример: Использование свойства замыкания пересечения

- Мы доказали с использованием леммы о накачке, что  $L_1 = \{0^n 1^n \mid n \geq 0\}$  - не регулярный язык.
- $L_2$  = множество строк с равным числом 0 и 1 тоже не РЯ, но этот факт трудно доказать напрямую.
- Предположим, что  $L_2$  – регулярный язык.
- Регулярные языки замкнуты относительно  $\cap$ .
- Если  $L_2$  регулярный, то  $L_2 \cap L(0^* 1^*) = L_1$  тоже должен бы быть регулярным, но мы уже установили, что это не так.

# Замыкание по вычитанию

- **Теорема.** Если  $L$  и  $M$  – регулярные языки, то таким является язык  $L - M$  (строки из  $L$ , но не из  $M$ ).
- **Доказательство:** Пусть  $A$  и  $B$  - КДА, языками которых являются  $L$  и  $M$  соответственно.
- Построим автомат  $C$  – произведение автоматов  $A$  и  $B$ .
- Сделаем финальными состояниями автомата  $C$  пары, которые состоят из финальных состояний  $A$  и нефинальных  $B$ .
- Автомат  $C$  допускает точно строки из  $L - M$ .
- Следовательно,  $L - M$  – регулярный язык.

## Пример: КДА — произведение для разности языков



Финальное состояние  $[B,D]$  недостижимо из начального состояния, так что данная версия автомата-произведения принимает пустой язык. Это должно быть так, потому что первый автомат принимает подмножество того, что принимает второй.

То есть, первый автомат принимает все строки, заканчивающиеся нечетным числом 1, а второй принимает все строки, заканчивающиеся по крайней мере одной 1, плюс пустая строка.

# Замыкание по дополнению

- *Дополнением* языка  $L$  (по отношению алфавита  $\Sigma$ , так что  $\Sigma^*$  содержит  $L$ ) является язык  $\Sigma^* - L$ .
- Так как  $\Sigma^*$  очевидно регулярный и регулярные языки замкнуты по операции вычитания, дополнение регулярного языка всегда будет регулярным языком.

# Замыкание по обращению

- Пусть дан язык  $L$ , и  $L^R$  - есть множество строк, обращение которых есть в  $L$ .
- **Пример:**  $L = \{0, 01, 100\}$ ;  $L^R = \{0, 10, 001\}$ .
- Если  $L$  – регулярный язык, то  $L^R$  – тоже регулярный язык.
- **Доказательство:** Пусть  $E$  есть РВ для  $L$ . Мы покажем как обратить  $E$ , чтобы произвести РВ  $E^R$  для  $L^R$ .

# Обращение регулярного выражения

□ **Базис:** Если  $E$  символ  $a$ ,  $\epsilon$ , или  $\emptyset$ , то  $E^R = E$ .

□ **Индукция:** если  $E$  есть

□  $F+G$ , то  $E^R = F^R + G^R$ .

□  $FG$ , то  $E^R = G^R F^R$

□  $F^*$ , то  $E^R = (F^R)^*$ .

## Пример: Обращение РВ

□ Пусть  $E = \mathbf{01}^* + \mathbf{10}^*$ .

□  $E^R = (\mathbf{01}^* + \mathbf{10}^*)^R = (\mathbf{01}^*)^R + (\mathbf{10}^*)^R$

□  $= (\mathbf{1}^*)^R \mathbf{0}^R + (\mathbf{0}^*)^R \mathbf{1}^R$

□  $= (\mathbf{1}^R)^* \mathbf{0} + (\mathbf{0}^R)^* \mathbf{1}$

□  $= \mathbf{1}^* \mathbf{0} + \mathbf{0}^* \mathbf{1}.$

# Гомоморфизмы

- *Гомоморфизмом* на алфавите является функция, которая каждый символ этого алфавита заменяет некоторой строкой.
- *Пример*:  $h(0) = ab$ ;  $h(1) = \epsilon$ .
- Расширим функцию на строки  $h(a_1 \dots a_n) = h(a_1) \dots h(a_n)$ .
- *Пример*:  $h(01010) = ababab$ .



# Замыкание по гомоморфизму

- Если  $L$  — регулярный язык, и  $h$  есть гомоморфизм на его алфавите, то  $h(L) = \{h(w) \mid w \in L\}$  также является регулярным языком.
- Доказательство: Пусть  $E$  — РВ для  $L$ .
- Применим  $h$  к каждому символу в  $E$ .
- Язык для результирующего РВ есть  $h(L)$ .

## Пример: Замыкание по гомоморфизму

- Пусть  $h(0) = ab$ ;  $h(1) = \epsilon$ .
- Пусть  $L$  – язык для регулярного выражения  $\mathbf{01^* + 10^*}$ .
- Тогда  $h(L)$  будет регулярным языком для РВ

$$ab\epsilon^* + \epsilon(ab)^*.$$


**Замечание:** используем скобки, чтобы сделать правильное группирование

## Пример – продолжение

- $ab\epsilon^* + \epsilon(ab)^*$  можно упростить.
- $\epsilon^* = \epsilon$ , так что  $ab\epsilon^* = ab\epsilon$ .
- $\epsilon$  - есть единица по конкатенации.
  - То есть,  $\epsilon E = E\epsilon = E$  для любого РВ  $E$ .
- Т.о.,  $ab\epsilon + \epsilon(ab)^* = ab + (ab)^*$ .
- Наконец,  $L(ab)$  содержится в  $L((ab)^*)$ , так что РЕ для  $h(L)$  есть  $(ab)^*$ .

# Обратный гомоморфизм

- Пусть  $h$  — гомоморфизм и  $L$  есть язык, алфавитом которого является результирующий язык функции  $h$ .
- Определим:
- $h^{-1}(L) = \{w \mid h(w) \text{ есть в } L\}$ .

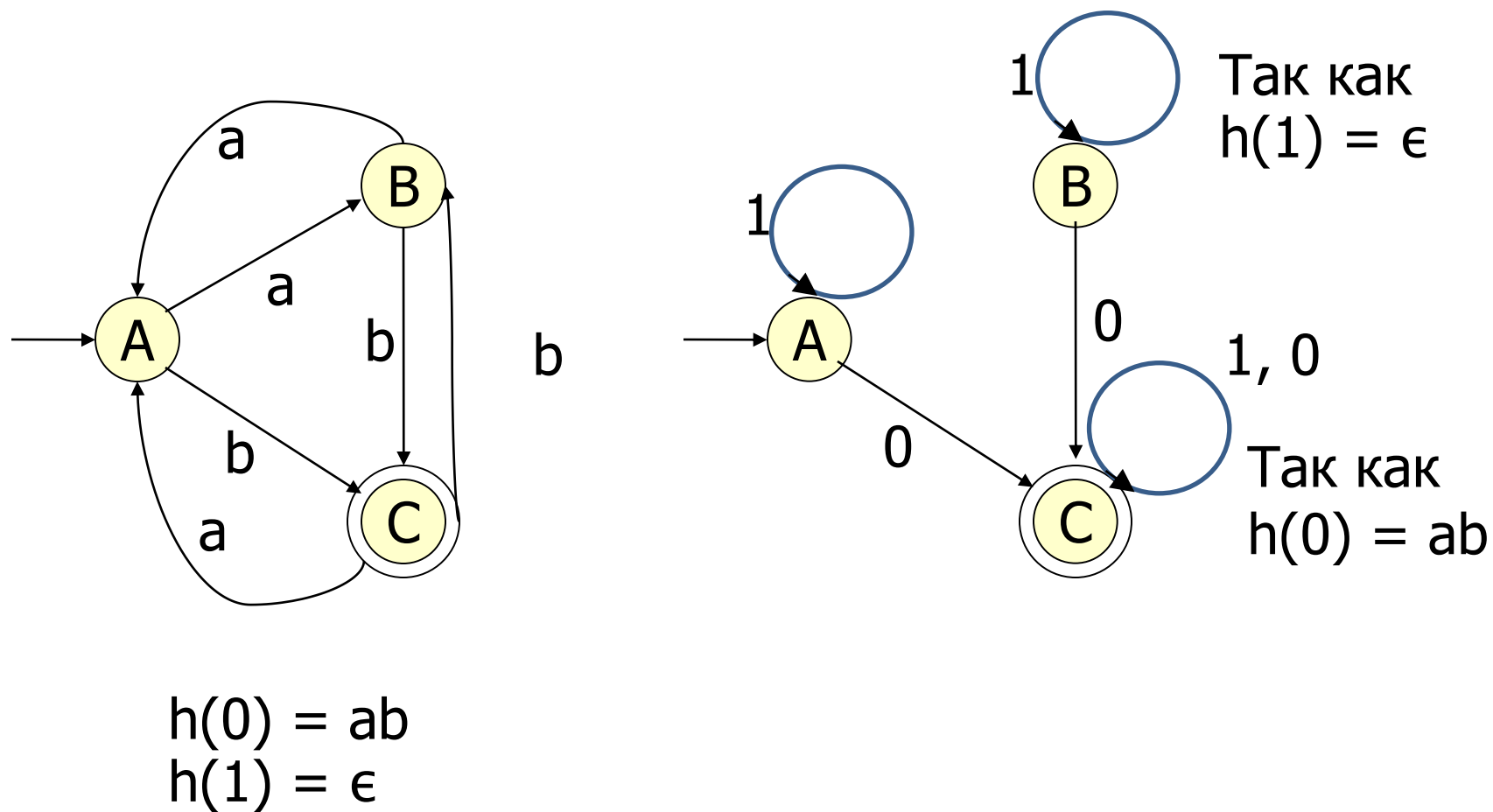
## Пример: обратный гомоморфизм

- Пусть  $h(0) = ab$ ;  $h(1) = \epsilon$ .
- Пусть  $L = \{abab, baba\}$ .
- $h^{-1}(L) =$  язык из 0 и любого числа 1  $= L(\mathbf{1^*01^*01^*})$ .

# Доказательство замыкания по обратному гомоморфизму

- Начнём с КДА  $A$  для  $L$ .
- Построим КДА  $B$  для  $h^{-1}(L)$  с:
  - Тем же самым множеством состояний, что и у  $A$ ,
  - Тем же начальным состоянием,
  - Теми же финальными состояниями.
  - Входной алфавит = символы, к которым применяется гомоморфизм  $h$ .
- Переходы для  $B$  вычисляются путём применения  $h$  к входному символу  $a$  и просмотра того, куда  $A$  мог бы пройти по последовательности символов  $h(a)$ .
- Формально,  $\delta_B(q, a) = \delta_A(q, h(a))$ .

# Пример: построение обратного гомоморфизма



# Доказательство – обратный гомоморфизм

□ Индукция по  $|w|$  показывает, что

$$\delta_B(q_0, w) = \delta_A(q_0, h(w)).$$

□ Таким образом,  $B$  принимает  $w$  т.и т.т., когда  $A$  принимает  $h(w)$ .

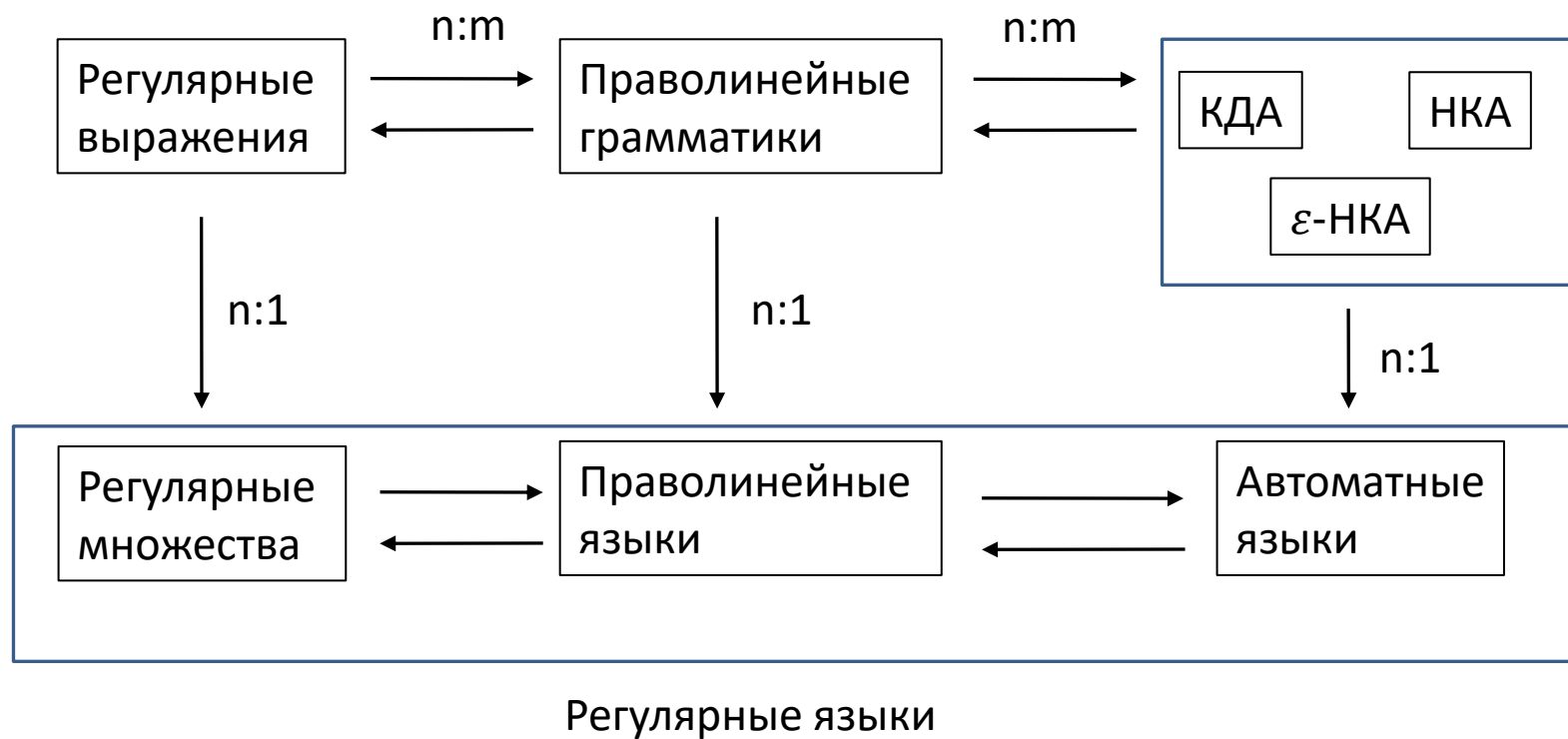


# Свойства замкнутости для регулярных языков

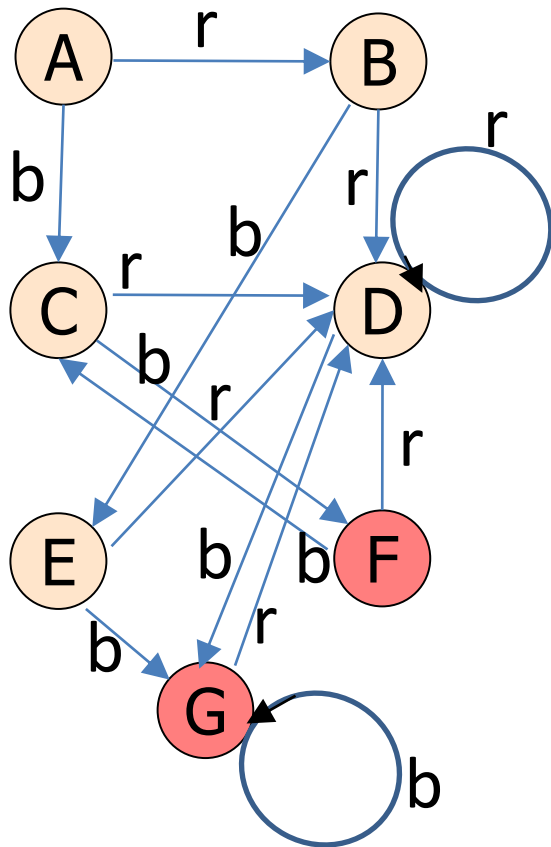
- Замкнутость класса регулярных языков относительно регулярных операций
- Замкнутость по дополнению
- Замкнутость по пересечению
- Замкнутость по вычитанию
- Замкнутость по обращению (слов)
- Замкнутость по гомоморфизму
- Замкнутость по обратному гомоморфизму

# Разрешимые свойства для регулярных языков

- Проблема принадлежности слова языку:  $w \in L$ ?
- Проблема пустоты:  $L = \emptyset$ ?
- Проблема эквивалентности:  $L = M$ ?
- Проблема вложения языков:  $L \subseteq M$ ?
- Проблема бесконечности языка:  $|L| = \infty$ ?



# Пример: минимизация числа состояний



→

|    | r | b |
|----|---|---|
| A  | B | C |
| B  | D | E |
| C  | D | F |
| D  | D | G |
| E  | D | G |
| *F | D | C |
| *G | D | G |

|   | G | F | E | D | C | B |
|---|---|---|---|---|---|---|
| A | x | x |   |   |   |   |
| B | x | x |   |   |   |   |
| C | x | x |   |   |   |   |
| D | x | x |   |   |   |   |
| E | x | x |   |   |   |   |
| F |   |   |   |   |   |   |
| G |   |   |   |   |   |   |

Начнём с пометки пар с одним из заключительных состояний F или G.

# Грамматика

# Языки

# Автоматы

